Advanced Robotics

Mihir Rao

Contents

1 Course Introduction **5**
1.1 What makes a robot autonomous? . . . 5
1.2 Three reusable mathematical ideas . . . 6
1.3 Driving: perception, prediction, and evaluation . . . 7
1.4 Flight, locomotion, and mapping . . . 8
1.5 Manipulation, demonstrations, and learned policies . . . 9
1.6 Reliability and the long tail . . . 9

2 Markov Decision Processes and Exact Methods **11**
2.1 Specifying the sequential decision problem . . . 11
2.2 Bellman recursion and value iteration . . . 11
2.3 Infinite-horizon convergence and error bounds . . . 13
2.4 Policy evaluation and policy iteration . . . 14
2.5 Linear programming and discounted occupancy . . . 15
2.6 Maximum-entropy decisions . . . 15

3 Discretizing Continuous-State MDPs **17**
3.1 Replacing an infinite state set by a finite model . . . 17
3.2 Nearest-neighbor and interpolated transitions . . . 17
3.3 Higher dimensions and Kuhn triangulation . . . 18
3.4 Acting between grid points . . . 19
3.5 Cross-entropy search for action sequences . . . 20
3.6 What approximation guarantees require . . . 21

4 Function Approximation and Feature Representations **22**
4.1 Representing values with reusable features . . . 22
4.2 Supervised fitting and generalization . . . 23
4.3 Fitted value iteration: where the targets come from . . . 24
4.4 Why approximation can destroy convergence . . . 25
4.5 Averagers and compatible norms . . . 26
4.6 Approximate linear programming . . . 26

5 LQR, Iterative LQR, and Differential Dynamic Programming **28**
5.1 A continuous problem that closes under Bellman backups . . . 28
5.2 Deriving the Riccati recursion . . . 28
5.3 Infinite horizon and the limits of local stabilization . . . 29
5.4 Extensions that preserve the useful structure . . . 30
5.5 Iterative LQR for nonlinear dynamics . . . 31
5.6 DDP retains curvature of the dynamics . . . 32
5.7 Replanning, bounds, and controllability . . . 32

6 Unconstrained Optimization **34**
6.1 Geometry, convexity, and stationary points . . . 34

6.2 Gradient descent and step selection . . . 34
6.3 Conditioning and coordinate scaling . . . 35
6.4 Newton's method and safeguards . . . 36
6.5 Gauss–Newton and natural gradient . . . 36
6.6 Momentum, RMSprop, and Adam . . . 37

7 Constrained Optimization **38**
7.1 Feasibility is part of the problem . . . 38
7.2 Exact penalties and sequential convex models . . . 38
7.3 Equality constraints and the null space . . . 39
7.4 Lagrange multipliers and KKT systems . . . 40
7.5 Log barriers and the central path . . . 41
7.6 Finding an interior point and interpreting dual updates . . . 42

8 Shooting, Collocation, and Model-Predictive Control **43**
8.1 Three independent modeling choices . . . 43
8.2 Shooting: eliminate the states by simulation . . . 43
8.3 Collocation: expose the temporal constraints . . . 44
8.4 Backpropagation through time . . . 45
8.5 Model-predictive control in operation . . . 46

9 Direct Collocation and Contact-Invariant Optimization **47**
9.1 Why optimize a trajectory directly? . . . 47
9.2 Inverse dynamics and local temporal structure . . . 47
9.3 Gauss–Newton exploits sparse residuals . . . 48
9.4 Why contact changes the search problem . . . 49
9.5 Contact-invariant optimization: introduce consistency variables . . . 49
9.6 From optimized motion clips to reusable policies . . . 50

10 Motion Planning in Configuration Space **51**
10.1 Geometry, dynamics, and configuration obstacles . . . 51
10.2 Local trajectory optimization and collision distances . . . 51
10.3 Probabilistic roadmaps for reusable connectivity . . . 52
10.4 RRT grows toward unexplored space . . . 53
10.5 RRT*, optimality, and path refinement . . . 54
10.6 Sampling positions and orientations correctly . . . 55

11 Probability, Bayes Filters, and Multivariate Gaussians **56**
11.1 Uncertainty is part of the state-estimation problem . . . 56
11.2 Bayes' rule and repeated evidence . . . 56
11.3 Actions predict; measurements correct . . . 57
11.4 Gaussian distributions and covariance geometry . . . 58
11.5 Marginalization and conditioning of a joint Gaussian . . . 59

12 Kalman Filtering, EKF, and UKF **60**
12.1 Linear Gaussian models preserve Gaussian beliefs . . . 60
12.2 Deriving the measurement update by conditioning . . . 60
12.3 Cross-covariance allows indirect state correction . . . 62
12.4 Extended Kalman filtering . . . 62
12.5 The unscented transform . . . 63
12.6 Observability, computation, and numerical limits . . . 64

13 Smoothing, MAP, Maximum Likelihood, and EM **66**
13.1 Filtering and smoothing answer different questions . . . 66
13.2 Kalman smoothing as a backward correction . . . 66

13.3 MAP trajectories and marginal estimates . . . 67
13.4 Maximum likelihood with observed data . . . 68
13.5 Priors, MAP parameters, and validation . . . 68
13.6 EM handles latent variables with soft information . . . 69
13.7 Mixture models and a complete soft-assignment example . . . 70
13.8 EM for state-space models . . . 70

14 Particle Filters and Monte Carlo Localization **72**
14.1 Represent a belief by weighted hypotheses . . . 72
14.2 Rejection sampling and importance sampling . . . 72
14.3 Deriving the bootstrap particle filter . . . 73
14.4 Systematic resampling in linear time . . . 74
14.5 Degeneracy, localization, and recovery . . . 74

15 POMDPs and Planning in Belief Space **76**
15.1 Actions must depend on what is known . . . 76
15.2 The belief MDP and contingent lookahead . . . 76
15.3 Worked example: the tiger problem . . . 77
15.4 Why most-likely-state control can miss sensing . . . 77
15.5 Gaussian belief-space planning . . . 78
15.6 Sensing discontinuities and absence of evidence . . . 79
15.7 Collision risk, sigma hulls, and continuous motion . . . 79
15.8 The separation principle and its boundary . . . 80

17 Imitation Learning **81**
17.1 Learning actions or learning intentions . . . 81
17.2 Behavioral cloning and distribution shift . . . 81
17.3 DAgger: ask for labels where the learner goes . . . 82
17.4 Inverse RL and feature expectations . . . 83
17.5 Apprenticeship learning and reward ambiguity . . . 83
17.6 Maximum-entropy inverse RL . . . 84
17.7 Demonstration interfaces and richer generalization . . . 85

18 Policy Gradients and Trust-Region Methods **86**
18.1 Optimize behavior when the dynamics are unknown . . . 86
18.2 Finite differences and common random numbers . . . 86
18.3 The likelihood-ratio derivation . . . 87
18.4 Reward-to-go and baselines reduce variance . . . 88
18.5 Critics, bootstrapping, and advantage estimates . . . 88
18.6 Why policy steps need special care . . . 89
18.7 TRPO and the importance-weighted surrogate . . . 90
18.8 PPO: a simpler clipped objective . . . 91

19 Off-Policy Reinforcement Learning **92**
19.1 Reuse transitions through Bellman structure . . . 92
19.2 Tabular Q-learning and its guarantees . . . 92
19.3 Function approximation and DQN stabilization . . . 93
19.4 Soft Q-learning and entropy-regularized values . . . 94
19.5 DDPG for continuous actions . . . 94
19.6 Soft actor-critic . . . 95

20 Model-Based Reinforcement Learning **97**
20.1 Learn a model, then use it for decisions . . . 97
20.2 Compounding error and model exploitation . . . 97
20.3 Ensembles and ME-TRPO . . . 98

20.4 MB-MPO: optimize for adaptation . . . 99
20.5 Asynchronous learning and the meaning of efficiency . . . 100
20.6 Learning models from images . . . 100

21 How Physics Simulators Work **102**
21.1 A simulator is a chain of approximations . . . 102
21.2 Lagrangian mechanics removes internal constraint forces . . . 102
21.3 Coupled and underactuated models . . . 103
21.4 Friction, drag, and robot description . . . 103
21.5 Numerical integration changes the dynamics . . . 104
21.6 Collision detection: reject cheaply, inspect precisely . . . 105
21.7 Impulses, unilateral contact, and time stepping . . . 105

22 Sim-to-Real Transfer and Domain Randomization **106**
22.1 Why simulated experience is attractive . . . 106
22.2 Small errors can compound through feedback . . . 106
22.3 System identification and identifiability . . . 107
22.4 Adaptation versus randomization . . . 108
22.5 Visual randomization and what the model should ignore . . . 108
22.6 Dynamics randomization and recurrent adaptation . . . 109
22.7 Structured variation and learning the simulator distribution . . . 110
22.8 Hard-case generation and automatic curricula . . . 110
22.9 Designing a meaningful transfer evaluation . . . 111

24 Autonomous Drones: Perception, Tracking, and Control **112**
24.1 A product must coordinate the whole robot . . . 112
24.2 Wide-angle vision and visual-inertial estimation . . . 112
24.3 Rolling shutter and vibration violate a single-pose image model . . . 113
24.4 Depth estimation serves different needs from odometry . . . 114
24.5 Failure cases that determine real reliability . . . 114
24.6 Moving objects and identity tracking . . . 115
24.7 Planning, fallback behavior, and experimental evidence . . . 116

25 How Robotics Research Ideas Develop **117**
25.1 Start from a bottleneck that matters . . . 117
25.2 From manipulation failures to stable policy optimization . . . 117
25.3 Inverse RL changes what the robot is asked to learn . . . 118
25.4 Structured prediction uses relationships between outputs . . . 119
25.5 InfoGAN: connect a missing property to an objective . . . 119
25.6 Guided policy search connects state-space teachers to vision . . . 120
25.7 Benchmarking makes competing claims comparable . . . 120
25.8 RL-squared and MAML target faster learning directly . . . 121
25.9 Shared infrastructure, opportunity cost, and changing direction . . . 122

26 Learning Autonomous Helicopter Maneuvers **123**
26.1 Why aggressive flight is a coupled learning problem . . . 123
26.2 Representing demonstrations as observations of a hidden trajectory . . . 123
26.3 Dynamic time warping aligns progress through a maneuver . . . 124
26.4 Smoothing extracts a coherent intended trajectory . . . 125
26.5 Trajectory-specific models exploit repeatable residuals . . . 125
26.6 Feedback control around the learned maneuver . . . 126
26.7 An iterative experimental workflow . . . 127

Lecture 1: Course Introduction

1.1 What makes a robot autonomous?

A robot must turn incomplete measurements into actions that change the physical world. Its hardware can be strong and precise while its behavior remains unreliable. A useful starting question is therefore: what must the software know, predict, and choose before the machine can act successfully?

- **State estimation.** Infer quantities such as position, velocity, orientation, and nearby obstacles from noisy sensors. The state is the information needed to predict what happens next, not simply the latest camera image.
- **Modeling.** Predict how the state changes when an action is applied. A model can come from mechanics, measured data, or a combination of both.
- **Planning and control.** Select a useful sequence of actions, then correct those actions as new measurements arrive. Planning looks ahead; feedback responds to what actually happens.
- **Learning.** Improve the model, representation, objective, or policy from experience. Learning is useful at several points in the loop, rather than being a single replacement for the whole system.

A policy maps available information to an action. A teleoperated robot receives those decisions from a person. Its successful demonstration establishes a hardware capability, but autonomous execution requires the machine to make the relevant decisions itself.

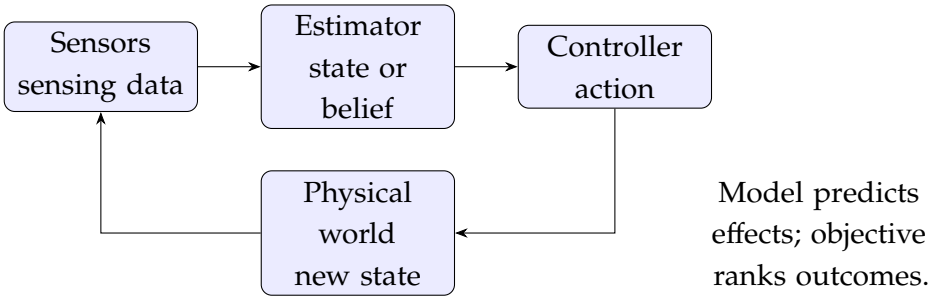


Figure 1: Autonomy closes the loop between sensing and physical action.

The arrows have different meanings. The estimator converts data into a description of the world. The controller converts that description into a decision. The world then changes, so the next decision must use new information.

1.2 Three reusable mathematical ideas

Very different robots share three foundations.

1. **Optimization asks what to choose.** A cost assigns numbers to undesirable outcomes, such as tracking error, energy use, or collision. Optimization searches for actions with low total cost while respecting physical restrictions.
2. **Probability asks what is known.** A distribution represents uncertainty about the robot and its surroundings. New evidence changes that distribution through conditioning.
3. **Learning asks what can improve from data.** Repeated trials can identify dynamics, imitate expert actions, or improve a policy using rewards. The resulting behavior still depends on what situations the training data cover.

For a discrete-time illustration, let x_t be the state, u_t the action, and y_t the measurement at time t. A model has the form

$$x_{t+1} = f(x_t, u_t, w_t), \qquad y_t = h(x_t, v_t),$$

where w_t and v_t represent disturbances and measurement noise. An objective might penalize distance from a target and control effort. These equations are a language for connecting the components; choosing them correctly is already part of the robotics problem.

1.2.1 Worked example: moving one step toward a target

Use an illustrative one-dimensional robot with $x_{t+1} = x_t + u_t$, current position $x_t = 2$, target 5, and cost

$$c(u) = (x_t + u - 5)^2 + u^2 = (u - 3)^2 + u^2.$$

1. **Separate the preferences.** The first term penalizes missing the target after one step. The second discourages a large action. Reaching the target immediately is therefore not automatically optimal.
2. **Optimize.** Expanding gives $c(u) = 2u^2 - 6u + 9$. Its derivative is $4u - 6$, so the unconstrained minimizer is $u = 1.5$, giving next position 3.5 and cost 4.5.
3. **Apply a physical restriction.** If $|u| \le 1$, the best feasible action is $u = 1$, with next position 3 and cost 5. The unconstrained answer cannot simply be executed.
4. **Close the loop.** If a disturbance leaves the robot at 2.8 instead, the next decision should start from 2.8. Replaying an old action sequence would ignore the observed error.

This small calculation separates the objective, model, constraint, and feedback. Confusing them makes larger systems difficult to diagnose.

1.3 Driving: perception, prediction, and evaluation

Autonomous driving combines localization, mapping, terrain recognition, object detection, prediction of other road users, and vehicle control. The DARPA desert and urban challenges made the difference between following a route and interacting with traffic especially clear. A road is not merely a geometric corridor: another vehicle can change the set of sensible actions.

- **LiDAR supplies geometry.** Reflected laser measurements help estimate distances and surfaces. Dark, transparent, reflective, or weather-obscured objects can produce difficult returns.
- **Cameras supply visual evidence.** Images help identify lanes, objects, and scene context, but lighting and occlusion change what is visible.
- **Multiple sensors supply complementary information.** Redundancy helps only to the extent that failure modes differ. Two sensors affected by the same condition need not fail independently.
- **Automation requires a defined responsibility.** Keeping a lane, handling a route under specified conditions, and handling every driving situation are different capabilities. Evaluation must state the conditions and who must respond when the system reaches its limit.

A disengagement occurs when a human takes over from an autonomous system. Miles per disengagement can describe a particular test, but comparisons require matched routes, conditions, and intervention rules. A larger number alone does not establish greater safety.

1.3.1 Worked example: why the testing distribution matters

Consider two invented test programs. System A drives 1,000 easy miles with one intervention. System B drives 100 difficult miles with two interventions.

1. The raw figures are 1,000 and 50 miles per intervention. That arithmetic is valid.
2. The conclusion that A handles difficult roads better does not follow: A has not been evaluated on the same roads.
3. A meaningful comparison separates scenario classes, measures performance in each, and weights them using the intended operating conditions. An overall average can hide weakness in a rare but consequential class.

The lesson is about evidence: a performance claim must match the population of situations actually tested.

1.4 Flight, locomotion, and mapping

1.4.1 Helicopters and feedback

A helicopter has coupled, unstable, nonlinear dynamics. A control change can affect several axes at once, and aerodynamic forces depend on the recent motion of the vehicle and surrounding air. Even a slow-looking maneuver near hover can be difficult because the vehicle interacts with its own disturbed airflow.

An aerobatic system must estimate the current state, learn a useful dynamics model, specify a dynamically plausible target trajectory, and apply feedback to track it. A maneuver is a sequence of related states, not an isolated desired pose. Later methods such as LQR and model-predictive control supply the mathematical tools for making local corrections along that sequence.

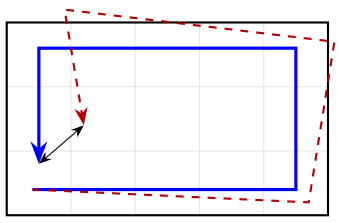
1.4.2 Legged locomotion and the distinction between model and reward

A model predicts the result of a foothold. A reward expresses whether the result is desirable. Improving one does not automatically fix the other. A planner rewarded only for forward progress may select a foothold that is fast under its model but fragile when the model is slightly wrong. Demonstrations can reveal a preference for stable configurations, which can then guide planning on new terrain.

For an illustrative choice, suppose a short route has predicted cost 3 but a 0.1 chance of a failure costing 50. Its expected cost is $3 + 0.1(50) = 8$. A stable route costing 6 is preferable under this model. If failure is omitted from the objective, the planner rationally chooses the wrong route for the real task.

1.4.3 Mapping and localization must support one another

Simultaneous localization and mapping, or SLAM, estimates a map and the robot's trajectory together. Wheel odometry alone accumulates errors from slip and imperfect calibration. Placing every laser scan at an erroneous pose creates a distorted map. Observations of previously seen structure can help correct the trajectory, which then improves the map.



Solid: physical loop
Dashed: drifting odometry

Recognizing a known place
constrains accumulated error.

Figure 2: A repeated place provides information about trajectory drift; the paths are schematic.

A loop closure is useful because the robot's observations indicate that two estimated positions correspond to the same place. It is a constraint on the trajectory, not permission to force every loop to be geometrically identical regardless of evidence.

1.5 Manipulation, demonstrations, and learned policies

Fetching an object combines navigation, recognition, grasp selection, and collision-free arm motion. Cloth and rope add shape variation: a useful action depends on the object's configuration, not only the robot's pose.

1. **Identify task-relevant structure.** Towel corners or rope segments give landmarks for describing the task. A twist can change which motion is appropriate.
2. **Express actions relative to that structure.** An action that succeeds at one absolute coordinate may miss an object placed elsewhere.
3. **Adapt and execute.** Aligning the demonstrated object shape with the current shape can transform a demonstrated motion into a new motion. Feedback and collision constraints still matter after the transformation.

For a simple illustrative rigid adaptation, a demonstrated grasp at $g = (1, 2)$ relative to an object translated by $d = (3, -1)$ becomes $g' = g + d = (4, 1)$. Rope adaptation is harder because different regions may move differently, requiring a nonrigid spatial transformation rather than one translation.

Guided policy search connects locally optimized behavior to a learned policy that uses observations. Policy-gradient methods improve policies from sampled returns. A tensegrity robot changes cable lengths to move a structure made of bars and cables. These examples differ physically but share estimation, optimization, and learning. Fleet learning adds data from multiple robots, which can improve a shared model if the collected experience and robot differences are handled appropriately.

1.6 Reliability and the long tail

A LONG TAIL consists of many individually uncommon situations. Their combined importance can be large. An object-picking system might work well on most rigid packages but encounter unfamiliar shapes, slippery materials, transparent objects, or fragile items. A suction cup can simplify grasping when the object population permits it; routing unsuitable objects elsewhere changes the task the robot must solve.

1.6.1 Worked example: many rare cases add up

Suppose, purely illustratively, that 100 disjoint difficult scenario types each occur with probability 0.001.

1. Each type is rare: one occurrence per thousand opportunities on average.
2. The probability of encountering some type in the collection is the sum, $100(0.001) = 0.1$, because the types are disjoint.
3. A policy that fails on every type succeeds on the common cases yet fails on 10% of opportunities. Reporting only its best demonstrations misses this weakness.

Training in simulation makes experiments repeatable and inexpensive, but a policy may exploit an inaccurate contact model, missing delay, or simulator bug. Transfer requires checking the assumptions that supported success. Physical experiments expose the difference between optimizing a mathematical task and reliably performing the intended task.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 1.

Lecture 2: Markov Decision Processes and Exact Methods

2.1 Specifying the sequential decision problem

A Markov decision process (MDP) describes an agent that observes its state, chooses an action, receives a reward, and moves to a new state. The Markov assumption says that the current state and action contain the information needed to predict the next state. A position-only description of a moving robot usually fails this assumption because velocity also affects its future.

- $\mathcal{S}$ and $\mathcal{A}(s)$ are the state set and available actions.
- $P_t(s' \mid s, a)$ is the next-state probability. It is nonnegative and sums to one over s'.
- $r_t(s, a, s')$ is the transition reward; $\bar{r}_t(s, a) = \sum_{s'} P_t(s' \mid s, a) r_t(s, a, s')$ is its expectation.
- H is the number of decisions in a finite horizon. The discount γ weights later rewards. Infinite-horizon contraction results below require $0 \le \gamma < 1$ and bounded rewards.

A policy $\pi_t(a \mid s)$ specifies a distribution over actions. A deterministic policy puts all probability on one action. The objective is

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{H-1} \gamma^t r_t(S_t, A_t, S_{t+1}) + \gamma^H g(S_H) \right],$$

where g is a terminal reward. A finite-horizon optimal policy can depend on time: an action useful with ten steps remaining may be pointless with one step remaining.

2.2 Bellman recursion and value iteration

Let $V_t(s)$ be the best expected future discounted reward starting at state s at time t, with discount measured relative to t. Set $V_H = g$. Splitting off the first transition gives

$$Q_t(s, a) = \sum_{s'} P_t(s' \mid s, a) \left[r_t(s, a, s') + \gamma V_{t+1}(s') \right],$$

$$V_t(s) = \max_a Q_t(s, a), \qquad \pi_t(s) \in \arg\max_a Q_t(s, a).$$

1. Fix a candidate first action. Average its immediate reward and continuation value over every possible next state.
2. At the next time, the state is observed, so the continuation can use the best action for that particular next state.

3. Compare the first-action values. Their maximum is the best value from the current state.

The second step explains why a policy is more expressive than a fixed action sequence. It can react differently to different outcomes.

Algorithm 2.1: Finite-horizon backward induction

Input: Transition model, rewards, horizon H, terminal reward g

$V_H(s) \leftarrow g(s)$ for every state
for $t = H-1, H-2, \ldots, 0$ **do**
  for each $s \in \mathcal{S}$ **do**
    $Q_t(s,a) \leftarrow \sum_{s'} P_t(s' \mid s,a)[r_t(s,a,s') + \gamma V_{t+1}(s')]$ for every a
    $V_t(s) \leftarrow \max_a Q_t(s,a)$; store a maximizing action $\pi_t(s)$
return V_0 and the time-indexed policy

At each backward step, V_{t+1} is already exact. Induction therefore proves correctness. A dense model costs $O(H|\mathcal{S}|^2|\mathcal{A}|)$ time. Values need two state-sized arrays if only V_0 is required, while storing every policy needs $O(H|\mathcal{S}|)$ action entries.

2.2.1 Worked example: a horizon changes the decision

Consider a robot in state A. It can *cash out*, receiving 2 and terminating, or *prepare*, receiving 0 and moving to B. In B it can receive 5 and terminate. Let $\gamma = 0.9$ and terminal values be zero.

1. With one action remaining, $V^{(1)}(A) = \max(2, 0) = 2$ and $V^{(1)}(B) = 5$. Preparing has no time to pay off.
2. With two actions remaining, preparing at A has value $0 + 0.9(5) = 4.5$. Cashing out remains worth 2, so $V^{(2)}(A) = 4.5$.
3. The correct policy at A changes with remaining time. A stationary action table would lose this distinction in a finite-horizon problem.

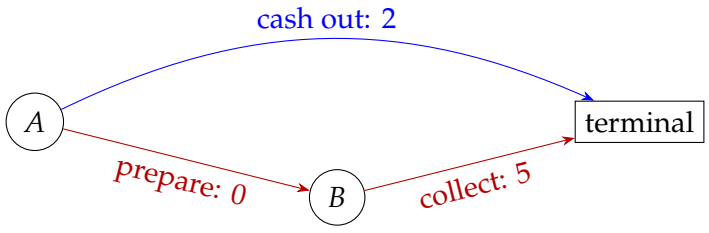


Figure 3: The action choice depends on how many decisions remain.

2.3 Infinite-horizon convergence and error bounds

For a time-independent discounted MDP, define the Bellman operator

$$(TV)(s) = \max_a \{\bar{r}(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s')\}.$$

Value iteration repeatedly applies $V_{k+1} = TV_k$. Its key property is contraction in the maximum norm $\|V\|_\infty = \max_s |V(s)|$.

Proof. For each fixed action, changing V to W changes its expected continuation by at most $\gamma\|V - W\|_\infty$, because the probabilities sum to one. Taking a maximum cannot increase the largest action-wise difference. Hence

$$\|TV - TW\|_\infty \le \gamma\|V - W\|_\infty.$$

Iterating shrinks differences geometrically. In the finite-dimensional value space this gives a unique fixed point V^* and convergence from any finite initialization. Bellman optimality identifies that fixed point as the optimal value. □

If $\delta = \|TV - V\|_\infty$, then

$$\|V - V^*\|_\infty \le \delta + \gamma\|V - V^*\|_\infty, \quad \text{so} \quad \|V - V^*\|_\infty \le \frac{\delta}{1-\gamma}.$$

A small update is meaningful only relative to $1 - \gamma$. At $\gamma = 0.99$, a residual of 0.01 permits value error as large as 1.

If $|r| \le R_{\max}$, omitting rewards after H steps introduces at most $\gamma^H R_{\max}/(1-\gamma)$ error. This follows by bounding each omitted reward and summing a geometric series. At $\gamma = 1$, neither this tail bound nor the contraction proof applies; termination or other additional assumptions are needed.

2.3.1 Noise and reward alter the preferred route

For an illustrative shortcut, suppose success gives 10, falling gives -20, and success probability is p. Its expected immediate reward is $10p - 20(1-p) = 30p - 20$. A safe alternative gives 6. The shortcut wins only if $p > 26/30 = 13/15$. A nominally shorter route can therefore be inferior once transition uncertainty is represented. Discounting introduces a separate preference for when the rewards arrive.

2.4 Policy evaluation and policy iteration

For a fixed stationary policy, let P_π be its transition matrix and r_π its expected reward vector. Then

$$V^\pi = r_\pi + \gamma P_\pi V^\pi, \qquad (I - \gamma P_\pi)V^\pi = r_\pi.$$

Solve this linear system, or repeatedly apply its evaluation update. There is no action maximum during evaluation because the policy is held fixed.

Policy iteration alternates exact evaluation with greedy improvement:

$$\pi_{k+1}(s) \in \arg\max_a \{\bar{r}(s,a) + \gamma P_a V^{\pi_k}\}.$$

Here $P_a V$ denotes the expected next-state value under action a at the current state.

1. The old action is one candidate, so the improved policy satisfies $T_{\pi_{k+1}} V^{\pi_k} \geq V^{\pi_k}$.
2. Applying its monotone evaluation operator repeatedly yields $V^{\pi_{k+1}} \geq V^{\pi_k}$.
3. With consistent tie handling, every actual policy change can be chosen to give a strict improvement somewhere. There are finitely many deterministic policies, so the procedure terminates.
4. If no action changes, $TV^\pi = V^\pi$. Uniqueness of the Bellman fixed point gives $V^\pi = V^*$.

Retain the current action when it is tied for best; otherwise cycling among equally good policies can obscure the termination test. Exact dense evaluation costs about $O(|\mathcal{S}|^3)$, while improvement costs one Bellman sweep. Modified policy iteration performs only a few evaluation sweeps between improvements.

2.4.1 Worked example: evaluate before improving

At a single nonterminal state, *wait* earns 1 and returns to that state; *leave* earns 3 and terminates. With $\gamma = 0.5$, the always-wait policy has $V = 1 + 0.5V$, hence $V = 2$. Improvement compares waiting, $1 + 0.5(2) = 2$, with leaving, 3. It switches to leaving. Re-evaluation gives $V = 3$, and waiting now scores 2.5, so the policy remains unchanged. Merely comparing immediate rewards would happen to work here, but evaluation is what justifies the policy-improvement decision in general.

2.5 Linear programming and discounted occupancy

Bellman optimality can also be expressed as

$$\begin{aligned} \min_{V} \quad & \sum_{s} \mu(s) V(s) \\ \text{subject to} \quad & V(s) \geq \bar{r}(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V(s') \quad \text{for every } s, a, \end{aligned}$$

where every $\mu(s) > 0$. The inequalities say that V upper-bounds every one-step action followed by continuation with V.

Proof. Any feasible V satisfies $V \geq TV$. Monotonicity gives $V \geq TV \geq T^2V \geq \cdots$, whose limit is V^*. Thus every feasible vector is componentwise at least V^*. The vector V^* itself is feasible, and positive objective weights choose it as the unique minimum. □

Reversing the inequality while still minimizing would not represent the same problem: arbitrarily low values could become attractive.

The dual variable $d(s,a) \geq 0$ has the meaning of discounted expected visits to a state-action pair from initial mass μ. Its flow equation is

$$\sum_{a} d(s,a) = \mu(s) + \gamma \sum_{z,a} P(s \mid z,a) d(z,a).$$

The dual maximizes $\sum_{s,a} d(s,a)\bar{r}(s,a)$. Incoming discounted visits plus initial mass equal outgoing visits. When μ sums to one, total occupancy is $1/(1-\gamma)$. At a visited state, recover a policy by normalizing $d(s,a)$ across actions. If the state has zero occupancy, that normalization does not define an action and a separate choice is needed.

2.6 Maximum-entropy decisions

A deterministic maximizer commits to a single action even when several actions are nearly as good. Entropy regularization rewards keeping options available. For action probabilities p_a, entropy is $\mathcal{H}(p) = -\sum_a p_a \log p_a$, using natural logarithms. Given action values q_a and temperature $\tau > 0$, solve

$$\max_{p \geq 0,\, \sum_a p_a = 1} \left\{ \sum_a p_a q_a - \tau \sum_a p_a \log p_a \right\}.$$

Introduce a multiplier λ for normalization. Differentiating the Lagrangian with respect to p_a gives

$$q_a - \tau(\log p_a + 1) + \lambda = 0.$$

Exponentiating and normalizing yields

$$p_a = \frac{e^{q_a/\tau}}{\sum_b e^{q_b/\tau}}, \qquad V_\tau = \tau \log \sum_a e^{q_a/\tau}.$$

Substitution verifies the value: $q_a - \tau \log p_a$ is the same normalizing constant for every action. In an MDP, compute $Q(s,a) = \bar{r}(s,a) + \gamma \mathbb{E}[V_\tau(S')]$ and use this soft maximum in each backup.

2.6.1 Worked example: temperature and stable computation

Let two action values be 2 and 1.

1. At $\tau = 1$, the higher-valued action has probability $1/(1+e^{-1}) \approx 0.731$, and $V_\tau = 2 + \log(1+e^{-1}) \approx 2.313$.
2. At $\tau = 0.1$, its probability is $1/(1+e^{-10}) \approx 0.99995$. Low temperature approaches a greedy choice.
3. Compute using $m = \max_a q_a$: $V_\tau = m + \tau \log \sum_a e^{(q_a - m)/\tau}$. Subtracting m avoids overflow without changing the probabilities.

For m actions, $\max q \le V_\tau \le \max q + \tau \log m$. The entropy bonus changes the objective, so a soft value exceeding the largest raw action value is expected. Randomization can provide alternative behaviors, but does not by itself guarantee robustness to every blocked route or model error.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 2.

Lecture 3: Discretizing Continuous-State MDPs

3.1 Replacing an infinite state set by a finite model

A robot's position and velocity are real-valued. Tabular value iteration cannot store a separate entry for every possible pair. Discretization chooses representative states $\xi_1, \dots, \xi_N$ and constructs a finite MDP on them. Its solution is exact for the constructed finite problem, while remaining an approximation to the original one.

For dynamics $X' = f(x, a, W)$, a typical construction has four decisions:

1. **Choose a domain.** State which region of position, velocity, angle, or other coordinates is represented. Decide what happens when a transition leaves it.
2. **Choose vertices and actions.** A regular grid is simple, but a nonuniform grid can devote resolution to important regions. Actions can also be discretized unless the backup can optimize them analytically.
3. **Distribute successors onto vertices.** Convert a continuous next state into one representative or a distribution over neighboring representatives.
4. **Solve and deploy.** Run a finite-state method, then define how the policy acts between the stored vertices.

With m values along each of d coordinates, there are m^d states. At $m = 20$, two coordinates need 400 entries, while six need 64 million. This exponential growth is the curse of dimensionality; faster code does not remove it.

3.2 Nearest-neighbor and interpolated transitions

3.2.1 Nearest-neighbor snapping

Let $q(x)$ be the nearest vertex to x. Define

$$\bar{P}(j \mid i, a) = \Pr(q(f(\xi_i, a, W)) = \xi_j).$$

For deterministic dynamics, one successor gets probability one. For stochastic dynamics, integrate over the regions assigned to vertices or estimate probabilities from sampled successors. Accumulated probabilities must sum to one; probability mass leaving the domain must be assigned explicitly rather than silently discarded.

Snapping is equivalent to evaluating a piecewise constant value function. It is inexpensive but creates abrupt changes at cell boundaries. A physically small motion can disappear if both the starting and next state snap to the same vertex.

3.2.2 Convex interpolation

Instead choose nonnegative weights $w_j(x')$ that sum to one and define

$$\hat{V}(x') = \sum_j w_j(x') V(\xi_j), \qquad \bar{P}(j \mid i, a) = \mathbb{E}_W[w_j(f(\xi_i, a, W))].$$

The same expression has two interpretations: interpolation of a value function and a stochastic transition among vertices. These artificial transition probabilities represent numerical approximation, even when the physical dynamics are deterministic.

On an interval $[\xi_i, \xi_{i+1}]$ of width h, let $\alpha = (x' - \xi_i)/h$. Then the weights are $1 - \alpha$ and α. They preserve affine functions exactly because $(1 - \alpha)\xi_i + \alpha\xi_{i+1} = x'$.

3.2.3 Worked example: snapping loses a useful movement

Let the grid be $0, 1, 2$, a successor be $x' = 0.4$, and stored values be $V(0) = 0$, $V(1) = 10$, $V(2) = 12$.

1. Nearest-neighbor evaluation gives $\hat{V}(0.4) = V(0) = 0$.
2. Linear interpolation uses weights 0.6 and 0.4, giving $\hat{V}(0.4) = 0.6(0) + 0.4(10) = 4$.
3. With immediate reward -1 and $\gamma = 0.9$, the action scores -1 under snapping and $-1 + 0.9(4) = 2.6$ under interpolation.
4. This difference is numerical, not a change in physical dynamics. Refining the grid or choosing a better interpolation can change the resulting action.

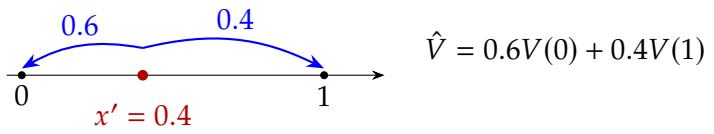


Figure 4: A continuous successor distributes its value between neighboring vertices.

3.3 Higher dimensions and Kuhn triangulation

In a two-dimensional unit cell, bilinear interpolation at (x, y) uses weights $(1 - x)(1 - y)$, $x(1 - y)$, $(1 - x)y$, and xy at the four corners. The weights are nonnegative inside the cell and sum to one. In d dimensions, multilinear interpolation may involve 2^d corners.

A SIMPLEX in d dimensions has $d+1$ vertices. Triangulating each grid cell allows piecewise affine interpolation with only $d+1$ weights. Kuhn triangulation finds a simplex by sorting the normalized coordinates.

1. In a two-dimensional cell, suppose $x \geq y$. Use vertices $(0,0), (1,0), (1,1)$.
2. Solve $(x, y) = w_0(0,0) + w_1(1,0) + w_2(1,1)$ with $\sum_i w_i = 1$.
3. The second coordinate gives $w_2 = y$, the first gives $w_1 = x - y$, and normalization gives $w_0 = 1 - x$.

For an illustrative point $(0.7, 0.2)$, the weights are $0.3, 0.5, 0.2$. They are nonnegative because $1 \geq x \geq y \geq 0$. If $y > x$, use the other triangle. The sorting construction generalizes to higher dimensions and avoids solving a new arbitrary geometric problem at every query.

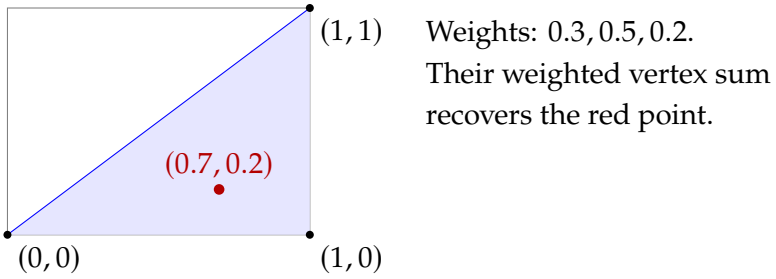


Figure 5: Simplex interpolation uses three vertices inside a two-dimensional cell.

3.4 Acting between grid points

A stored vertex policy does not yet define a controller on the physical state space. Three approaches make different tradeoffs.

- **No lookahead.** Use the nearest vertex's action, sample neighboring actions according to interpolation weights, or interpolate numerical controls. Averaging controls requires a meaningful continuous action representation; averaging labels such as "left" and "right" does not define a new valid label.
- **One-step lookahead.** At the actual state x, compute $\arg\max_a \mathbb{E}[r(x, a, X') + \gamma \hat{V}(X')]$. This uses the continuous dynamics before applying the approximate terminal value.
- **Longer lookahead.** Optimize a sequence of h actions, accumulating their rewards and ending with $\gamma^h \hat{V}(X_h)$. Execute the first action and replan when new information arrives.

For stochastic dynamics, a fixed action sequence is an open-loop approximation; a fully adaptive lookahead would branch on future observations. Even a short model-based lookahead can improve over simply copying a nearby action, especially near a decision boundary.

Mountain car illustrates why a locally backward movement can help a long-term objective. A weak engine may need to move away from the goal to build energy. A coarse grid that erases small velocity changes can miss this strategy. Position and velocity must both be represented, and the domain boundaries must reflect the actual system.

3.5 Cross-entropy search for action sequences

When sequence enumeration costs $|\mathcal{A}|^h$ and derivatives are unavailable, the CROSS-ENTROPY METHOD (CEM) provides a sampling-based optimizer. It maintains a distribution over candidate action sequences rather than a single sequence.

1. Initialize a mean μ and covariance Σ over the concatenated action vector $U = (u_0, \ldots, u_{h-1})$.
2. Sample M candidates, simulate each, and evaluate the same objective for each.
3. Retain the K best candidates, called elites. Replace the mean by their average and optionally replace the covariance by their empirical covariance.
4. Repeat for a fixed budget or until the distribution stops changing. Retain the best feasible candidate encountered.

The simulation cost is $O(IMh)$ model steps for I iterations, ignoring state dimension. This is an approximate search, not a proof of global optimality. A covariance floor helps prevent premature collapse. Bounds can be handled by a valid bounded sampling scheme, with the consequences of clipping understood.

3.5.1 Worked example: one CEM update

Optimize an illustrative scalar action with reward $F(u) = -(u-2)^2$. Suppose samples are $-1, 0, 1, 2, 3$ and two elites are retained.

1. Their scores are $-9, -4, -1, 0, -1$. The best action is 2; the second is tied between 1 and 3.
2. If the tie rule retains 1, the elite mean is $(2 + 1)/2 = 1.5$ and the maximum-likelihood variance is $[(2 - 1.5)^2 + (1 - 1.5)^2]/2 = 0.25$.
3. The next distribution concentrates near the good region. It does not establish that 1.5 is better than the best sampled action 2; keep that best candidate separately.

For discrete action sequences, fit a categorical probability at each time using elite action frequencies. Smoothing prevents an action from becoming impossible solely because it was absent from a small elite set.

3.6 What approximation guarantees require

Suppose the numerical Bellman operator $\tilde{T}$ is a γ-contraction and its error at the true value is at most ε:

$$\|\tilde{T}V^* - TV^*\|_\infty \le \varepsilon.$$

If $\tilde{V} = \tilde{T}\tilde{V}$, add and subtract $\tilde{T}V^*$ to obtain

$$\|\tilde{V} - V^*\|_\infty \le \gamma\|\tilde{V} - V^*\|_\infty + \varepsilon, \quad \|\tilde{V} - V^*\|_\infty \le \frac{\varepsilon}{1-\gamma}.$$

Smoothness assumptions on dynamics and rewards can make ε decrease with grid width. Without such assumptions, refinement alone does not provide a particular rate.

If an approximate value satisfies $\|\hat{V} - V^*\|_\infty \le \eta$ and π is exactly greedy with respect to it in the original MDP, then

$$\|V^* - V^\pi\|_\infty \le \frac{2\gamma\eta}{1-\gamma}.$$

To see the factor two, compare the approximate scores of the optimal action and the chosen action. Each can differ from its true continuation score by at most $\gamma\eta$, so the one-step disadvantage is at most $2\gamma\eta$. Repeating that disadvantage over future decisions adds a geometric factor $1/(1-\gamma)$. Approximate action search introduces another error term and must not be ignored.

3.6.1 Time resolution and artificial self-loops

For continuous-time dynamics $\dot{x} = f(x,u)$, a step Δt produces approximately $x' = x + \Delta t f(x,u)$. If this displacement is much smaller than the cell width, nearest-neighbor snapping can send every action back to the same vertex. The discretized robot then appears unable to move.

State and time resolution must therefore be chosen together. With continuous discount rate ρ, a transition lasting Δt uses discount $e^{-\rho\Delta t}$, and its stage reward approximates the reward accumulated over that interval. Changing time steps without changing rewards and discounts changes the optimization problem. Adaptive resolution can improve efficiency, but it must preserve these temporal meanings.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 3.

Lecture 4: Function Approximation and Feature Representations

4.1 Representing values with reusable features

A table stores one unrelated number per state. A function approximator shares parameters across states, allowing information from one state to influence another. For linear approximation,

$$\hat{V}_\theta(s) = \theta^\top \phi(s),$$

where $\phi(s) \in \mathbb{R}^d$ is the feature vector and $\theta \in \mathbb{R}^d$ contains adjustable weights. Linear refers to the parameters, not necessarily to the state: features can include squares, indicators, and nonlinear measurements.

Tetris provides a useful example because the complete board space is enormous while several geometric properties are easy to compute. Features can include each column's height, differences between neighboring heights, maximum height, the number of holes, and a constant. They encode hypotheses about which distinctions matter for future success.

- **Height** measures how close a column is to overflowing.
- **Height differences** describe an uneven surface that may be difficult to fill.
- **Holes** identify empty cells trapped below occupied cells.
- **A constant feature** allows an overall offset rather than forcing value zero whenever all other features vanish.

Two different boards can have identical features. No choice of weights can distinguish them under this representation. Better optimization cannot recover information that the features discard.

4.1.1 Worked example: compute a board value

For an illustrative four-column board, use heights $(2, 2, 4, 0)$, adjacent differences $(0, 2, 4)$, maximum height 4, no holes, and a constant. Thus

$$\phi = (2, 2, 4, 0,\ 0, 2, 4,\ 4, 0, 1).$$

With weights $\theta = (-1, -1, -1, -1, -2, -2, -2, -3, -2, 10)$:

1. Height contributions sum to $-2 - 2 - 4 + 0 = -8$.
2. Unevenness contributes $-2(0 + 2 + 4) = -12$.

3. Maximum height contributes $-3(4) = -12$, holes contribute 0, and the constant contributes 10.
4. The total is $-8 - 12 - 12 + 10 = -22$. The value is a learned score, not a probability, so negative values are valid.

A small feature change can affect several terms at once. Increasing one column can change its height, adjacent differences, maximum height, and holes; the model must be evaluated on the resulting complete feature vector.

4.2 Supervised fitting and generalization

Suppose training states s_i have target values y_i. Put their feature vectors into the rows of a matrix Φ. Least squares minimizes

$$L(\theta) = \frac{1}{2}\|\Phi\theta - y\|_2^2, \qquad \nabla L = \Phi^\top(\Phi\theta - y).$$

A minimizer satisfies $\Phi^\top\Phi\theta = \Phi^\top y$. In computation, QR or SVD is preferable to explicitly forming an inverse. Rank deficiency means several weight vectors can produce the same fitted values; it does not imply that the targets are exactly representable.

Ridge regularization adds $\lambda\|\theta\|^2/2$ and gives normal equations $(\Phi^\top\Phi + \lambda I)\theta = \Phi^\top y$. Feature scaling matters because a fixed weight penalty has different effects for differently scaled features.

4.2.1 Worked example: one regression fit

Use $\phi(s) = (1, s)$ and targets $(s, y) = (0, 1), (1, 2), (2, 2)$.

1. The design matrix has rows $(1, 0), (1, 1), (1, 2)$. Therefore

$$\Phi^\top\Phi = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}, \qquad \Phi^\top y = \begin{pmatrix} 5 \\ 6 \end{pmatrix}.$$

2. Solve $3\theta_0 + 3\theta_1 = 5$ and $3\theta_0 + 5\theta_1 = 6$. Subtraction gives $\theta_1 = 1/2$, followed by $\theta_0 = 7/6$.
3. Predictions are $7/6, 5/3, 13/6$. Residuals are $1/6, -1/3, 1/6$, whose squares sum to $1/6$.
4. The nonzero residual is unavoidable for this linear family: the three targets do not lie on one line. Adding training iterations cannot remove a representation error.

A neural network replaces fixed features with a composition of learned transformations, for example $h = \sigma(Ws + b)$ and $\hat{V} = w^\top h + c$. ReLU uses $\sigma(z) = \max(0, z)$; sigmoid and hyperbolic tangent are other choices. Nonlinearity is essential: composing affine layers without it still produces one affine map.

Universal approximation results concern the existence of a sufficiently large network for continuous functions on compact domains under suitable activation assumptions. They do not guarantee that gradient descent finds the weights, that finite data identify the function, or that extrapolation is accurate. Validation data, regularization, reduced model size, and early stopping address different aspects of overfitting.

4.3 Fitted value iteration: where the targets come from

In dynamic programming, supervised targets are generated by a Bellman backup rather than supplied as ground truth. At iteration k:

1. Choose representative training states $s_1, \ldots, s_M$.
2. Freeze the current parameters θ_k and form
$$y_i = \max_a \mathbb{E}[r(s_i, a, S') + \gamma \hat{V}_{\theta_k}(S')].$$
3. Fit new parameters to (s_i, y_i) by supervised learning.
4. Repeat, monitoring both fitting behavior and the quality of the resulting decisions.

Freezing the old parameters while constructing targets makes this a well-defined sequence of regressions. Differentiating through both prediction and target instead would optimize a different objective.

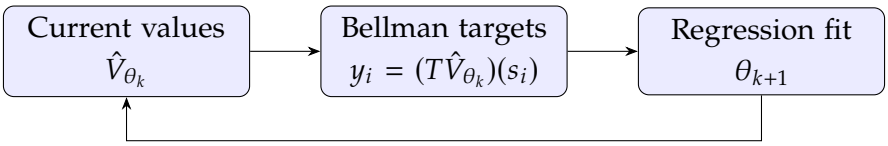


Figure 6: Fitted value iteration alternates target generation and approximation.

The feedback arrow is also the source of difficulty: approximation errors change later targets. Good supervised training loss at one iteration is not a convergence guarantee for the entire loop.

4.3.1 Worked example: backup first, then fit

Consider two training states with scalar coordinates 0 and 1. At 0, one action terminates with reward 1, while another earns 0 and moves to 1. At 1, the only action terminates with reward 2. Let $\gamma = 0.9$ and initial approximation $\hat{V}_0(s) = 0$.

1. First targets are $y_0 = \max(1, 0) = 1$ and $y_1 = 2$.
2. With $\phi = (1, s)$, the exact first fit is $\hat{V}_1(s) = 1 + s$.

3. New targets are $y_0 = \max(1, 0.9 \cdot 2) = 1.8$ and $y_1 = 2$.
4. The second fit is $\hat{V}_2(s) = 1.8 + 0.2s$. The preferred action at 0 now moves to 1, because the delayed payoff has propagated backward.

If the approximator allowed only a constant, it could not retain both target values. That coupling can help generalization, but can also distort the backup sequence.

4.4 Why approximation can destroy convergence

Let Π denote the fitting operator on value vectors. Idealized approximate value iteration is $V_{k+1} = \Pi T V_k$. Although T contracts the maximum norm, Π need not be nonexpansive in that norm.

4.4.1 Worked counterexample: zero rewards and growing values

Take two states, both deterministically transitioning to state 2, with zero reward. Represent values by $V_\theta = (\theta, 2\theta)$ and fit equally weighted least squares over both states. Set $\gamma = 0.9$.

1. The Bellman target at either state is $0.9(2\theta) = 1.8\theta$.
2. For a new weight z, minimize $(z - 1.8\theta)^2 + (2z - 1.8\theta)^2$.
3. Differentiating gives $2(z - 1.8\theta) + 4(2z - 1.8\theta) = 0$, hence $10z = 10.8\theta$ and $z = 1.08\theta$.
4. Starting at $\theta = 1$, the sequence is $1, 1.08, 1.1664, \ldots$, diverging even though the true value is zero and is representable.

This is a failure of the combined backup-and-projection operation. More accurate regression does not fix it; the regression is already exact for its stated objective.

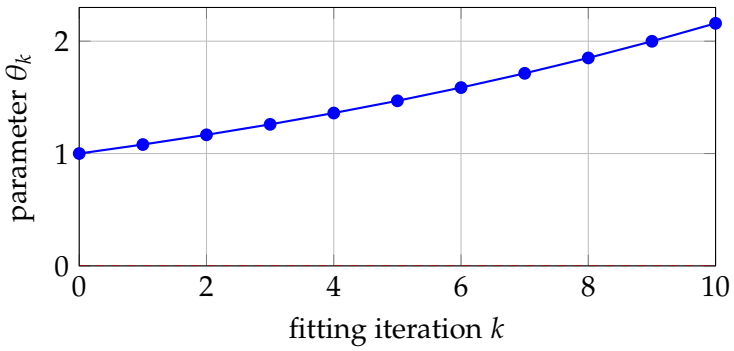


Figure 7: Exact fitting can diverge under repeated backups even when the true parameter is zero.

4.5 Averagers and compatible norms

An AVERAGER predicts a value by a convex combination of stored values, with fixed nonnegative weights summing to one. If A is such an operator,

$$|(AV)(s) - (AW)(s)| \le \sum_j w_j(s)|V_j - W_j| \le \|V - W\|_\infty.$$

Thus A is nonexpansive in maximum norm, and AT is a γ-contraction. Nearest-neighbor aggregation and simplex interpolation are examples. Negative interpolation weights can invalidate this argument.

If $\bar{V} = AT\bar{V}$, then

$$\begin{aligned}\|\bar{V} - V^*\|_\infty &\le \|AT\bar{V} - ATV^*\|_\infty + \|AV^* - V^*\|_\infty \\ &\le \gamma\|\bar{V} - V^*\|_\infty + \|AV^* - V^*\|_\infty.\end{aligned}$$

Therefore the fixed-point error is bounded by the representation error divided by $1 - \gamma$. This distinguishes convergence from accuracy: a stable method can converge to a poor approximation.

For fixed-policy evaluation, a different compatible norm is available. If d is a stationary state distribution for P_π, Jensen's inequality gives

$$\|P_\pi v\|_d^2 = \sum_s d(s)\left(\sum_{s'} P_\pi(s,s')v(s')\right)^2 \le \sum_{s'} d(s')v(s')^2,$$

where $\|v\|_d^2 = \sum_s d(s)v(s)^2$. Weighted orthogonal projection onto a linear feature space is nonexpansive in this same norm. The projected policy-evaluation operator is consequently a contraction on the supported states when $\gamma < 1$. Changing the sampling weights or adding a control maximum can remove the shared-norm argument; a guarantee for policy evaluation is not automatically a guarantee for control.

4.6 Approximate linear programming

Another approach inserts $V = \Phi\theta$ into Bellman inequalities:

$$\begin{aligned}\min_\theta \quad & \sum_s \mu(s)\phi(s)^\top\theta \\ \text{subject to} \quad & \phi(s)^\top\theta \ge \bar{r}(s,a) + \gamma\mathbb{E}[\phi(S')^\top\theta] \quad \text{for all retained } s,a.\end{aligned}$$

This is a linear program in the weights even when the features are nonlinear functions of state. It avoids an unstable fitted-iteration loop, but feasibility, constraint coverage, and approximation quality still matter.

With all state-action constraints, positive state weights, and a constant feature, a useful bound is

$$\|V^* - \Phi\theta_{\text{ALP}}\|_{1,\mu} \leq \frac{2}{1-\gamma} \min_{\theta} \|V^* - \Phi\theta\|_\infty,$$

for normalized μ, where the left norm is a weighted absolute error. The role of the constant feature can be seen constructively. If a candidate has maximum error η, its Bellman violation is at most $(1+\gamma)\eta$. Adding a constant $c = (1+\gamma)\eta/(1-\gamma)$ increases $V - TV$ by $(1-\gamma)c$, making the shifted candidate feasible. Its distance above V^* is at most $\eta + c = 2\eta/(1-\gamma)$, and the LP chooses no worse a weighted objective.

Sampling constraints makes the program smaller, but constraints omitted from the program need not hold. The solver's successful termination establishes optimality for the sampled linear program, not accuracy everywhere in the original continuous robot problem.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 4.

Lecture 5: LQR, Iterative LQR, and Differential Dynamic Programming

5.1 A continuous problem that closes under Bellman backups

The linear quadratic regulator (LQR) is an optimal controller for linear dynamics and quadratic cost. It avoids a state grid because the value function belongs to a family that remains unchanged under a Bellman backup: quadratics become quadratics.

Use a finite horizon with state $x_t \in \mathbb{R}^n$, control $u_t \in \mathbb{R}^m$, dynamics $x_{t+1} = A_t x_t + B_t u_t$, and cost

$$J = x_H^\top Q_H x_H + \sum_{t=0}^{H-1} (x_t^\top Q_t x_t + u_t^\top R_t u_t).$$

Assume $Q_t \succeq 0$ and $R_t \succ 0$. The notation $M \succeq 0$ means $z^\top M z \geq 0$ for every z; positive definiteness uses a strict inequality for every nonzero z. The state is fully observed, controls are unconstrained, and the dynamics and costs are known.

5.2 Deriving the Riccati recursion

Set $V_H(x) = x^\top P_H x$ with $P_H = Q_H$. Suppose $V_{t+1}(x) = x^\top P_{t+1} x$. The backup is

$$V_t(x) = \min_u \{x^\top Q_t x + u^\top R_t u + (A_t x + B_t u)^\top P_{t+1} (A_t x + B_t u)\}.$$

1. **Collect control terms.** Define $G_t = R_t + B_t^\top P_{t+1} B_t$ and $F_t = B_t^\top P_{t+1} A_t$. The expression becomes

$$x^\top (Q_t + A_t^\top P_{t+1} A_t) x + 2u^\top F_t x + u^\top G_t u.$$

2. **Minimize the quadratic.** Its control gradient is $2G_t u + 2F_t x$. Since $G_t \succ 0$, the unique minimum is $u = -G_t^{-1} F_t x = -K_t x$.

3. **Substitute the minimum.** Completing the square gives

$$P_t = Q_t + A_t^\top P_{t+1} A_t - A_t^\top P_{t+1} B_t G_t^{-1} B_t^\top P_{t+1} A_t.$$

4. **Store the controller.** Compute backward from H to 0, then execute $u_t = -K_t x_t$ using the current measured state.

In code, solve $G_t K_t = F_t$ rather than forming an inverse. With comparable state and control dimensions, the backward pass costs $O(Hn^3)$ and storing all gains costs $O(Hmn)$. This dependence is polynomial in dimension, unlike a grid.

5.2.1 Worked example: two-step scalar LQR

Let $x_{t+1} = x_t + u_t$, $Q_0 = Q_1 = R_0 = R_1 = Q_2 = 1$, and $x_0 = 2$.

1. Terminal $P_2 = 1$. At $t = 1$, $G_1 = 2$, $K_1 = 1/2$, and $P_1 = 1 + 1 - 1/2 = 3/2$.
2. At $t = 0$, $G_0 = 1 + 3/2 = 5/2$, $K_0 = (3/2)/(5/2) = 3/5$, and $P_0 = 1 + 3/2 - (3/2)^2/(5/2) = 8/5$.
3. Execute $u_0 = -(3/5)2 = -6/5$, giving $x_1 = 4/5$. Then $u_1 = -2/5$ and $x_2 = 2/5$.
4. Total cost is $4 + 36/25 + 16/25 + 4/25 + 4/25 = 32/5$, matching $P_0 x_0^2 = (8/5)4$.

The recursion begins with $P_2 = 1$. The rows follow the backward pass from $t = 1$ to $t = 0$; each computed P_t becomes the next row's P_{t+1}.

Time t	P_{t+1}	G_t	K_t	P_t
1	1	2	1/2	3/2
0	3/2	5/2	3/5	8/5

The final state is not zero because forcing it there immediately would cost more control effort. Changing R changes this balance.

5.3 Infinite horizon and the limits of local stabilization

For constant matrices, the backward recursion may converge to an algebraic Riccati solution. Standard sufficient assumptions are stabilizability of (A, B), detectability of $(Q^{1/2}, A)$, $Q \succeq 0$, and $R \succ 0$. Stabilizability allows uncontrollable modes if they are already stable. Detectability rules out unstable modes that the state cost cannot see. Controllability is sufficient for stabilizability, but stronger than necessary.

The closed-loop system is $x_{t+1} = (A - BK)x_t$. Discrete-time asymptotic stability requires its eigenvalues to lie strictly inside the unit circle. A controller designed by linearizing a nonlinear system is generally only locally valid. It does not certify recovery from every possible state or respect actuator limits that were omitted from the design.

For example, a cart-pole's state includes cart position, pole angle, and both velocities. Linearization near upright can stabilize small deviations, while a downward pole may require a swing-up strategy first. Applying the local gain everywhere confuses local stabilization with global planning.

5.4 Extensions that preserve the useful structure

5.4.1 Affine dynamics and tracking

An affine system $x_{t+1} = A_t x_t + B_t u_t + c_t$ leads to a value function $x^\top P_t x + 2p_t^\top x + d_t$ and an affine controller $u = -K_t x - k_t$. Alternatively augment the state by a constant coordinate $z = (x, 1)$ so the dynamics become linear in z. The last coordinate remains one; it is not an independently controllable state.

For a target trajectory (x_t^*, u_t^*) satisfying $x_{t+1}^* = f(x_t^*, u_t^*)$, define deviations $\delta x_t = x_t - x_t^*$ and $\delta u_t = u_t - u_t^*$. Linearization gives

$$\delta x_{t+1} \approx A_t \delta x_t + B_t \delta u_t, \quad A_t = f_x(x_t^*, u_t^*), \quad B_t = f_u(x_t^*, u_t^*).$$

Time-varying LQR produces $u_t = u_t^* - K_t(x_t - x_t^*)$. If the target is dynamically infeasible, a defect $f(x_t^*, u_t^*) - x_{t+1}^*$ remains and must be retained as an affine term.

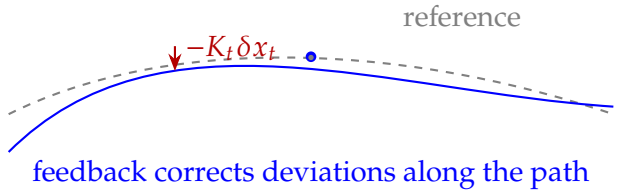


Figure 8: Trajectory tracking adds a local correction to the reference control.

5.4.2 Additive and multiplicative noise

For $x' = Ax + Bu + w$ with independent zero-mean additive noise of covariance W,

$$\mathbb{E}[x'^\top P x'] = (Ax + Bu)^\top P (Ax + Bu) + \operatorname{tr}(PW).$$

The cross term vanishes because $\mathbb{E}[w] = 0$. The trace term changes the value by a constant, so the optimal gain is unchanged in this fully observed, unconstrained quadratic setting. This does not say that noise never affects optimal control.

If noise multiplies the control, $x' = Ax + (B + B_w w)u$ with scalar w of variance σ^2, the expected cost contains $\sigma^2 u^\top B_w^\top P B_w u$. It depends on the action and increases its effective penalty. The additive-noise argument no longer applies.

5.4.3 Penalizing changes in control

A cost on $\Delta u_t = u_t - u_{t-1}$ discourages rapid switching. Choose $z_t = (x_t, u_{t-1})$ as state and $v_t = \Delta u_t$ as control. Then

$$z_{t+1} = \begin{pmatrix} A & B \\ 0 & I \end{pmatrix} z_t + \begin{pmatrix} B \\ I \end{pmatrix} v_t.$$

The original control is recovered as $u_t = u_{t-1} + v_t$. A penalty on u_t may introduce state-control cross terms in the augmented quadratic cost, which generalized LQR handles. Remembering the previous action restores the Markov description of the smoothness objective.

5.5 Iterative LQR for nonlinear dynamics

For general smooth f and stage cost ℓ, ITERATIVE LQR (iLQR) repeatedly builds a local LQ problem around a nominal rollout $(\bar{x}_t, \bar{u}_t)$.

1. **Initialize a feasible rollout.** Choose controls and simulate the true nonlinear model. The states and controls must agree through the dynamics.
2. **Build local models.** Linearize dynamics and quadratically expand costs in deviations.
3. **Run a backward pass.** Compute an affine correction $\delta u_t = k_t + K_t \delta x_t$ from local value derivatives.
4. **Try a new rollout.** Use $u_t = \bar{u}_t + \alpha k_t + K_t(x_t - \bar{x}_t)$ and simulate the nonlinear dynamics. Reduce α until the actual cost improves sufficiently.
5. **Repeat or stop.** Accept an improving rollout, adjust regularization, and stop when improvements and local gradients are small or the budget is exhausted.

Here K_t includes the sign of the feedback correction; it is not the positive-gain convention used in $u = -Kx$ above.

To make the backward pass explicit, write local future derivatives as V_x, V_{xx} at the next state. The local state-action function has

$$\begin{aligned} Q_u &= \ell_u + B^\top V_x, & Q_x &= \ell_x + A^\top V_x, \\ Q_{uu} &= \ell_{uu} + B^\top V_{xx} B, & Q_{ux} &= \ell_{ux} + B^\top V_{xx} A, \\ Q_{xx} &= \ell_{xx} + A^\top V_{xx} A. \end{aligned}$$

Minimizing its quadratic gives $k = -Q_{uu}^{-1} Q_u$ and $K = -Q_{uu}^{-1} Q_{ux}$. Substitution yields $V_x = Q_x - Q_{xu} Q_{uu}^{-1} Q_u$ and $V_{xx} = Q_{xx} - Q_{xu} Q_{uu}^{-1} Q_{ux}$. These compact formulas assume the local minimizer exists and $Q_{uu} \succ 0$.

A local model can predict improvement far from the region where it is accurate. Damping Q_{uu}, adding a penalty for leaving the nominal trajectory, and line search control this problem. They improve reliability but do not turn a nonconvex trajectory problem into a globally solved one.

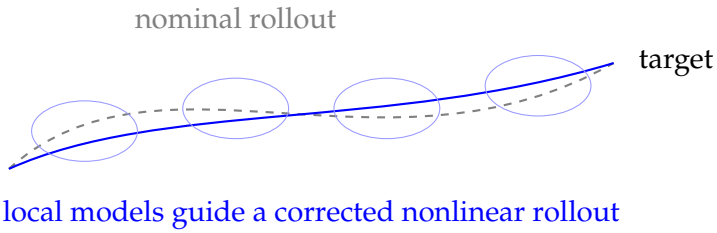


Figure 9: Local approximations are useful near the trajectory on which they were built.

5.6 DDP retains curvature of the dynamics

Differential dynamic programming (DDP) expands the composed Bellman function to second order. Besides iLQR's terms, it retains contributions such as

$$Q_{uu}^{\mathrm{DDP}} = \ell_{uu} + B^\top V_{xx} B + \sum_i (V_x)_i (f_i)_{uu}.$$

Analogous terms appear in Q_{xx} and Q_{ux}. The extra term differentiates the dynamics twice. It vanishes for affine dynamics but can matter for nonlinear systems.

For an illustrative scalar $f(u) = u^2$ and $V(x') = x'$, the composition is $V(f(u)) = u^2$ with second derivative 2. iLQR's curvature contribution $f_u^2 V_{xx}$ is zero because $V_{xx} = 0$. DDP retains $V_x f_{uu} = 1 \cdot 2 = 2$. More curvature can improve the local model, but may also make the quadratic indefinite and require regularization.

5.7 Replanning, bounds, and controllability

Receding-horizon control recomputes a trajectory from the current state, executes its first action, and repeats. It can recover from deviations that a fixed trajectory cannot, subject to computational latency and model quality. A terminal value can account for costs beyond a short planning horizon.

A smooth parameterization $u = u_{\max} \tanh v$ enforces a symmetric control bound, but near saturation its derivative becomes small. Optimizing v can then become poorly conditioned, and this parameterization reaches the boundary only as $|v| \to \infty$. Explicit constrained methods offer another approach.

For an LTI system, repeated substitution gives

$$x_T = A^T x_0 + [A^{T-1}B \; A^{T-2}B \; \cdots \; B] \begin{pmatrix} u_0 \\ u_1 \\ \vdots \\ u_{T-1} \end{pmatrix}.$$

Here A^T is a matrix power; transposes elsewhere use $\top$. Controllability requires the block matrix to span the state space. By Cayley–Hamilton, testing $[B \; AB \; \cdots \; A^{n-1}B]$ suffices for an n-state LTI system.

Feedback linearization instead seeks a state/input transformation that makes nonlinear dynamics linear. For $\ddot{q} = \sin q + u$, choosing $u = v - \sin q$ gives $\ddot{q} = v$. Exact cancellation assumes the model and actuation allow it; saturation or an inaccurate $\sin q$ term leaves residual dynamics. Latent linear models similarly seek coordinates useful for control.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 5.

Lecture 6: Unconstrained Optimization

6.1 Geometry, convexity, and stationary points

Trajectory optimization reduces control to minimizing a function of many variables. Before adding constraints, consider $\min_x f(x)$ with $x \in \mathbb{R}^n$. Its gradient $g = \nabla f(x)$ gives first-order changes, and its Hessian $H = \nabla^2 f(x)$ gives local curvature:

$$f(x + d) \approx f(x) + g^\top d + \tfrac{1}{2} d^\top H d.$$

At an unconstrained differentiable local minimum, $g = 0$. A positive-definite Hessian there is sufficient for a strict local minimum. A zero gradient alone also occurs at maxima and saddle points.

A function is convex if

$$f(\alpha x + (1 - \alpha)y) \le \alpha f(x) + (1 - \alpha) f(y), \quad 0 \le \alpha \le 1.$$

Its graph lies below every chord. For a twice differentiable function on an open convex domain, a positive-semidefinite Hessian characterizes convexity. Every local minimum of a convex function is global, but it need not be unique. For example, $f(x, y) = x^2$ has every point $(0, y)$ as a minimum. Strict convexity, rather than convexity alone, gives uniqueness when a minimum exists.

6.2 Gradient descent and step selection

Among directions with Euclidean length at most one, $-g/\|g\|$ minimizes $g^\top d$ by Cauchy–Schwarz. This explains the negative-gradient direction. Its useful length is a separate decision:

$$x_{k+1} = x_k - \alpha_k g_k.$$

A step that is too large can increase the objective even if the direction is initially downhill.

1. **Exact line search** minimizes $f(x + \alpha d)$ over $\alpha \ge 0$. It can be worthwhile when evaluating the one-dimensional problem is cheap.
2. **Backtracking** begins with a trial step, then repeatedly multiplies it by $\beta \in (0, 1)$ until the Armijo condition holds:
$$f(x + \alpha d) \le f(x) + c\alpha g^\top d, \qquad 0 < c < 1.$$

3. **Stopping** should inspect gradient size and meaningful progress, while accounting for variable scaling. A small change in objective alone can also indicate a tiny step or a poorly conditioned problem.

For a genuine descent direction $g^\top d < 0$, differentiability ensures sufficiently small steps pass backtracking. If the direction is not a descent direction, shrinking the step is not a complete remedy.

6.2.1 Worked example: unstable and stable step sizes

Let $f(x) = 5x^2$, so $g = 10x$ and the update is $x_{k+1} = (1 - 10\alpha)x_k$.

1. Convergence requires $|1 - 10\alpha| < 1$, or $0 < \alpha < 0.2$.
2. At $\alpha = 0.1$, one step reaches zero exactly.
3. At $\alpha = 0.19$, the sign alternates and the magnitude shrinks by 0.9 each step.
4. At $\alpha = 0.25$, the magnitude grows by 1.5 each step. The negative-gradient direction was correct, but the length was not.

6.3 Conditioning and coordinate scaling

For $f(x) = \frac{1}{2}x^\top Hx$ with positive-definite H, each eigenvector component evolves by a factor $1 - \alpha\lambda_i$. A step safe for a large eigenvalue may make very slow progress in a direction with a small eigenvalue. The ratio $\kappa = \lambda_{\max}/\lambda_{\min}$ is the spectral condition number.

6.3.1 Worked example: a narrow quadratic valley

Let $f(x, y) = \frac{1}{2}(x^2 + 100y^2)$ and start at $(1, 1)$. With $\alpha = 0.01$, the next point is $(0.99, 0)$. The stiff y direction is solved immediately, but the x coordinate decreases only by one percent per step. A step $\alpha = 1$ would solve x but explode in y. Scaling variables or using curvature information addresses this mismatch.

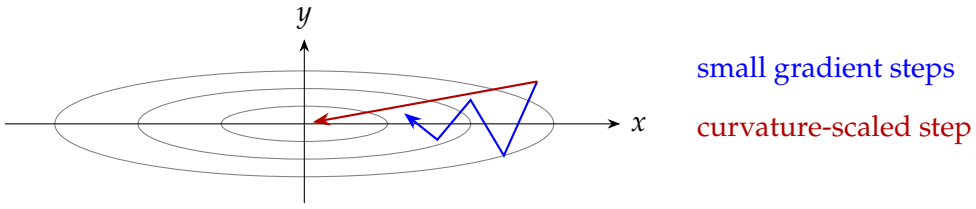


Figure 10: Unequal curvature can cause slow, alternating progress; the paths are schematic.

6.4 Newton's method and safeguards

Minimizing the quadratic model gives $Hd = -g$, hence $d = -H^{-1}g$ when H is nonsingular. Solve the system instead of computing an inverse. On a positive-definite quadratic, an undamped Newton step reaches the minimizer in one iteration.

For the previous example, $H = \operatorname{diag}(1, 100)$ and $g = (1, 100)$, so $d = (-1, -1)$ reaches $(0, 0)$. The inverse curvature rescales the two directions appropriately.

For a nonconvex function, H can be indefinite and the Newton direction may increase f. One safeguard uses $(H + \lambda I)d = -g$ with sufficiently large λ to make the matrix positive definite. Line search then tests the actual nonlinear objective. Near a regular minimizer, full Newton steps can converge quadratically, but that local rate is not a guarantee from an arbitrary initialization.

6.4.1 Why Newton is invariant to affine coordinates

Let $x = Ay + b$ for invertible A, and $\tilde{f}(y) = f(Ay + b)$. Then

$$\nabla_y \tilde{f} = A^\top g, \qquad \nabla_y^2 \tilde{f} = A^\top H A.$$

The Newton direction in y is

$$d_y = -(A^\top H A)^{-1} A^\top g = -A^{-1} H^{-1} g.$$

Multiplying by A gives $A d_y = -H^{-1} g = d_x$. Thus corresponding full steps produce the same physical point. Gradient descent lacks this property because its Euclidean notion of length changes under a nonorthogonal transformation. Compatible line-search rules preserve the corresponding step lengths.

6.5 Gauss–Newton and natural gradient

For nonlinear least squares $f(\theta) = \frac{1}{2} \sum_i r_i(\theta)^2$, let J be the residual Jacobian. Then

$$\nabla f = J^\top r, \qquad \nabla^2 f = J^\top J + \sum_i r_i \nabla^2 r_i.$$

Gauss–Newton drops the second term and solves $(J^\top J)d = -J^\top r$. The retained matrix is positive semidefinite and uses only first residual derivatives. Damping helps with rank deficiency. The approximation is especially useful when residuals are small or nearly linear, but can be poor far from a solution.

6.5.1 Worked example: nonlinear least squares

For $r(\theta) = \theta^2 - 4$ at $\theta = 1$, $r = -3$ and $J = 2$.

1. The gradient is $Jr = -6$.
2. Gauss–Newton curvature is $J^2 = 4$, giving $d = 1.5$ and trial point 2.5.
3. The exact Hessian is $J^2 + rr'' = 4 - 6 = -2$. An undamped Newton step would be $d = -(-6)/(-2) = -3$, moving to -2 through a locally misleading model.
4. A line search evaluates trial costs. At $\theta = 2.5$, cost is $\frac{1}{2}(2.25)^2 = 2.53125$, smaller than the original 4.5, so the Gauss–Newton trial improves this objective.

For a statistical model $p_\theta(z)$, natural gradient uses the Fisher information

$$F(\theta) = \mathbb{E}_{z\sim p_\theta}[\nabla_\theta \log p_\theta(z) \nabla_\theta \log p_\theta(z)^\top].$$

A small parameter change has local KL divergence approximately $\frac{1}{2} d^\top F d$. Minimizing a linearized loss while limiting this distributional change gives a direction proportional to $-F^{-1}\nabla f$. The metric measures changes in represented distributions rather than raw parameter distance.

Under a smooth invertible reparameterization with Jacobian J, gradients and Fisher matrices transform as $g_{\text{new}} = J^\top g$ and $F_{\text{new}} = J^\top F J$. Their combination transforms as a direction vector. This is a statement about the infinitesimal direction; arbitrary finite Euler steps in two nonlinear parameterizations need not land on exactly the same distribution. Empirical Fisher approximations, damping, or singular models also require care.

6.6 Momentum, RMSprop, and Adam

These methods use inexpensive running statistics instead of a full Hessian.

- **Momentum** smooths gradients: $m_k = \beta m_{k-1} + (1-\beta) g_k$, followed by $x_{k+1} = x_k - \alpha m_k$. Persistent directions accumulate while alternating directions partially cancel.
- **RMSprop** tracks squared gradients: $v_k = \beta v_{k-1} + (1-\beta) g_k^{\odot 2}$ and uses $x_{k+1} = x_k - \alpha g_k / (\sqrt{v_k} + \varepsilon)$, coordinatewise.
- **Adam** combines both statistics with coefficients β_1, β_2. Starting at zero introduces bias, corrected by $\hat{m}_k = m_k/(1-\beta_1^k)$ and $\hat{v}_k = v_k/(1-\beta_2^k)$. The step is $-\alpha \hat{m}_k/(\sqrt{\hat{v}_k} + \varepsilon)$.

For an illustrative first scalar gradient $g_1 = 2$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, the raw statistics are $m_1 = 0.2$ and $v_1 = 0.004$. Bias correction returns $\hat{m}_1 = 2$, $\hat{v}_1 = 4$, so the first Adam step is approximately $-\alpha$. Adaptive scaling alone does not guarantee convergence on a nonconvex objective.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 6.

Lecture 7: Constrained Optimization

7.1 Feasibility is part of the problem

Write a constrained problem as

$$\min_x f(x) \quad \text{subject to} \quad g_i(x) \le 0, \quad h_j(x) = 0.$$

Inequalities can represent joint limits or obstacle clearance; equalities can represent dynamics and initial conditions. A point with a low objective but violated constraints is not a solution. Convex optimization requires a convex objective and inequality functions, affine equalities, and a convex domain.

7.2 Exact penalties and sequential convex models

A penalty method replaces constraints by a cost on violation:

$$M_\mu(x) = f(x) + \mu \sum_i \max(0, g_i(x)) + \mu \sum_j |h_j(x)|.$$

Only positive inequality violation is penalized: making g_i more negative is feasible and should not itself incur a violation cost. The absolute-value penalty is nonsmooth. Under appropriate regularity and a sufficiently large μ, it can be exact, meaning a constrained local solution also minimizes the penalized problem locally.

7.2.1 Worked example: linear versus quadratic penalties

Minimize $(x-2)^2$ subject to $x \le 1$. The constrained solution is $x = 1$.

1. For the linear penalty, the objective is $(x-2)^2 + \mu \max(0, x-1)$. On $x > 1$, its derivative is $2(x-2) + \mu$.
2. If $0 < \mu < 2$, its minimum is $x = 2 - \mu/2 > 1$, so the constraint is violated.
3. If $\mu \ge 2$, moving right from 1 cannot decrease the objective, while the feasible-side minimum is at 1. Thus a finite penalty suffices.
4. With squared penalty $\mu \max(0, x-1)^2$, the minimizer is $x = (2+\mu)/(1+\mu) = 1 + 1/(1+\mu)$, violating the constraint for every finite μ.

Large penalties can improve feasibility while worsening numerical conditioning. The penalty parameter is not a substitute for reporting final constraint residuals.

For nonconvex functions, sequential convex programming constructs a local model around $\bar{x}$. Linearize nonconvex constraints, retain convex terms exactly, and restrict $\|x - \bar{x}\| \leq \Delta$ to a trust region. Absolute values and positive parts of affine expressions remain convex, so a convex subproblem can be solved reliably.

1. Compute the local model and its predicted decrease.
2. Solve inside the trust region.
3. Evaluate the *original* merit function at the trial point.
4. Compare actual decrease with predicted decrease. Accept good agreement and possibly enlarge Δ; reject poor agreement and shrink it.
5. Increase μ in an outer loop if constraint violation remains significant, subject to a maximum budget.

A model with no predicted decrease needs a stopping or failure decision; dividing by its predicted decrease is not meaningful. Rejected steps leave the accepted point unchanged.

7.3 Equality constraints and the null space

For $Ax = b$, choose one feasible point x_p and a matrix F whose columns span $\ker A$. Every feasible point is $x = x_p + Fz$, because $AF = 0$. The constrained problem becomes unconstrained minimization of $f(x_p + Fz)$.

If $A = U\Sigma V^\top$, the appropriate null-space columns come from V, the right singular vectors. The left singular vectors belong to the equation space and generally have the wrong dimension for this purpose. Elimination reduces variables but can destroy sparsity, an important drawback for long trajectories.

7.3.1 Worked example: eliminate one equality

Minimize $\frac{1}{2}(x_1^2 + x_2^2)$ subject to $x_1 + x_2 = 1$.

1. Set $x_2 = 1 - x_1$, giving $f(x_1) = \frac{1}{2}[x_1^2 + (1 - x_1)^2]$.
2. The derivative is $2x_1 - 1$, so $x_1 = x_2 = 1/2$.
3. The minimum cost is $1/4$. The unconstrained minimizer $(0, 0)$ is inadmissible.

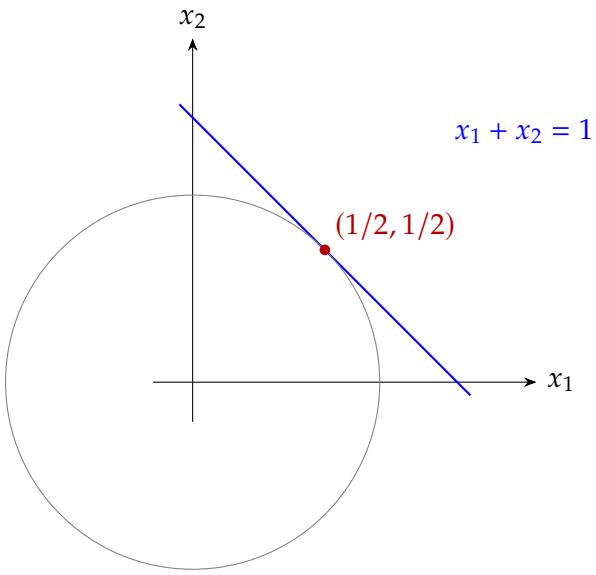


Figure 11: The minimum-norm feasible point is where a circular level set touches the equality line.

7.4 Lagrange multipliers and KKT systems

For equality constraints, define $L(x, \nu) = f(x) + \nu^\top (Ax - b)$. A stationary feasible point satisfies

$$\nabla f(x) + A^\top \nu = 0, \qquad Ax = b.$$

The multiplier balances the objective gradient against normals to the feasible set. At a constrained optimum, the objective need not have zero gradient in the full space; it must have no improving feasible tangent direction.

For a feasible Newton step, minimize $g^\top d + \frac{1}{2} d^\top H d$ subject to $Ad = 0$. The first-order conditions give

$$\begin{pmatrix} H & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} d \\ \eta \end{pmatrix} = - \begin{pmatrix} g \\ 0 \end{pmatrix}.$$

This saddle-point matrix is generally indefinite even for a convex objective. Solvability depends on constraint rank and positive curvature on feasible directions, not on positive definiteness of the entire block matrix.

Starting infeasibly, use residuals $r_d = \nabla f(x) + A^\top \nu$ and $r_p = Ax - b$ and solve

$$\begin{pmatrix} H & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} d_x \\ d_\nu \end{pmatrix} = - \begin{pmatrix} r_d \\ r_p \end{pmatrix}.$$

A full step corrects the linear equality residual exactly; damping may be needed to reduce the combined residual for nonlinear objectives.

With differentiable inequalities, the Karush–Kuhn–Tucker conditions additionally require

$$g_i(x) \le 0, \quad \lambda_i \ge 0, \quad \lambda_i g_i(x) = 0, \quad \nabla f + \sum_i \lambda_i \nabla g_i + \sum_j \nu_j \nabla h_j = 0.$$

Complementary slackness says that an inactive inequality has zero multiplier. Under constraint qualifications these conditions are necessary locally; for convex problems they are sufficient. Nonconvex KKT points need not be global minima.

For $(x-2)^2$ with $g(x) = x - 1 \le 0$, stationarity at $x = 1$ gives $-2 + \lambda = 0$, so $\lambda = 2$. This also explains why the exact penalty needed weight at least 2 in the earlier example.

7.5 Log barriers and the central path

For strictly feasible inequalities, define the barrier

$$\phi(x) = -\sum_{i=1}^{m} \log[-g_i(x)].$$

It tends to infinity as a constraint boundary is approached from inside. The barrier subproblem is

$$\min_x \; t f(x) + \phi(x) \quad \text{subject to } Ax = b.$$

Increasing t reduces the barrier's relative importance. Its gradient and Hessian terms are

$$\nabla\phi = \sum_i \frac{\nabla g_i}{-g_i}, \qquad \nabla^2\phi = \sum_i \left[\frac{\nabla g_i \nabla g_i^\top}{g_i^2} + \frac{\nabla^2 g_i}{-g_i} \right].$$

For convex g_i, both contributions are positive semidefinite inside the domain.

1. Begin at a strictly feasible point and moderate t.
2. Solve the equality-constrained barrier subproblem, using a line search that remains strictly feasible.
3. Warm-start the next subproblem at the current solution and multiply t by a factor greater than one.
4. Under the standard convex duality assumptions and sufficiently accurate centering, stop when the bound m/t is below the desired objective-gap tolerance.

The barrier keeps iterates inside the feasible set. It does not start from a violated inequality because the logarithm is then undefined.

7.5.1 Worked example: the barrier approaches an active bound

Minimize $-x$ subject to $x < 1$, with the intended closed constraint $x \le 1$. The barrier objective is $-tx - \log(1-x)$.

1. Stationarity gives $-t + 1/(1-x) = 0$, hence $x_t = 1 - 1/t$.
2. At $t = 2, 10, 100$, the points are $0.5, 0.9, 0.99$. They approach the optimum from inside.
3. The original objective gap is $1 - x_t = 1/t$, matching m/t with one inequality.
4. Barrier curvature is $1/(1-x_t)^2 = t^2$. Accuracy increases while conditioning can become more difficult.

7.6 Finding an interior point and interpreting dual updates

A phase-I problem introduces slack s:

$$\min_{x,s} s \quad \text{subject to } g_i(x) \le s, \quad Ax = b.$$

Given equality-feasible x, a sufficiently large s initializes the inequalities. Finding $s < 0$ proves strict feasibility for the original inequalities. If the best s is zero, strict feasibility may fail even though boundary-feasible points exist. A sum-of-violations variant uses separate nonnegative slacks and can preserve more satisfied inequalities.

The dual function is $q(\lambda, \nu) = \inf_x L(x, \lambda, \nu)$, giving lower bounds for a minimization primal when $\lambda \ge 0$. The dual problem *maximizes* this concave function. If an exact inner minimizer x_k is available, a projected dual ascent update is

$$\lambda_i^{k+1} = \max(0, \lambda_i^k + \alpha_k g_i(x_k)), \qquad \nu_j^{k+1} = \nu_j^k + \alpha_k h_j(x_k).$$

A violated inequality increases its price; a slack one can decrease it toward zero. Some formulations negate the dual objective and call the same operation dual descent. Always check the sign convention against the residual interpretation. Unlike a common penalty coefficient, multipliers adapt separately to individual constraints, but convergence still requires appropriate assumptions and step sizes.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 7.

Lecture 8: Shooting, Collocation, and Model-Predictive Control

8.1 Three independent modeling choices

An optimal-control formulation can be classified along three axes:

1. **Variables: shooting or collocation.** Shooting optimizes controls and generates states by simulation. Collocation optimizes states as well and imposes dynamics explicitly.
2. **Output: action sequence or feedback policy.** A sequence chooses $u_0, \dots, u_{H-1}$. A policy $u_t = \pi_\theta(x_t)$ also specifies what to do if the state changes.
3. **Execution: commit or replan.** An open-loop execution follows the planned controls. Model-predictive control repeatedly solves a new problem after observing the current state.

These axes should not be conflated. An optimizer can produce an open-loop sequence while its first-action-only execution creates a feedback controller through replanning.

Use dynamics $x_{t+1} = f_t(x_t, u_t)$ and objective $\sum_{t=0}^{H-1} \ell_t(x_t, u_t) + g(x_H)$. State and control constraints can be added where needed.

8.2 Shooting: eliminate the states by simulation

Starting from known x_0, substitute the dynamics repeatedly into the cost. The decision vector is $U = (u_0, \dots, u_{H-1})$, and every evaluation rolls out a physically consistent trajectory under the model.

- **Benefit.** Dynamics are satisfied automatically; there are fewer explicit variables and no dynamics equality residuals.
- **Difficulty.** An early action can affect every future state, creating long chains of derivatives. Unstable dynamics amplify sensitivities.
- **Initialization.** Guessing a good state path does not immediately give controls that reach it. An arbitrary action guess can lead far from the intended region.

A feedback shooting formulation substitutes $u_t = \pi_\theta(x_t)$ and optimizes policy parameters. Its rollouts are also dynamically consistent, but the controller must be evaluated over relevant initial states and disturbances, rather than one nominal trajectory alone.

8.2.1 Worked example: sensitivity grows with the horizon

For $x_{t+1} = 2x_t + u_t$, repeated substitution gives

$$x_H = 2^H x_0 + \sum_{k=0}^{H-1} 2^{H-1-k} u_k.$$

1. The sensitivity to u_0 is 2^{H-1}; the sensitivity to u_{H-1} is 1.
2. At $H = 10$, these are 512 and 1. A single common step scale must handle very different effects.
3. With terminal cost x_H^2, gradients inherit these unequal factors. Numerical errors and model disturbances also propagate through the unstable dynamics.

The issue is conditioning, not an incorrect chain rule. Feedback can stabilize the propagation, while collocation changes the optimization representation.

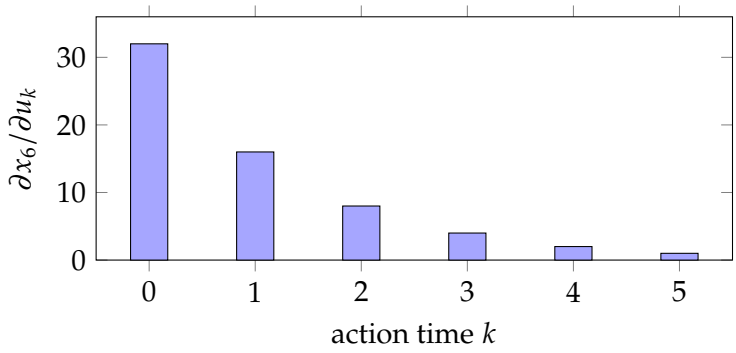


Figure 12: For $x_{t+1} = 2x_t + u_t$, early controls have much larger terminal-state sensitivities.

8.3 Collocation: expose the temporal constraints

Direct collocation retains $X = (x_1, \ldots, x_H)$ and U as variables:

$$\begin{aligned} \min_{X,U} \quad & \sum_{t=0}^{H-1} \ell_t(x_t, u_t) + g(x_H) \\ \text{subject to} \quad & x_{t+1} - f_t(x_t, u_t) = 0, \qquad x_0 = x_{\text{measured}}. \end{aligned}$$

Each dynamics constraint involves only neighboring times. Its Jacobian therefore has a sparse block structure. The optimizer can move a later state directly, instead of changing an early control and waiting for that change to propagate through a rollout.

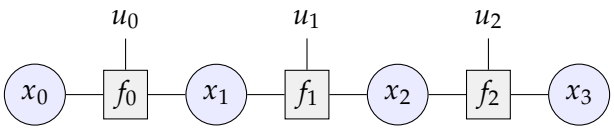


Figure 13: Each collocation constraint couples adjacent states and one control.

An initial state path can be interpolated between start and goal even if it is not dynamically feasible. That flexibility helps search, but intermediate optimizer iterates cannot necessarily be executed. Before deployment, inspect dynamics defects and all other constraints.

For a policy version, add $u_t = \pi_\theta(x_t)$ or substitute it into the local dynamics constraints. Policy parameters can couple many times and trajectories even though physical dynamics remain local in time.

8.3.1 Worked example: equivalent optima, different variables

Let $x_{t+1} = x_t + u_t$, $x_0 = 0$, and minimize $u_0^2 + u_1^2$ subject to $x_2 = 2$.

1. Shooting substitutes $x_2 = u_0 + u_1$, giving the equality $u_0 + u_1 = 2$.
2. Collocation retains x_1, x_2 with $x_1 = u_0$, $x_2 = x_1 + u_1$, and $x_2 = 2$.
3. In either representation, $u_1 = 2 - u_0$ and the cost is $u_0^2 + (2 - u_0)^2$. Differentiation gives $u_0 = u_1 = 1$, $x_1 = 1$, $x_2 = 2$, and cost 2.
4. Collocation could initialize at $x_1 = 1, x_2 = 2, u_0 = u_1 = 0$, which has a useful geometric path but violates dynamics. Shooting with those controls necessarily generates $x_1 = x_2 = 0$.

Equivalent mathematical optima do not imply identical numerical behavior during optimization.

8.4 Backpropagation through time

Shooting gradients can reuse computations. Let $A_t = f_x(x_t, u_t)$ and $B_t = f_u(x_t, u_t)$. Define a costate, or accumulated future gradient, $\lambda_H = \nabla g(x_H)$. A backward recursion gives

$$\lambda_t = \ell_{x,t} + A_t^\top \lambda_{t+1}, \qquad \nabla_{u_t} J = \ell_{u,t} + B_t^\top \lambda_{t+1}.$$

1. Run the trajectory forward and store the states and derivative information.
2. Initialize the terminal gradient.
3. Move backward, adding each local state-cost gradient to the future gradient propagated through the dynamics.
4. Use each control's local cost gradient plus its effect on the next state.

The method avoids separately differentiating every future cost with respect to every earlier action. Its graph traversal is linear in the horizon, with matrix operations determined by state and action dimension. Storing the rollout uses $O(Hn)$ state memory; checkpointing trades extra forward computations for memory.

For a feedback policy, use state Jacobian $f_x + f_u \pi_x$ and local state-cost gradient $\ell_x + \pi_x^\top \ell_u$ in the costate recursion. Shared parameters contribute $\pi_\theta^\top (\ell_u + f_u^\top \lambda_{t+1})$ at each time. Omitting the policy's state dependence loses part of the gradient.

8.4.1 Worked example: a two-step gradient

Let $x_{t+1} = 2x_t + u_t$, $x_0 = 0$, and $J = \frac{1}{2}x_2^2 + \frac{1}{2}(u_0^2 + u_1^2)$. At $(u_0, u_1) = (1, 0)$, the rollout is $x_1 = 1$, $x_2 = 2$.

1. $\lambda_2 = x_2 = 2$.
2. $\nabla_{u_1} J = u_1 + \lambda_2 = 2$, and $\lambda_1 = 2\lambda_2 = 4$.
3. $\nabla_{u_0} J = u_0 + \lambda_1 = 5$.
4. Direct expansion gives $J = \frac{1}{2}(2u_0 + u_1)^2 + \frac{1}{2}(u_0^2 + u_1^2)$, whose derivatives at $(1, 0)$ are also $(5, 2)$.

8.5 Model-predictive control in operation

At time t, MPC solves a finite-horizon problem starting from the current estimate $\hat{x}_t$, executes only its first action, receives a new measurement, and solves again. A shifted previous solution provides a warm start: discard the executed control, move the remaining controls earlier, and append a terminal guess.

Algorithm 8.1: Receding-horizon control

Input: Model, horizon h, cost, constraints, initial control guess

for each control time t **do**
- Estimate current state $\hat{x}_t$
- Solve the horizon-h problem with $x_0 = \hat{x}_t$
- Check feasibility and solver status; select the first usable control
- Apply that control and acquire the next measurement
- Shift the solution to initialize the next optimization

A short horizon can ignore a consequence just beyond its endpoint. Terminal costs and terminal constraints can address this, but stability and recursive feasibility require actual design assumptions; replanning alone is not a theorem that the robot remains safe. Solver latency also matters: the action is applied to a moving system, so delayed estimates and computation must be represented or compensated. For a car, the model must also retain its nonholonomic motion constraints.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 8.

Lecture 9: Direct Collocation and Contact-Invariant Optimization

9.1 Why optimize a trajectory directly?

Shooting restricts every candidate to a simulated motion. That is convenient when good controls are easy to find, but can make search difficult when most control guesses collide or fall. Direct collocation instead lets the optimizer propose states near a desirable motion before all dynamics are satisfied. It searches a larger space and gradually enforces consistency.

For $x_{t+1} = f(x_t, u_t)$, retain both states and controls and constrain their relationship. A dynamics defect $d_t = x_{t+1} - f(x_t, u_t)$ measures inconsistency. Hard equality constraints require $d_t = 0$; a soft formulation penalizes $\sum_t \|d_t\|^2$ and must check remaining defects before execution. A small objective alone does not establish that the path is physically realizable.

9.2 Inverse dynamics and local temporal structure

A forward model answers: given state and action, what happens next? An inverse model answers: what action could produce this transition? An inverse mapping need not exist or be unique. Underactuated systems cannot realize every proposed acceleration, and redundant actuators can realize one acceleration in multiple ways.

For generalized coordinates q, rigid-body mechanics has the form

$$M(q)\ddot{q} + h(q, \dot{q}) = Bu + J(q)^\top f_c,$$

where M is the mass matrix, h collects gravity and velocity-dependent terms, B maps actuator forces, and $J^\top f_c$ maps contact forces into generalized forces. Contact forces are distinct from actuator commands.

At uniform time spacing Δt, a configuration trajectory gives approximations

$$\dot{q}_t \approx \frac{q_{t+1} - q_{t-1}}{2\Delta t}, \qquad \ddot{q}_t \approx \frac{q_{t+1} - 2q_t + q_{t-1}}{\Delta t^2}.$$

Thus an inverse-dynamics residual at time t depends only on three neighboring configurations. Boundary times need appropriate one-sided formulas or specified endpoint velocities.

9.2.1 Worked example: a vertical particle

Take upward position q, mass $m = 2$, downward gravitational acceleration $g = 9.8$, and actuator force u upward. The equation is $m\ddot{q} = u - mg$, so $u = m(\ddot{q} + g)$.

1. A stationary trajectory $q_{t-1} = q_t = q_{t+1} = 1$ has $\ddot{q} = 0$, requiring $u = 19.6$. Holding still does not mean zero force.
2. With $\Delta t = 0.1$ and positions $1, 1.01, 1.04$, acceleration is $(1.04 - 2.02 + 1)/0.01 = 2$, requiring $u = 23.6$.
3. If $u \leq 20$, the second trajectory is infeasible even though its positions look smooth. An inverse-dynamics check exposes this directly.

In a floating-base legged robot, some rows of B are zero. Contact forces must account for the corresponding base acceleration. Allowing arbitrary base forces would create a fictitious actuator and an invalid motion.

9.3 Gauss–Newton exploits sparse residuals

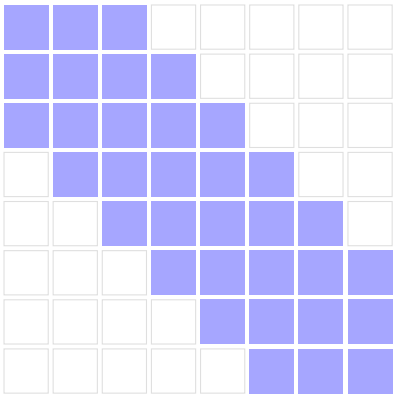
Let X collect the trajectory variables and $r_k(X)$ include task errors and dynamics residuals. For

$$C(X) = \tfrac{1}{2} \sum_k r_k(X)^\top W_k r_k(X),$$

with positive-semidefinite weights W_k, the gradient is $\sum_k J_k^\top W_k r_k$, and Gauss–Newton approximates the Hessian by $\sum_k J_k^\top W_k J_k$. Solve

$$(H_{\mathrm{GN}} + \lambda I)d = -\nabla C$$

and evaluate a damped step on the actual cost. The matrix is sparse because each residual depends on a small set of neighboring variables. A three-time residual can couple times up to two steps apart; it does not make every pair of times interact.



Each square is a time block. Only nearby times couple through local residuals.

Figure 14: Local trajectory residuals produce a banded Hessian pattern.

Sparse linear algebra is central to the efficiency claim. Treating the Hessian as a dense matrix discards the advantage gained by the formulation. Long-range task constraints or shared policy parameters add further couplings that must be represented explicitly.

9.4 Why contact changes the search problem

A foot alternates between being free in the air and constrained by the ground. The force can change abruptly when contact begins. A fixed contact schedule simplifies the dynamics but requires the schedule to be supplied; searching over schedules can become combinatorial.

A physical unilateral contact at signed gap $\phi(q)$ and normal force f_n obeys

$$\phi(q) \geq 0, \qquad f_n \geq 0, \qquad \phi(q) f_n = 0.$$

The robot cannot penetrate the ground, the ground cannot pull through a nonadhesive normal contact, and positive force requires zero gap. Friction constrains tangential forces, for example $\|f_{\text{tan}}\| \leq \mu f_n$ for Coulomb friction. Sticking additionally constrains tangential velocity. These conditions make contact optimization nonconvex and often nonsmooth.

9.5 Contact-invariant optimization: introduce consistency variables

Contact-invariant optimization introduces contact activity variables $c_{t,k}$ for candidate contact k at time t. A relaxed activity lies between zero and one. It allows the optimizer to explore a motion while contact and dynamics consistency are encouraged by penalties.

- **Active contact consistency.** When $c_{t,k}$ is high, penalize distance from the surface and unwanted sliding at that contact.
- **Inactive force consistency.** When activity is low, strongly penalize using that contact's force. Otherwise a remote foot could support the robot with an impossible force.
- **Dynamics consistency.** The chosen actuator and contact forces must explain the proposed accelerations.
- **Task cost.** Progress, pose, effort, and other task preferences rank the physically consistent solutions.

A schematic penalty might contain $c^2\phi(q)^2$, $c^2\|v_{\text{tan}}\|^2$, and $(1-c)^2\|f_c\|^2$, together with dynamics and other physical constraints. This illustrates the roles of the terms rather than specifying every detail of a particular implementation.

9.5.1 Worked failure case: optimizing with a fictitious support force

Suppose a proposed pose leaves a foot 0.2 meters above the floor but assigns it a 100-newton upward force.

1. The force might balance gravity in the dynamics equation, giving a small acceleration residual.
2. If the contact is marked active, a surface-consistency term penalizes the nonzero gap.
3. If it is marked inactive, an inactive-force penalty penalizes the 100-newton force.
4. Both tests are needed. Dynamics consistency alone cannot distinguish a real support from an invented one.

The enlarged search space can provide useful gradients while contact patterns change. It does not eliminate local minima or make an unfinished relaxed solution executable. Final gaps, forces, friction, and motion consistency must be checked.

9.6 From optimized motion clips to reusable policies

A motion clip describes one execution. A policy must also react to perturbations and different initial conditions. One approach optimizes several trajectories and trains a network to imitate their actions. A difficulty appears when individually good trajectories demand inconsistent actions at similar observed states.

Joint trajectory-policy optimization couples them through a fitting penalty:

$$\min_{\theta,\{X^i,U^i\}} \sum_i \left[J(X^i,U^i) + \lambda \sum_t \|\pi_\theta(x_t^i) - u_t^i\|^2 \right],$$

with dynamics constraints or corresponding residuals for each trajectory.

1. Hold the policy fixed and improve trajectories while encouraging actions the policy can reproduce.
2. Hold trajectories fixed and regress the policy onto their state-action pairs.
3. Collect relevant new executions, revise the trajectories, and alternate as needed.

Trajectory optimization now considers learnability, and policy fitting receives task-oriented examples. Separate trajectory subproblems can be computed in parallel conceptually, while shared network training combines their data. The mathematical idea is to add auxiliary variables and softly enforce their agreement, exchanging a harder direct search for coordinated simpler subproblems.

Source. Igor Mordatch, *Advanced Robotics*, Fall 2019, Lecture 9.

Lecture 10: Motion Planning in Configuration Space

10.1 Geometry, dynamics, and configuration obstacles

Motion planning asks how to move from a start configuration to a goal while avoiding obstacles and respecting the robot's motion restrictions. A configuration q specifies the robot's pose or joint angles. Its configuration space contains every possible such assignment. A configuration obstacle is the set of assignments for which any part of the robot collides.

For a translating disk of radius r, replace the disk by a point and expand each workspace obstacle outward by r. This Minkowski-sum construction transfers robot geometry into obstacle geometry. An articulated arm's configuration obstacles can have much more complicated shapes than the physical objects.

A geometric path $q(s)$ is not automatically a dynamically feasible motion. A car cannot translate sideways at will, and an underactuated robot cannot command every acceleration independently. Connecting configurations can require a two-point boundary-value problem: find controls that move one state to the other.

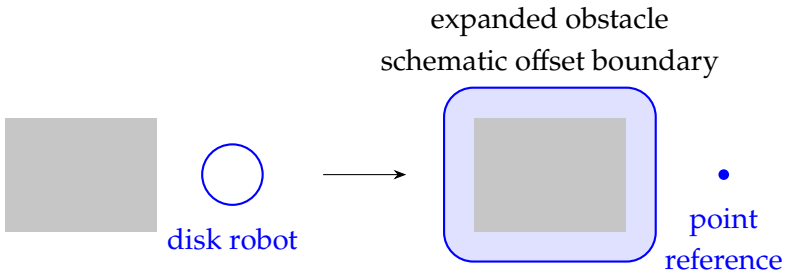


Figure 15: Expanding an obstacle accounts for a translating robot's size in configuration space.

10.2 Local trajectory optimization and collision distances

Trajectory optimization can minimize smoothness and effort while imposing collision constraints. Let $d(q)$ be signed separation, positive outside collision and negative during penetration. A clearance requirement is $d(q) \geq d_{\text{safe}}$. A hinge penalty is

$$\mu \max(0, d_{\text{safe}} - d(q)).$$

Near $\bar{q}$, approximate $d(q) \approx d(\bar{q}) + \nabla d(\bar{q})^{\top}(q - \bar{q})$. The gradient combines a separating direction with the Jacobian of the relevant robot point. Signed-distance methods such as GJK and EPA supply geometric information for convex bodies.

Sequential convex optimization repeatedly solves local approximations inside a trust region, evaluates the true geometry, and accepts or rejects the candidate. A local optimizer can get trapped on the wrong side of an obstacle, so initialization matters. Potential-field reactive methods have the related problem that attractive and repulsive forces can cancel away from the goal.

10.2.1 Worked example: waypoints can miss a collision

A point robot moves along the segment from $(-2, 0)$ to $(2, 0)$. An obstacle is a disk of radius 0.5 centered at the origin.

1. Both endpoints have clearance $2 - 0.5 = 1.5$, so checking only them reports no collision.
2. The segment passes through $(0, 0)$, whose signed clearance is -0.5.
3. Continuous collision checking must examine the swept motion, or use a conservative representation that covers it. Adding more waypoints helps only if the resolution is justified relative to geometry and motion.

For a translating convex body, a convex hull of endpoint placements covers its linear sweep. Rotating or articulated geometry needs additional care; endpoint hulls do not universally bound every nonlinear intermediate motion.

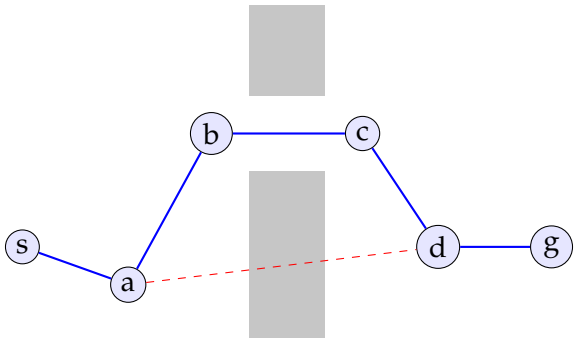
10.3 Probabilistic roadmaps for reusable connectivity

A PROBABILISTIC ROADMAP (PRM) approximates free-space connectivity by a graph.

1. Sample configurations and retain those that are collision-free.
2. Connect nearby samples using a local planner.
3. Validate each proposed connection along its whole motion. A collision-free endpoint pair does not establish a collision-free edge.
4. Add start and goal, connect them, and search the graph for a path.

The roadmap can support multiple queries in the same environment. Collision checking often dominates runtime. Neighbor searches can use spatial data structures, although their effectiveness depends on dimension and metric.

Under suitable robust-feasibility and sampling assumptions, PRM is probabilistically complete: if a solution with adequate clearance exists, the probability of missing a solution tends to zero as samples increase. This is not a finite-time certificate that no path exists after an unsuccessful run. Narrow passages may occupy tiny volume and require targeted sampling or many trials.



Dashed shortcut crosses an obstacle;
the validated route uses the gap.

Figure 16: A roadmap stores checked connections through free configuration space.

10.4 RRT grows toward unexplored space

A RAPIDLY EXPLORING RANDOM TREE (RRT) starts from the initial state and grows one search tree.

Algorithm 10.1: Basic RRT expansion

```
Input: Start state x_s, goal test, local propagation method
T ← {x_s}
while the computation budget remains do
    Sample a target x_rand, occasionally choosing the goal
    Find x_near ∈ T nearest to the target
    Propagate toward the target to obtain a candidate motion
    if the whole motion satisfies dynamics and collision checks then
        Add its endpoint and parent edge to T
        if the endpoint can validly connect to the goal then
            return the parent path to the goal
return failure within the available budget
```

Uniform target sampling does not imply uniform selection of existing nodes. A node with a large Voronoi region is nearest to more targets, so it is selected more often. This favors expansion toward less-covered regions.

For a holonomic system, steering can use a short straight segment. For kinodynamic planning, simulate candidate control primitives and choose a promising endpoint. The nearest-state metric should reflect useful reachability; raw Euclidean distance can be misleading for orientation or velocity.

10.4.1 Worked example: one expansion and one rejection

Suppose the tree has states $(0, 0)$, $(1, 0)$, and $(1, 1)$, and the target is $(2, 1)$. With Euclidean distance, $(1, 1)$ is nearest. A steering limit of 0.5 proposes $(1.5, 1)$.

1. If the connecting segment is free, add $(1.5, 1)$ with parent $(1, 1)$.
2. If an obstacle crosses that segment, add nothing, even if $(1.5, 1)$ itself is free.
3. On the next iteration a new random target may select a different part of the tree, allowing exploration around the obstacle.

A failed expansion supplies information about that attempt, not a proof that the node or goal is unreachable.

Bidirectional RRT grows from both start and goal and tries to connect the trees. This can reduce exploration, but backward growth must correspond to valid forward-time motions. Arbitrarily reversing dynamics is inappropriate for systems whose controls or transitions are not reversible.

A resolution-oriented variant tracks failed expansion frequency and reduces repeated attempts from nodes blocked by obstacles, propagating a smaller penalty to ancestors. Such heuristics must retain the assumptions needed for their particular completeness claims; permanently forbidding a region after a few failures can miss a valid narrow route.

10.5 RRT*, optimality, and path refinement

Basic RRT seeks feasibility and generally does not converge to the shortest path merely by continuing to grow. RRT* adds local cost-based parent selection and rewiring.

1. Find nearby vertices for a proposed new state.
2. Choose the collision-free parent with the least start-to-new-state cost.
3. For each neighbor, check whether routing it through the new state decreases its cost and remains valid.
4. Rewire improving connections and update affected descendant costs consistently.

Under suitable geometric, connection-radius, cost, and sampling assumptions, the best path cost approaches the optimum asymptotically. Kinodynamic versions need stronger steering capabilities or different carefully designed algorithms; geometric rewiring alone does not solve an arbitrary dynamical boundary-value problem.

For an illustrative new node with start cost 4, a neighbor currently costing 8 can be rewired if the checked new edge costs 1.5, since $4 + 1.5 = 5.5 < 8$. A different neighbor with cost 5 should retain its old parent if the new route would cost 6. Distance to the new node alone is not the decision criterion.

Randomized paths are often jagged. Shortcutting attempts to replace a subpath by a valid direct connection; trajectory optimization can then improve smoothness and effort. Every shortcut must

retain collision and dynamics checks. A geometrically shorter curve can demand impossible acceleration or violate joint limits.

LQR-trees add local stabilizing controllers and certified regions around trajectories. A sampled state already inside a known region can use its controller; new trajectories expand the covered region. The object being grown is therefore more than a collection of open-loop paths: it includes feedback behavior and the conditions under which that behavior reaches the goal.

The planning methods solve different computational problems. A common practical combination uses a sampling planner for an initial feasible route and local optimization for refinement.

Method	Representation	Useful role	Main difficulty
PRM	Graph of checked local paths	Repeated queries in one environment	Narrow passages and local connections
RRT	Tree rooted at a query start	Finding a feasible route	Path quality and dynamics-aware steering
RRT*	Cost-aware rewired tree	Asymptotic improvement	Valid connections and sufficient sampling
Trajectory optimization	States and controls along a path	Smooth task-specific refinement	Initialization and local minima

10.6 Sampling positions and orientations correctly

Independent uniform coordinates sample a rectangular box uniformly. To sample uniformly on the surface of a unit sphere, draw an isotropic Gaussian vector and normalize its length. Rotational symmetry of the Gaussian gives rotational symmetry of its direction.

For three-dimensional rotations, independent uniform Euler angles do not give uniform orientations. One option is to draw four independent standard normal values, normalize the resulting quaternion, and interpret it as a rotation. Antipodal unit quaternions q and $-q$ represent the same physical orientation; distances and interpolation should respect this equivalence. A three-dimensional pose combines translation with a rotation in $SE(3)$, not six unrelated Euclidean coordinates with globally valid angle arithmetic.

Source. Huazhe Xu and Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 10.

Lecture 11: Probability, Bayes Filters, and Multivariate Gaussians

11.1 Uncertainty is part of the state-estimation problem

A robot rarely knows its exact state. Wheel encoders measure actuator motion rather than perfect displacement; a helicopter's inertial sensors contain noise and bias; a camera provides indirect evidence about geometry. A probability distribution describes which states remain plausible after the available data.

For discrete variables, probabilities sum to one. For a continuous variable, a density integrates to one, and $\Pr(a < X < b) = \int_a^b p(x)\,dx$. A density value is not a point probability and can exceed one. For example, a uniform density on an interval of width 0.1 has height 10, while the probability of the whole interval remains one.

The joint distribution, conditional distribution, and marginal satisfy

$$p(x, y) = p(x \mid y)p(y), \qquad p(x) = \int p(x, y)\,dy.$$

Independence means $p(x, y) = p(x)p(y)$. Conditional independence given z means $p(x, y \mid z) = p(x \mid z)p(y \mid z)$. Independence without conditioning and independence after conditioning are different statements.

11.2 Bayes' rule and repeated evidence

Bayes' rule converts a sensor model into a state update:

$$p(x \mid z) = \frac{p(z \mid x)p(x)}{\int p(z \mid x')p(x')\,dx'}.$$

The prior $p(x)$ describes knowledge before this measurement; likelihood $p(z \mid x)$ scores how compatible the measurement is with each candidate state; the denominator normalizes the posterior. Likelihoods need not sum to one across candidate states.

11.2.1 Worked example: two door measurements

Initially a door is open with probability $1/2$. A first report has likelihoods $p(z_1 \mid \text{open}) = 0.6$ and $p(z_1 \mid \text{closed}) = 0.3$.

1. Unnormalized open and closed weights are $0.6(1/2) = 0.3$ and $0.3(1/2) = 0.15$.

2. Normalize by 0.45, giving open probability 2/3.
3. A conditionally independent second report has likelihoods 0.5 and 0.6. New weights are (1/2)(2/3) = 1/3 and (3/5)(1/3) = 1/5.
4. Their sum is 8/15, so the final open probability is (1/3)/(8/15) = 5/8.

The second report lowers the probability of being open because it is more likely under a closed door. It does not erase the first report.

If reports are conditionally independent given the state, their likelihoods multiply. Reusing an identical stored measurement is not independent evidence. Counting it repeatedly can create unjustified certainty, even though every individual normalization is arithmetically correct.

11.3 Actions predict; measurements correct

A BELIEF $b_t(x)$ is the posterior over the state after the observations available at time t. With action u_t taking X_{t-1} to X_t and measurement z_t about X_t, the Bayes filter has two steps:

$$\bar{b}_t(x) = \int p(x \mid x', u_t) b_{t-1}(x')\, dx',$$
$$b_t(x) = \eta\, p(z_t \mid x) \bar{b}_t(x).$$

The first line sums over unknown previous states; the second conditions the predicted state on new evidence. The normalizer η is the reciprocal integral of the unnormalized posterior.

1. Assume the previous belief summarizes the relevant history.
2. Use the Markov transition assumption to replace dependence on the entire history by dependence on (x', u_t).
3. Marginalize x' to obtain the prediction.
4. Use the observation's conditional independence given the current state to multiply by $p(z_t \mid x)$.

The state can include a changing environment when the model requires it. A static world is a useful modeling choice in some localization problems, not a universal requirement of Bayesian filtering.

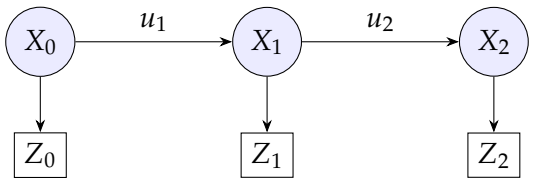


Figure 17: Transitions connect states; each measurement depends on its current state.

11.3.1 Worked example: act on an uncertain door

After the previous measurements, the door is open with probability 5/8. A close command succeeds with probability 0.9 if open and leaves an already closed door closed.

1. The next open probability is $(0.1)(5/8) + (0)(3/8) = 1/16$.
2. The next closed probability is $(0.9)(5/8) + (1)(3/8) = 15/16$.
3. This is a prediction, not a sensor update: it uses how the action changes the world, rather than treating the command as proof that it succeeded.

Motion often spreads location uncertainty, but it need not always increase every uncertainty measure. This close-door action makes the binary state more concentrated under the specified model.

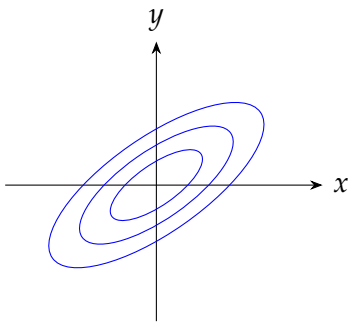
11.4 Gaussian distributions and covariance geometry

A multivariate Gaussian $X \sim \mathcal{N}(\mu, \Sigma)$ has mean $\mu = \mathbb{E}[X]$ and covariance $\Sigma = \mathbb{E}[(X - \mu)(X - \mu)^\top]$. For nonsingular covariance in n dimensions,

$$p(x) = \frac{\exp[-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)]}{(2\pi)^{n/2}|\Sigma|^{1/2}}.$$

Diagonal entries are variances; off-diagonal entries describe covariance. A valid covariance is symmetric positive semidefinite. Its eigenvectors give principal directions and its eigenvalues give variances along those directions.

The quadratic form is squared Mahalanobis distance. A large displacement along a high-variance direction can be more plausible than a smaller displacement along a low-variance direction. Correlation can rotate uncertainty ellipses, so separate one-dimensional error bars lose information.



Positive covariance: a higher x makes a higher y more plausible.

Figure 18: Gaussian level sets encode both scale and correlation.

For an affine transformation $Y = AX + b$, expectations give $\mu_Y = A\mu + b$ and covariance expansion gives $\Sigma_Y = A\Sigma A^\top$. If independent Gaussian noise of covariance Q is added, covariance becomes $A\Sigma A^\top + Q$. Independence matters: otherwise cross-covariance terms remain.

The central limit theorem explains why sums of many suitably independent finite-variance contributions often look Gaussian after normalization. It does not say every noisy sensor, nonlinear transformation, or multimodal localization distribution is Gaussian.

11.5 Marginalization and conditioning of a joint Gaussian

Partition a Gaussian as

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_x \\ \mu_y \end{pmatrix}, \begin{pmatrix} \Sigma_{xx} & \Sigma_{xy} \\ \Sigma_{yx} & \Sigma_{yy} \end{pmatrix}\right).$$

Marginalization simply selects a block: $X \sim \mathcal{N}(\mu_x, \Sigma_{xx})$. Conditioning is different:

$$\mu_{x|y} = \mu_x + \Sigma_{xy}\Sigma_{yy}^{-1}(y - \mu_y),$$
$$\Sigma_{x|y} = \Sigma_{xx} - \Sigma_{xy}\Sigma_{yy}^{-1}\Sigma_{yx}.$$

Assume Σ_{yy} is invertible. To understand the formula, define $K = \Sigma_{xy}\Sigma_{yy}^{-1}$ and residual $\epsilon = X - \mu_x - K(Y - \mu_y)$. Its covariance with Y is $\Sigma_{xy} - K\Sigma_{yy} = 0$. Joint Gaussianity makes this residual independent of Y. Conditioning therefore fixes the $K(Y - \mu_y)$ term while leaving the residual distribution unchanged. Expanding its covariance gives the subtraction formula.

11.5.1 Worked example: use a correlated measurement

Let means be zero and covariance be $\begin{pmatrix} 4 & 2 \\ 2 & 9 \end{pmatrix}$. Observe $Y = 3$.

1. The regression gain is $K = 2/9$.
2. The conditional mean of X is $(2/9)3 = 2/3$.
3. Its conditional variance is $4 - (2/9)2 = 32/9$, smaller than the original 4.
4. Observing $Y = -3$ changes the mean to $-2/3$ but leaves variance $32/9$. The conditional covariance depends on the model, not on the observed value in this linear Gaussian setting.

In information form, precision is $\Lambda = \Sigma^{-1}$ and information vector is $\eta = \Lambda\mu$. The exponent is $-\frac{1}{2}x^\top \Lambda x + \eta^\top x$ plus a constant. Independent Gaussian factors add their precisions and information vectors when multiplied. Conditioning selects the relevant quadratic terms, while marginalization involves a Schur complement in precision form.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 11.

Lecture 12: Kalman Filtering, EKF, and UKF

12.1 Linear Gaussian models preserve Gaussian beliefs

The Kalman filter evaluates the Bayes filter exactly when the initial state is Gaussian and dynamics and observations are linear with independent Gaussian noise. Use

$$x_t = A_t x_{t-1} + B_t u_t + w_t, \qquad z_t = C_t x_t + v_t,$$

where $w_t \sim \mathcal{N}(0, Q_t)$ and $v_t \sim \mathcal{N}(0, R_t)$ are mutually independent and independent of the previous state. Let the previous posterior be $\mathcal{N}(\mu_{t-1}, P_{t-1})$. Here Q_t and R_t are noise covariances, not LQR cost weights.

The prediction follows directly from affine Gaussian transformation:

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t, \qquad \bar{P}_t = A_t P_{t-1} A_t^\top + Q_t.$$

The control affects the mean because it is known. A deterministic known control contributes no extra covariance; uncertain controls would require additional terms.

12.2 Deriving the measurement update by conditioning

The predicted state and its next measurement are jointly Gaussian with mean $(\bar{\mu}, C\bar{\mu})$ and covariance

$$\begin{pmatrix} \bar{P} & \bar{P}C^\top \\ C\bar{P} & C\bar{P}C^\top + R \end{pmatrix}.$$

Define innovation $e = z - C\bar{\mu}$ and innovation covariance $S = C\bar{P}C^\top + R$. The joint-Gaussian conditioning formula gives

$$K = \bar{P}C^\top S^{-1}, \qquad \mu = \bar{\mu} + Ke, \qquad P = \bar{P} - KSK^\top.$$

The innovation is the surprising part of the measurement: if the observed measurement equals the predicted measurement, the mean stays unchanged, although uncertainty can still decrease.

1. **Predict a measurement.** Compute $C\bar{\mu}$ from the predicted state.
2. **Measure the discrepancy.** Form e in measurement coordinates, not state coordinates.
3. **Weight the discrepancy.** The gain compares state uncertainty with measurement uncertainty and includes cross-correlations.

4. **Correct the state and covariance.** Apply the innovation once, using the same predicted covariance to compute both updates.

Solve a linear system involving S instead of explicitly inverting it. A numerically robust covariance formula is the Joseph form

$$P = (I - KC)\bar{P}(I - KC)^\top + KRK^\top.$$

It is algebraically equivalent for the exact Kalman gain and more resistant to loss of symmetry or positive semidefiniteness in finite precision.

12.2.1 Worked example: predict and correct a scalar state

Let the previous position estimate have mean 0 and variance 4. Apply a known displacement $u = 1$ with $A = B = C = 1$, process variance $Q = 1$, sensor variance $R = 4$, and observe $z = 3$.

1. Prediction is $\bar{\mu} = 1$ and $\bar{P} = 4 + 1 = 5$.
2. Innovation is $e = 3 - 1 = 2$, with variance $S = 5 + 4 = 9$.
3. Gain is $K = 5/9$, so $\mu = 1 + (5/9)2 = 19/9$.
4. Posterior variance is $P = 5 - (5/9)9(5/9) = 20/9$.
5. A more accurate sensor with $R = 1$ would give $K = 5/6$, mean $8/3$, and variance $5/6$. Greater trust moves the estimate closer to 3 and reduces uncertainty more.

The estimate does not select either prediction or measurement wholesale. It combines them according to their uncertainty under the model.

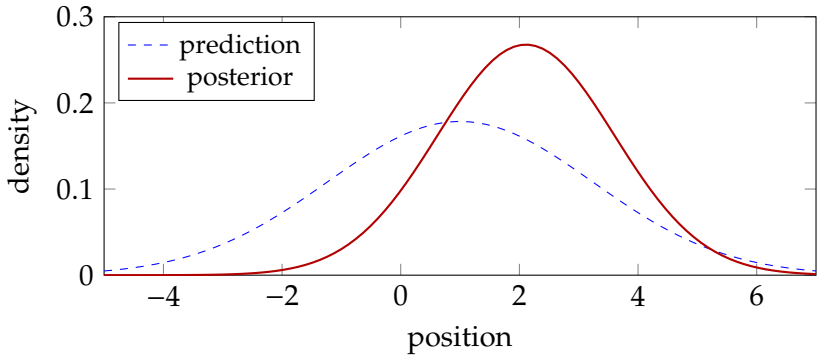


Figure 19: The measurement moves the mean and narrows the predicted distribution in the scalar example.

12.3 Cross-covariance allows indirect state correction

A position measurement can update velocity when position and velocity are correlated. Consider predicted covariance

$$\bar{P} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}, \qquad C = (1\ 0), \qquad R = 1.$$

Then $S = 5$ and $K = (4/5, 2/5)^\top$. With predicted mean $(0, 1)^\top$ and measured position 2, the posterior mean is $(1.6, 1.8)^\top$. The velocity changes even though it was not directly measured. The posterior covariance is

$$P = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix} - \frac{1}{5}\begin{pmatrix} 16 & 8 \\ 8 & 4 \end{pmatrix} = \begin{pmatrix} 0.8 & 0.4 \\ 0.4 & 2.2 \end{pmatrix}.$$

Discarding off-diagonal terms would lose this information flow.

12.4 Extended Kalman filtering

For nonlinear dynamics $x_t = f(x_{t-1}, u_t) + w_t$ and observation $z_t = h(x_t) + v_t$, the EXTENDED KALMAN FILTER (EKF) linearizes the functions near the current mean.

$$\begin{aligned} \bar{\mu}_t &= f(\mu_{t-1}, u_t), & A_t &= f_x(\mu_{t-1}, u_t), \\ \bar{P}_t &= A_t P_{t-1} A_t^\top + Q_t, & C_t &= h_x(\bar{\mu}_t), \\ e_t &= z_t - h(\bar{\mu}_t), & S_t &= C_t \bar{P}_t C_t^\top + R_t. \end{aligned}$$

Use the usual gain and correction with these quantities. The observation Jacobian is evaluated at the predicted mean, while the dynamics Jacobian is evaluated at the previous posterior mean. Mixing these points changes the approximation.

12.4.1 Worked failure case: a symmetric nonlinear transform

Let $X \sim \mathcal{N}(0, 1)$ and $Y = X^2$.

1. The true mean is $\mathbb{E}[Y] = \mathbb{E}[X^2] = 1$ and variance is $\mathbb{E}[X^4] - 1 = 3 - 1 = 2$.
2. Linearizing $f(x) = x^2$ at zero gives $f(0) + f'(0)x = 0$.
3. The EKF transform therefore predicts mean zero and variance zero before adding any explicit noise, missing both effects of the nonlinear transformation.

A nonlinear observation such as squared range can also create multiple plausible states. A single Gaussian can be inadequate even when its first two moments are well estimated. Large uncertainty relative to the region of accurate linearization makes this problem more severe.

12.5 The unscented transform

The unscented Kalman filter (UKF) transforms a deterministic set of sigma points through the actual nonlinear functions. It approximates the distribution rather than replacing the function by its tangent.

For an n-dimensional Gaussian with mean μ and covariance P, one original sigma-point rule uses $c = n + \kappa > 0$:

$$\chi_0 = \mu, \quad \chi_i = \mu + (\sqrt{cP})_{:i}, \quad \chi_{i+n} = \mu - (\sqrt{cP})_{:i},$$

with $w_0 = \kappa/c$ and $w_i = 1/(2c)$ for the other $2n$ points. The square root satisfies $LL^\top = cP$. These weights sum to one and reproduce the original mean and covariance.

1. Compute $y_i = f(\chi_i)$ without linearizing f.
2. Approximate the transformed mean by $\bar{y} = \sum_i w_i y_i$.
3. Approximate covariance by $\sum_i w_i (y_i - \bar{y})(y_i - \bar{y})^\top$, with process noise added as appropriate.
4. For measurement correction, transform predicted-state sigma points through h and compute both measurement covariance and state-measurement cross-covariance. Their ratio supplies the gain.

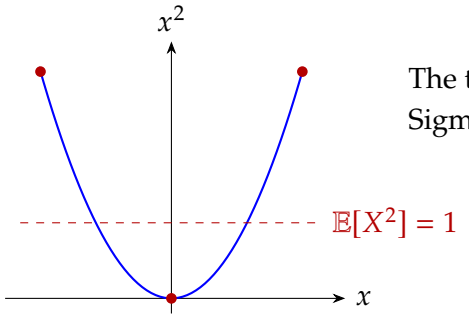
Scaled UKF variants use separate mean and covariance weights and additional parameters; their formulas should be kept consistent rather than mixed with this original rule. Some parameter choices produce negative weights, requiring attention to numerical covariance validity.

12.5.1 Worked example: recover the square's moments

For $X \sim \mathcal{N}(0, 1)$, choose $n = 1, \kappa = 2$, so points are $0, \sqrt{3}, -\sqrt{3}$ with weights $2/3, 1/6, 1/6$.

1. Transform through $f(x) = x^2$, giving $0, 3, 3$.
2. The weighted mean is $(2/3)0 + (1/6)3 + (1/6)3 = 1$.
3. Variance is $(2/3)(0-1)^2 + (1/6)(3-1)^2 + (1/6)(3-1)^2 = 2$.

This rule exactly captures the first two transformed moments in this example. It still represents the result by moments; X^2 itself is not Gaussian. Better moment propagation does not make every posterior Gaussian or every nonlinear update exact.



The tangent at zero misses curvature.
Sigma points sample both sides.

Figure 20: Symmetric sigma points preserve information that a zero-slope tangent loses.

Filter	Belief representation	Propagation mechanism	Important limitation
KF	Gaussian mean and covariance	Exact affine transformation and conditioning	Exactness requires the stated linear Gaussian model
EKF	One Gaussian approximation	Jacobians at estimated means	Can miss curvature or multiple modes
UKF	One Gaussian approximation	Nonlinear transformation of sigma points	Moment accuracy does not imply an exact posterior
Particle	Weighted state hypotheses	Sampling, likelihood weighting, resampling	Missing support and weight degeneracy

12.6 Observability, computation, and numerical limits

For constant A, C in a noiseless model, outputs satisfy $z_t = CA^t x_0$ after known inputs are removed. The initial state is identifiable exactly when

$$O = \begin{pmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{pmatrix}$$

has full column rank. This is observability, dual to controllability. With continuing process and sensor noise, observability does not imply that estimation error becomes exactly zero. Under standard detectability and noise assumptions, covariance and gain can approach steady-state values while nonzero uncertainty remains.

For dense matrices, ordinary Kalman computations are polynomial, typically including $O(n^3)$ state operations and $O(k^3)$ innovation factorization for k measurements, plus mixed matrix products. Sparsity and special structure can reduce these costs. A UKF propagates $2n + 1$ points and therefore adds multiple nonlinear function evaluations.

Square-root filters propagate covariance factors to improve numerical stability. Information filters can exploit sparse precision matrices; ensemble filters approximate large covariance structure with samples. These are representation choices, not permission to ignore model mismatch. Incorrect noise covariances, unmodeled bias, correlated measurements, and angular wraparound can make even a correctly implemented filter overconfident.

Source. Ignasi Clavera and Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 12.

Lecture 13: Smoothing, MAP, Maximum Likelihood, and EM

13.1 Filtering and smoothing answer different questions

Filtering estimates $p(x_t \mid z_{0:t})$ using observations available at time t. Smoothing estimates $p(x_t \mid z_{0:T})$ for $T > t$, allowing later evidence to revise an earlier state. This is useful for mapping, learning dynamics, and reconstructing demonstrations after data collection.

Define forward message $\alpha_t(x) = p(x_t = x, z_{0:t})$ and backward message $\beta_t(x) = p(z_{t+1:T} \mid x_t = x)$. The recursions are

$$\alpha_{t+1}(x') = p(z_{t+1} \mid x') \int p(x' \mid x)\alpha_t(x)\, dx,$$
$$\beta_t(x) = \int p(x' \mid x)p(z_{t+1} \mid x')\beta_{t+1}(x')\, dx'.$$

Initialize $\alpha_0 = p(x_0)p(z_0 \mid x_0)$ and $\beta_T = 1$. Their product, normalized, is the smoothed marginal. Discrete models replace integrals by sums. Rescaling messages prevents numerical underflow without changing normalized results.

The pairwise posterior needed for learning is proportional to

$$\alpha_t(x)p(x' \mid x)p(z_{t+1} \mid x')\beta_{t+1}(x').$$

It retains dependence between neighboring states. Multiplying their separately smoothed marginals would generally give the wrong pairwise statistics.

13.2 Kalman smoothing as a backward correction

Run a Kalman filter and store filtered (μ_t, P_t) and one-step predictions $(\bar{\mu}_{t+1}, \bar{P}_{t+1})$. For linear dynamics $x_{t+1} = A_t x_t + b_t + w_t$, define

$$G_t = P_t A_t^\top \bar{P}_{t+1}^{-1}.$$

The Rauch–Tung–Striebel backward recursion is

$$\mu_t^s = \mu_t + G_t(\mu_{t+1}^s - \bar{\mu}_{t+1}),$$
$$P_t^s = P_t + G_t(P_{t+1}^s - \bar{P}_{t+1})G_t^\top.$$

Start with the final filtered distribution, which already uses all observations. The difference $\mu^s_{t+1} - \bar{\mu}_{t+1}$ is the information learned about the next state after the original prediction; the gain transfers the relevant part backward.

13.2.1 Worked example: a future observation revises the past

Suppose $X_0 \sim \mathcal{N}(0,1)$, $X_1 = X_0 + w$ with $w \sim \mathcal{N}(0,1)$, and only $Z_1 = X_1 + v$ is observed, with $v \sim \mathcal{N}(0,1)$ and $z_1 = 3$.

1. Predict X_1 with mean 0 and variance 2. Its measurement gain is $2/3$, so filtering gives $\mu_1 = 2$, $P_1 = 2/3$.
2. The smoothing gain for time zero is $G_0 = 1/2$.
3. The old state mean becomes $\mu^s_0 = 0 + (1/2)(2 - 0) = 1$.
4. Its variance becomes $P^s_0 = 1 + (1/4)(2/3 - 2) = 2/3$.

A positive later position is evidence that the earlier position was also positive, though the process noise prevents attributing the entire change to the initial state.

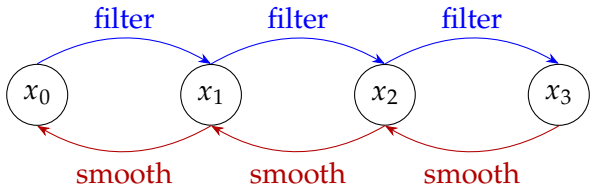


Figure 21: Filtering carries evidence forward; smoothing sends later corrections backward.

13.3 MAP trajectories and marginal estimates

A maximum a posteriori (MAP) trajectory maximizes the *joint* posterior $p(x_{0:T} \mid z_{0:T})$. Choosing the most likely state independently at each time need not maximize that joint probability. In discrete chains, Viterbi dynamic programming replaces the forward sum by a maximum and stores backpointers, taking $O(TN^2)$ time for N states in a dense chain.

In a nonsingular linear Gaussian model, the entire trajectory posterior is Gaussian. Its joint mode equals its mean, and each smoothed marginal mean is the corresponding block of that joint mean. Thus Kalman smoothing supplies the MAP trajectory as well as marginal estimates in this special setting.

Equivalently, minimize the negative log posterior:

$$\tfrac{1}{2}\|x_0 - \mu_0\|^2_{P_0^{-1}} + \tfrac{1}{2}\sum_t \|x_{t+1} - A_t x_t - b_t\|^2_{Q_t^{-1}} + \tfrac{1}{2}\sum_t \|z_t - C_t x_t\|^2_{R_t^{-1}},$$

where $\|e\|_M^2 = e^\top M e$. The resulting sparse quadratic system exposes the same neighboring-time structure as smoothing. Nonlinear models give a nonlinear least-squares problem whose local mode need not equal a posterior mean.

13.4 Maximum likelihood with observed data

Maximum likelihood (ML) chooses parameters that make the observed data probable. For independent data z_i, maximize $\prod_i p_\theta(z_i)$ or equivalently $\ell(\theta) = \sum_i \log p_\theta(z_i)$. Logarithms preserve the optimizer and turn products into sums.

- **Bernoulli.** With n_1 ones and n_0 zeros, $\ell = n_1 \log\theta + n_0 \log(1-\theta)$. Differentiation gives $\hat\theta = n_1/(n_0+n_1)$, including appropriate boundary cases when one count is zero.
- **Categorical.** Normalize each outcome count by the total. For fully observed Markov chains, normalize transition counts separately for each source state.
- **Exponential.** For density $\lambda e^{-\lambda z}$ on $z \ge 0$, $\hat\lambda = N/\sum_i z_i$, provided the sum is positive.
- **Uniform on $[0, a]$.** Likelihood is a^{-N} when $a \ge \max_i z_i$ and zero otherwise, so $\hat a = \max_i z_i$. The support dependence matters; differentiating only the smooth factor misses the boundary optimum.
- **Gaussian.** $\hat\mu = N^{-1}\sum_i z_i$ and $\hat\Sigma = N^{-1}\sum_i (z_i-\hat\mu)(z_i-\hat\mu)^\top$. The ML covariance uses N, not the unbiased-estimation divisor $N-1$.

For conditional linear Gaussians $y_i = W x_i + \epsilon_i$, maximum likelihood with common isotropic noise gives least squares. For fully observed state-space data, fit dynamics from (x_t, u_t) to x_{t+1} and the observation model from x_t to z_t; estimate noise covariances from residuals. These are separate regressions because the states are known.

13.4.1 Worked example: an exponential parameter

For observed waiting times $3.1, 8.2, 1.7$, the sum is 13 and $\ell(\lambda) = 3\log\lambda - 13\lambda$. The derivative $3/\lambda - 13$ vanishes at $\hat\lambda = 3/13$. The second derivative $-3/\lambda^2$ is negative, so this is the unique maximum. The fitted mean waiting time is $1/\hat\lambda = 13/3$, the sample mean.

13.5 Priors, MAP parameters, and validation

A prior $p(\theta)$ leads to $\theta_{\mathrm{MAP}} = \arg\max_\theta [\ell(\theta) + \log p(\theta)]$. For a Bernoulli parameter with Beta(a, b) prior and an interior posterior mode,

$$\hat\theta_{\mathrm{MAP}} = \frac{n_1 + a - 1}{n_1 + n_0 + a + b - 2}.$$

The posterior mean instead is $(n_1 + a)/(n_1 + n_0 + a + b)$. These are different estimators. A uniform Beta$(1, 1)$ prior gives the ML mode but the add-one posterior mean. A Beta$(2, 2)$ prior gives an add-one MAP formula.

With five successes and no failures, ML gives 1. Beta$(2, 2)$ MAP gives $6/7$, while its posterior mean gives $7/9$. The prior prevents a small sample from forcing absolute certainty, but its influence depends on the estimator and parameters.

A Gaussian prior on regression weights produces a quadratic penalty, connecting MAP estimation to ridge regression. Choose prior strength or regularization using held-out predictive performance. Data used to choose hyperparameters are validation data, so a separate final test set is needed for an unbiased final assessment. Time-series splits should avoid training on future information when evaluating forward prediction.

13.6 EM handles latent variables with soft information

When states or mixture labels are hidden, the likelihood includes a sum or integral over them:

$$\ell(\theta) = \log \int p_\theta(z, x)\, dx.$$

Expectation maximization (EM) alternates posterior inference for the hidden variables with parameter fitting from expected complete-data statistics.

For any normalized $q(x)$, Jensen's inequality gives

$$\ell(\theta) = \log \mathbb{E}_q\left[\frac{p_\theta(z, X)}{q(X)}\right] \geq \mathbb{E}_q[\log p_\theta(z, X)] - \mathbb{E}_q[\log q(X)] \equiv \mathcal{L}(q, \theta).$$

In fact $\ell(\theta) - \mathcal{L}(q, \theta) = \mathrm{KL}(q \| p_\theta(x \mid z)) \geq 0$.

1. **E-step.** Set $q_k(x) = p_{\theta_k}(x \mid z)$. The bound touches the true log likelihood at the current parameter.
2. **M-step.** Maximize $\mathbb{E}_{q_k}[\log p_\theta(z, X)]$ over θ. The entropy of q_k is constant during this step.
3. **Monotonicity.** Raising the bound from a point where it touches the likelihood cannot lower the likelihood. This requires an exact E-step and an M-step that actually improves the bound.

EM does not replace each hidden variable by its most likely value. It averages sufficient statistics over the posterior. It can converge to a local optimum, and mixture models with unrestricted covariances can have singular likelihoods unless regularized or constrained.

13.7 Mixture models and a complete soft-assignment example

For a Gaussian mixture with component weights π_k, means μ_k, and covariances Σ_k, responsibilities are

$$r_{ik} = \frac{\pi_k \mathcal{N}(z_i; \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(z_i; \mu_j, \Sigma_j)}.$$

Let $N_k = \sum_i r_{ik}$. Then

$$\pi'_k = N_k/N, \quad \mu'_k = \frac{\sum_i r_{ik} z_i}{N_k}, \quad \Sigma'_k = \frac{\sum_i r_{ik}(z_i - \mu'_k)(z_i - \mu'_k)^\top}{N_k}.$$

A component with essentially zero responsibility needs an explicit numerical treatment; division by zero is not an update.

13.7.1 Worked example: two uncertain labels

Use two equally weighted unit-variance Gaussians with current means 0 and 2, observing $z = 0$ and $z = 2$.

1. At $z = 0$, the density ratio of component 2 to component 1 is e^{-2}. Thus component-1 responsibility is $a = 1/(1 + e^{-2}) \approx 0.8808$.
2. At $z = 2$, the responsibilities reverse. Each component has effective count 1.
3. The new means are $\mu'_1 = 0 \cdot a + 2(1 - a) \approx 0.2384$ and $\mu'_2 = 0(1 - a) + 2a \approx 1.7616$.
4. The means move inward because each observation has some probability of belonging to the other component. Hard assignment would keep means at 0 and 2 and would be a different algorithm.

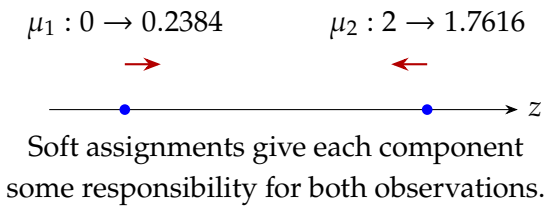


Figure 22: A soft EM update moves the means inward in the two-observation example.

13.8 EM for state-space models

For a discrete hidden Markov model, smoothing supplies expected state counts and expected transition counts. Normalize outgoing transition counts to update the transition matrix, and normalize state-conditioned observation counts to update the emission model.

For a linear Gaussian system without controls, smoothing supplies $\mathbb{E}[x_t]$, $\mathbb{E}[x_t x_t^\top]$, and $\mathbb{E}[x_{t+1} x_t^\top]$. The dynamics M-step solves

$$A' = \left(\sum_t \mathbb{E}[x_{t+1} x_t^\top] \right) \left(\sum_t \mathbb{E}[x_t x_t^\top] \right)^{-1},$$

with a suitable solve or pseudoinverse when needed. The process covariance becomes

$$Q' = \frac{1}{T} \sum_t \mathbb{E}[(x_{t+1} - A' x_t)(x_{t+1} - A' x_t)^\top].$$

Observation parameters have analogous expected-regression formulas. With controls, augment predictors by the known input and fit the corresponding blocks jointly.

Using only smoothed means would omit covariance. For instance, $\mathbb{E}[x_t x_t^\top] = P_t^s + \mu_t^s (\mu_t^s)^\top$, so uncertainty contributes directly to the normal equations. The lag covariance contributes to the cross moment as well.

A useful likelihood diagnostic comes from the Kalman innovations:

$$\log p(z_{0:T}) = -\tfrac{1}{2} \sum_t \left[k \log(2\pi) + \log |S_t| + e_t^\top S_t^{-1} e_t \right].$$

For exact linear-Gaussian EM it should not decrease apart from numerical error. An EKF-based approximate E-step loses the exact monotonicity guarantee. Interpolating parameter updates and checking the likelihood can help, while preserving valid covariance matrices and acknowledging that the evaluated nonlinear likelihood may itself be approximate.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 13.

Lecture 14: Particle Filters and Monte Carlo Localization

14.1 Represent a belief by weighted hypotheses

A Gaussian represents one elliptical concentration of probability. A robot in a repeated hallway may instead have several separated location hypotheses. A particle filter approximates the belief by states $x_t^{(i)}$ with nonnegative normalized weights $w_t^{(i)}$:

$$b_t(dx) \approx \sum_{i=1}^{N} w_t^{(i)} \delta_{x_t^{(i)}}(dx).$$

The symbol δ_x is a unit point mass at x. An expectation becomes $\mathbb{E}_b[f(X)] \approx \sum_i w_i f(x_i)$. The particles are a numerical representation of uncertainty, not multiple physical robots.

More particles in a region, or greater total weight there, indicate more probability. A finite particle set is not an exact arbitrary density; its ability to represent a mode depends on whether particles actually reach that mode.

14.2 Rejection sampling and importance sampling

Rejection sampling proposes x and accepts it using an envelope that bounds the target. For a target proportional to $f(x)$ on a bounded region and uniform proposals, draw a height uniformly between zero and an upper bound $M \geq f(x)$ and accept if it lies below $f(x)$. Poor envelopes waste samples, especially in high dimensions.

Importance sampling instead keeps every proposal and corrects its contribution. If the target density is proportional to $f(x)$ and proposals have density $q(x)$, use unnormalized weight $\tilde{w} = f(x)/q(x)$, then normalize across samples. The proposal must have positive density wherever the target has positive density. Otherwise missing regions cannot be recovered by any finite weight.

For conditionally independent landmark observations, a proposal based on one landmark, $q(x) = p(x \mid z_1)$, already incorporates that observation. The remaining importance weight is proportional to $\prod_{k \neq 1} p(z_k \mid x)$, with prior and first-likelihood terms canceling. Multiplying the first landmark again would double-count it.

14.2.1 Worked example: correct a biased proposal

Let target probabilities on states A, B be $(0.8, 0.2)$ and proposal probabilities be $(0.5, 0.5)$.

1. The importance factors are $0.8/0.5 = 1.6$ and $0.2/0.5 = 0.4$.
2. If one sample of each state is drawn, normalized weights are $1.6/2 = 0.8$ and $0.4/2 = 0.2$.
3. A plain unweighted average would incorrectly give equal probability to the states. The ratio corrects how often the proposal visits each state.

If the proposal never samples B, the support condition fails. Weighting cannot create a missing hypothesis.

14.3 Deriving the bootstrap particle filter

The Bayes prediction is a mixture of transition distributions from previous states. A bootstrap filter samples that mixture directly:

1. Select an ancestor index j with probability $w_{t-1}^{(j)}$.
2. Propagate $x_t^{(i)} \sim p(x_t \mid x_{t-1}^{(j)}, u_t)$.
3. Weight the new state by $\tilde{w}_t^{(i)} = p(z_t \mid x_t^{(i)})$.
4. Normalize the weights. Keep the weighted set, or resample to obtain an equally weighted set for the next stage.

The weight is just the likelihood because the proposal is already the predictive mixture. This ordering resamples ancestors before propagation. An equivalent common implementation propagates previously equally weighted particles, weights them, and resamples afterward. Do not combine both conventions by accidentally multiplying already-resampled old weights a second time.

For a general proposal $q(x_t \mid x_{t-1}, z_t, u_t)$ without prior resampling, the update is

$$\tilde{w}_t^{(i)} = w_{t-1}^{(i)} \frac{p(z_t \mid x_t^{(i)}) p(x_t^{(i)} \mid x_{t-1}^{(i)}, u_t)}{q(x_t^{(i)} \mid x_{t-1}^{(i)}, z_t, u_t)}.$$

This makes the cancellation and the role of old weights explicit.

14.3.1 Worked example: one measurement update

Suppose four propagated particles have equal prior weights. Their observation likelihoods are 0.1, 0.4, 0.2, and 0.3.

1. Their likelihoods sum to one, so the posterior weights are already $(0.1, 0.4, 0.2, 0.3)$.
2. For state values $(0, 1, 2, 3)$, the weighted mean is $0(0.1) + 1(0.4) + 2(0.2) + 3(0.3) = 1.7$.

3. If these states instead described two widely separated physical modes, the mean could fall in an implausible location. Preserve the multimodal belief rather than treating its mean as a guaranteed feasible pose.

14.4 Systematic resampling in linear time

Resampling draws particles according to their current weights and resets each selected copy to weight $1/N$. It reallocates computation toward plausible states; it does not add new observation information.

Algorithm 14.1: Systematic resampling

Input: Particles $x_1, \ldots, x_N$ and normalized weights $w_1, \ldots, w_N$

$c_i \leftarrow \sum_{j=1}^{i} w_j$ for all i
Sample $r \sim \text{Uniform}[0, 1/N)$; set $i \leftarrow 1$
for $j = 0, 1, \ldots, N-1$ **do**
  $u \leftarrow r + j/N$
  while $u > c_i$ **do**
    $i \leftarrow i + 1$
  Output a copy of x_i with weight $1/N$

Thresholds are ordered, so the cumulative pointer advances at most $N-1$ times. The total cost is $O(N)$, compared with $O(N \log N)$ for independent draws using binary search. Set the final cumulative probability to one numerically to avoid roundoff sending the pointer out of range.

For the weights above, cumulative values are $(0.1, 0.5, 0.7, 1)$. With illustrative offset $r = 0.08$, thresholds are 0.08, 0.33, 0.58, and 0.83. They select particles $1, 2, 3, 4$. With $r = 0.18$, thresholds $0.18, 0.43, 0.68, 0.93$ select $2, 2, 3, 4$. A high-weight particle can receive multiple copies, and a lower-weight particle can disappear.

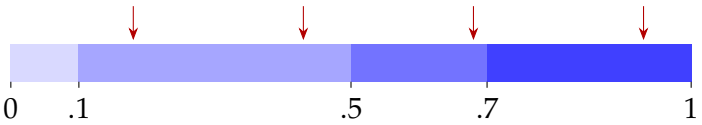


Figure 23: Equally spaced random thresholds sample the cumulative weight intervals.

14.5 Degeneracy, localization, and recovery

The effective sample-size diagnostic $N_{\text{eff}} = 1/\sum_i w_i^2$ ranges from 1 to N. It is N for equal weights and near one when one particle dominates. Resampling when this diagnostic is small can avoid needless resampling at every step. Repeated copies, however, reduce diversity; deterministic dynamics can then leave many identical particles.

For mobile-robot localization, each particle is a pose (x, y, θ). The motion model spreads hypotheses according to odometry uncertainty. A sensor model scores predicted ranges or visual features

against the actual observation. Repeated structures naturally create several modes, which later motion and observations may distinguish.

- **Tracking** begins near the correct pose and maintains a local estimate.
- **Global localization** initializes broadly and searches among many possible poses.
- **Kidnapped-robot recovery** must recover after the actual pose jumps outside the supported particle cloud.

A standard filter cannot recover a lost region if its proposal never reaches it. Injecting occasional global particles adds support for unexpected relocation. Increasing injection when average observation likelihood becomes unusually low can adapt recovery effort, though likelihood scale and model mismatch must be considered. Zero total weight requires an explicit recovery action, not division by zero.

Products of many small sensor likelihoods can underflow. Accumulate log weights, subtract their maximum, exponentiate, and normalize. Correlated sensor beams should not be treated as arbitrarily many independent confirmations. In high dimensions or with very sharp likelihoods, most particles may receive negligible weight, motivating better proposals and more structured representations rather than assuming a fixed small particle count will suffice.

Source. Wolfram Burgard, *Advanced Robotics*, Fall 2019, Lecture 14.

Lecture 15: POMDPs and Planning in Belief Space

15.1 Actions must depend on what is known

A partially observable MDP (POMDP) adds observations to an MDP. The physical state is hidden, so a policy cannot directly use it. A belief $b(s) = \Pr(S = s \mid \text{history})$ summarizes the relevant history under the model. The policy maps beliefs to actions, including actions whose main benefit is information.

Let $P(s' \mid s, a)$ be transition probability and $O(o \mid s', a)$ the likelihood of observation o after acting. The belief update is

$$\bar{b}_a(s') = \sum_s P(s' \mid s, a) b(s),$$
$$p(o \mid b, a) = \sum_{s'} O(o \mid s', a) \bar{b}_a(s'),$$
$$b_{a,o}(s') = \frac{O(o \mid s', a)\bar{b}_a(s')}{p(o \mid b, a)}.$$

An observation with zero probability under the model does not have a defined posterior through this formula. It calls for checking the model or numerical representation.

15.2 The belief MDP and contingent lookahead

Beliefs form a continuous state space even when the physical state set is finite. The immediate expected reward is $r(b, a) = \sum_s b(s)\bar{r}(s, a)$. Bellman recursion becomes

$$V_h(b) = \max_a \left[r(b, a) + \gamma \sum_o p(o \mid b, a) V_{h-1}(b_{a,o}) \right].$$

The future maximum is evaluated separately after each possible observation. This is what allows sensing to be useful: the agent can choose a different action after learning something.

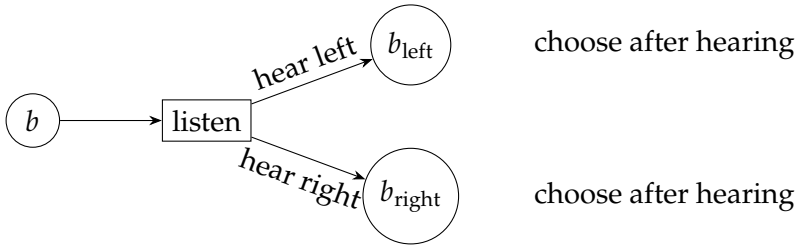


Figure 24: Information has value because later decisions can depend on its outcome.

A horizon-h tree branches over both actions and observations, growing exponentially in h. Belief discretization, point-based value methods, and sampling-based online search reduce computation with different approximation tradeoffs. Replacing the whole belief by its most likely state is cheaper but discards the reason to gather information.

15.3 Worked example: the tiger problem

A tiger is behind one of two doors. Opening its door gives reward -100; opening the other gives 10. Listening costs 1 and reports the tiger's side correctly with probability 0.85. Assume listening does not move the tiger, opening ends the episode, $\gamma = 1$, and at most two decisions remain.

1. **Act immediately at equal belief.** With $b(\text{left}) = 1/2$, opening either door gives $\frac{1}{2}(10)+\frac{1}{2}(-100) = -45$.
2. **Update after hearing left.** Bayes' rule gives $b(\text{left} \mid \text{hear left}) = 0.85$, since the report probability is 0.5 by symmetry.
3. **Choose the safer door.** Open right. Its expected reward is $0.85(10) + 0.15(-100) = -6.5$. Hearing right produces the symmetric decision.
4. **Evaluate the two-step sensing plan.** Its value is $-1 + \frac{1}{2}(-6.5) + \frac{1}{2}(-6.5) = -7.5$, much better than -45 if opening is required after this listen.

If the action set permits spending the final decision listening with no terminal obligation to open, listening on the last step yields -1, which exceeds -6.5. Then the exact two-step optimum at equal belief is to listen twice for total -2. Terminal conditions matter; a forced-opening calculation must not be presented as the unconstrained finite-horizon optimum.

With one forced opening remaining and left-tiger probability p, opening right yields $110p - 100$, while opening left yields $10 - 110p$. At $p = 0.85$, these are -6.5 and -83.5. A high probability can still be insufficient when the failure penalty is large. Confidence thresholds come from rewards and horizon, not from a universal rule such as "act when more than fifty percent sure."

15.4 Why most-likely-state control can miss sensing

Suppose the state estimate slightly favors tiger-left, with probability 0.51. A controller that treats that state as certain opens right, whose actual expected reward is $110(0.51) - 100 = -43.9$. It has thrown away the 0.49 probability of a catastrophic outcome.

A fully observed MDP policy has no reason to listen when the tiger's location is already known. Applying that policy to a guessed state therefore cannot recover the missing information-seeking behavior. This is a structural limitation of the approximation, not an error in the underlying fully observed planner.

15.5 Gaussian belief-space planning

For continuous robots, represent belief approximately by mean μ and covariance Σ. A symmetric $n \times n$ covariance has $n(n+1)/2$ unique entries, so the belief state has $n + n(n+1)/2$ parameters. At $n = 6$, this is $6 + 21 = 27$.

An EKF-like prediction and update define approximate belief dynamics. The covariance depends on where the robot goes because observation quality and visibility change. A path that passes a landmark may be longer in state space yet preferable in belief space because it reduces uncertainty before a narrow passage or a grasp.

1. Specify a cost involving task progress, effort, and relevant uncertainty or collision risk.
2. Propagate mean and covariance under candidate actions and sensor models.
3. Optimize the resulting belief trajectory by shooting or collocation.
4. During execution, update with the actual observation and replan from the actual posterior.

A common deterministic approximation assumes future observations equal their predicted values, so innovations vanish along the nominal mean. Covariance can still decrease. This approximation is computationally convenient, but it does not enumerate all possible future beliefs or exactly solve the POMDP.

15.5.1 Worked example: a detour for information

Suppose a robot must pass a narrow gate. Route A costs 2 and leaves lateral standard deviation 0.5. Route B costs 3 but passes a landmark that reduces it to 0.1. For illustrative objective $J = \text{route cost} + 10\sigma^2$, A scores $2 + 10(0.25) = 4.5$, while B scores $3 + 10(0.01) = 3.1$.

The detour is chosen because of the specified uncertainty penalty. If the task had a wide unobstructed corridor, that penalty might overvalue localization. A task-relevant chance constraint can be more meaningful than minimizing covariance everywhere indiscriminately.

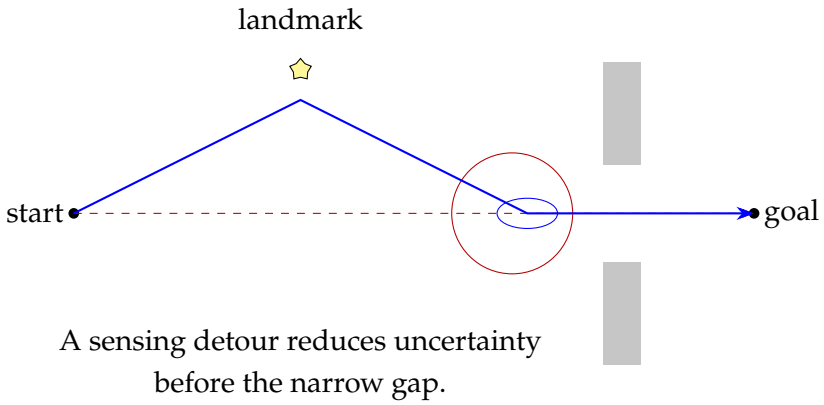


Figure 25: Belief-space planning can prefer a longer path with more useful observations; geometry is schematic.

15.6 Sensing discontinuities and absence of evidence

Visibility changes abruptly when a landmark enters a field of view or becomes occluded. A local optimizer in an entirely unobserved region can see zero gradient toward a useful sensing region. A homotopy method first replaces the hard visibility boundary by a smooth transition, optimizes that easier problem, and gradually sharpens the transition toward the actual model.

Signed distance to a sensing boundary supplies a useful smooth surrogate. The final solution must be reintegrated and checked with the intended visibility rule; success on the smoothed surrogate does not establish success under hard occlusion.

No detection can itself be evidence. Suppose a sensor detects an object perfectly whenever its state lies in visible region V. Then

$$p(x \mid \text{no detection}) \propto \mathbf{1}\{x \notin V\} p(x).$$

The posterior is a truncated distribution, generally not Gaussian. Treating no detection as "skip the update" loses this information when visibility is state-dependent. Conversely, a sensor with nonzero missed-detection probability needs that likelihood, rather than a hard truncation.

15.7 Collision risk, sigma hulls, and continuous motion

For uncertainty along one normal direction, suppose clearance $D \sim \mathcal{N}(\mu_D, \sigma_D^2)$. Requiring $\Pr(D \le 0) \le \delta$ gives

$$\mu_D \ge \Phi^{-1}(1-\delta)\sigma_D,$$

where Φ is the standard normal CDF. This reduction assumes the scalar clearance model is appropriate. Nonlinear articulated geometry can violate a simple Gaussian clearance approximation.

A SIGMA HULL takes the convex hull of a robot link's placements at selected joint-space sigma points. Checking obstacles against this hull accounts for several uncertainty directions. It is an approximation to uncertain geometry, not a universal probability certificate: finite sigma points do not automatically cover the full Gaussian support or every nonlinear transformed confidence region.

Likewise, collision checking at discrete times can miss motion between them. Conservative swept-volume bounds and appropriate interpolation assumptions are needed. Larger uncertainty margins generally increase clearance but may prevent a feasible route, revealing the tradeoff between risk and task completion.

15.8 The separation principle and its boundary

In the unconstrained linear-Gaussian-quadratic (LQG) setting, estimation and control separate. Compute the fully observed LQR gains offline, run a Kalman filter online, and apply the LQR law to the posterior mean:

$$u_t = -K_t \mu_t.$$

The control-independent covariance recursion means there is no extra benefit from choosing an action solely to improve estimation in this special model. The expected cost still contains uncertainty terms, but they do not alter the minimizing action.

State-dependent sensing, nonlinear dynamics, contact, constraints, and nonquadratic penalties can break that argument. A robot moving a camera to reveal an occluded object is precisely a case where actions affect future information, so treating the LQG separation rule as universally optimal would omit a central planning decision.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 15.

Lecture 17: Imitation Learning

17.1 Learning actions or learning intentions

Imitation learning uses expert behavior to reduce the need for trial-and-error discovery. Given demonstrations (s_t, a_t), there are two main questions: what action would the expert take here, and what objective explains the expert's behavior?

- **Behavioral cloning** fits a policy directly from observed state-action pairs.
- **Inverse reinforcement learning** infers a reward, then solves a control problem using that reward.

The distinction matters when the robot's body differs from the demonstrator's, the expert is imperfect, or the deployment environment requires a new path to the same goal. Copying an action and pursuing an inferred intention need not produce the same behavior.

17.2 Behavioral cloning and distribution shift

For a stochastic policy, a standard cloning objective is

$$\min_{\theta} - \sum_{(s,a)\in D} \log \pi_\theta(a \mid s).$$

For deterministic continuous actions, squared error $\sum \|\pi_\theta(s) - a\|^2$ is common, corresponding to a fixed-variance Gaussian output model. Multiple valid expert actions can make the conditional distribution multimodal; squared loss may average them into an invalid action.

Training states come from the expert's trajectories, while deployment states come from the learned policy. A small action error can move the robot into a state missing from the demonstrations, where the next error is larger. This is covariate shift generated by the policy's own actions.

17.2.1 Worked example: small steering errors accumulate

Imagine an illustrative lane-centering task with lateral dynamics $y_{t+1} = y_t + u_t$. Expert demonstrations mostly have $y_t \approx 0$ and correction $u_t \approx 0$. A cloned policy with a constant bias $u_t = 0.02$ starts correctly but reaches $y_{50} = 1$ after fifty steps.

1. The action error on each expert state is small.
2. The deployed state distribution moves away from the training region.

3. A dataset containing only centered driving does not teach recovery from $y = 1$.
4. Adding off-center observations with expert recovery labels addresses the missing behavior more directly than repeatedly fitting the centered examples.

Under a simplified model with per-step error probability ε on expert states and potentially costly behavior after the first error, the chance of having erred by time t is at most $t\varepsilon$. Summing possible future losses gives an $O(T^2\varepsilon)$ worst-case horizon dependence. This bound explains the mechanism; it is not a claim that every cloning policy actually exhibits quadratic error growth.

17.3 DAgger: ask for labels where the learner goes

Dataset Aggregation (DAgger) changes the training distribution by collecting expert actions at learner-visited states.

Algorithm 17.1: Dataset Aggregation

Input: Expert policy π^*, initial policy, interaction budget
$D \leftarrow$ initial expert demonstrations
for each aggregation round **do**
    Roll out the current learner, optionally mixing in expert control
    For each visited state s, query the expert action $\pi^*(s)$
    Add all queried pairs $(s, \pi^*(s))$ to D
    Train a new policy on the aggregated dataset
Select a policy using appropriate validation rollouts

An expert label says what should be done at the observed state, even when the learner caused that state. Aggregation retains earlier recovery examples rather than replacing them with only the latest round. Expert intervention or a mixing schedule may be needed while early policies are poor.

For the lane example, if the learner reaches $y = 0.4$, an expert might label a corrective $u = -0.2$. The important data point is $(0.4, -0.2)$, not the learner's erroneous action. Querying an expert can be expensive or impossible for some observations; that is a practical limitation of the method.

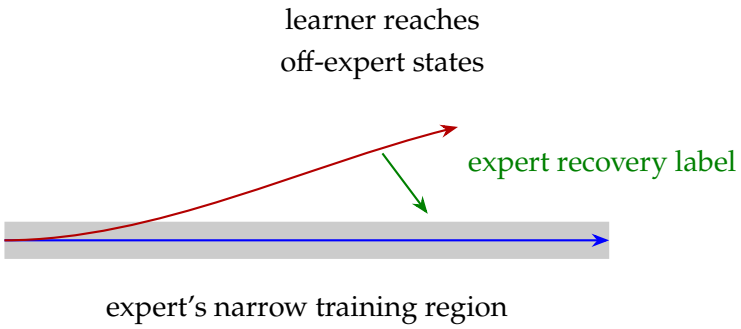


Figure 26: DAgger adds expert labels at states visited by the learner, including recovery situations.

17.4 Inverse RL and feature expectations

Suppose reward is linear in features, $r_w(s) = w^\top \phi(s)$. Define discounted feature expectation

$$\mu(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \phi(S_t) \right].$$

The expected return is $J_w(\pi) = w^\top \mu(\pi)$. Demonstrations estimate μ_E, the expert's feature expectation. If a candidate policy matches those features, it matches the expert's return for every reward in the specified linear class.

For dual norms, Hölder's inequality gives

$$|J_w(\pi) - J_w(E)| \le \|w\|_* \|\mu(\pi) - \mu_E\|.$$

For example, $\|w\|_1 \le 1$ and maximum feature mismatch at most ε imply return mismatch at most ε. The guarantee concerns the chosen features; omitted preferences such as fragility or social comfort remain unidentified.

17.4.1 Worked example: matching what a reward can express

Let features be discounted progress and discounted hazardous steps. Expert expectation is $(10, 1)$, while candidate A has $(10.2, 1.1)$ and candidate B has $(12, 4)$. For reward weights $(1, -3)$, returns are 7, 6.9, and 0.

1. A is close in both features and nearly matches the expert's reward.
2. B achieves more progress but incurs enough hazardous steps to be much worse.
3. If hazard had been omitted from the feature vector, the inference problem could incorrectly favor B. Reward identifiability depends on representation.

17.5 Apprenticeship learning and reward ambiguity

Many rewards explain the same demonstrations. The zero reward makes every policy optimal; adding a constant or scaling by a positive constant often leaves preferred actions unchanged. Recovering the demonstrator's uniquely true reward is therefore generally impossible from behavior alone.

A max-margin apprenticeship procedure chooses weights that separate the expert from policies found so far:

$$\max_{w,\rho} \rho \quad \text{subject to} \quad w^\top(\mu_E - \mu(\pi_i)) \ge \rho \text{ for every } i, \quad \|w\| \le 1.$$

1. Begin with a candidate policy and estimate its feature expectation.
2. Find a reward under which the expert most outperforms all current candidates.
3. Solve the forward control problem for that reward and add the resulting policy.
4. Continue until no substantial separation remains, possibly returning a mixture of candidate policies to match expert features.

A mixture samples which policy to follow, so its expected feature vector is the corresponding convex combination. It is not generally equivalent to averaging the policies' raw actions at every state. Imperfect demonstrations can require slack variables or a model of expert suboptimality.

17.6 Maximum-entropy inverse RL

A probabilistic model of expert trajectories avoids committing to one exactly optimal path. In a simple fixed-horizon, deterministic-dynamics setting, use

$$p_w(\tau) = \frac{\exp(w^\top \Phi(\tau))}{Z(w)}, \qquad \Phi(\tau) = \sum_t \phi(s_t, a_t).$$

More general formulations include base trajectory probabilities and use causal entropy to avoid letting the policy depend on future randomness. The partition function $Z(w)$ normalizes over feasible trajectories.

The demonstration log-likelihood gradient is

$$\nabla_w \ell = \sum_{\tau \in D} \Phi(\tau) - |D| \mathbb{E}_{p_w}[\Phi(\tau)].$$

To derive it, differentiate the linear trajectory score, then differentiate $\log Z$: its gradient is the model's expected feature vector. Learning increases reward weights for features more common in demonstrations than in model-generated behavior.

For an illustrative two-trajectory model with feature values 0 and 1, $p_w(\Phi = 1) = e^w/(1 + e^w)$. If the demonstrations use the second trajectory 80% of the time, matching the expectation gives $e^w/(1 + e^w) = 0.8$, hence $w = \log 4$. The reward is a probabilistic explanation within the model, not proof that the expert consciously used that numerical objective.

17.7 Demonstration interfaces and richer generalization

17.7.1 VR teleoperation and auxiliary predictions

Manipulation demonstrations need a usable interface. Kinesthetic teaching can put a person's hand in the camera view; teleoperation can provide unobstructed observations and continuous control labels. A policy may combine images with robot proprioception, such as end-effector position.

Auxiliary losses can ask the network to predict current or final task-relevant poses in addition to actions. A combined objective $L = L_{\text{clone}} + \lambda L_{\text{aux}}$ encourages a representation useful for the goal. The auxiliary task uses labels available from the demonstrations and can improve sample efficiency, but poor weighting can distract from action prediction. Performance must be checked through execution rather than action loss alone.

17.7.2 One-shot imitation is conditioned on a learned prior

A policy $\pi(a \mid s, D_{\text{new}})$ can use a new demonstration as task context. Demonstration, context, and manipulation networks can attend to relevant objects and time segments. Training on pairs of demonstrations teaches how one example identifies a task and another shows successful execution.

For block construction, the context might specify which blocks should be stacked, while the current observation determines which step is next. One-shot performance does not mean learning all robotics from one example: substantial cross-task training has supplied a prior over relevant tasks and object relationships.

17.7.3 Third-person observation and embodiment gaps

A human video may lack robot actions and may show a different viewpoint or body. Context translation can map demonstrations into a representation of the robot's scene, then use a learned similarity objective to guide behavior. Meta-learning can train rapid adaptation across such contexts.

Viewpoint, background, object positions, and morphology create distinct ambiguities. Matching raw pixels would reward copying the camera view rather than completing the task. A useful representation preserves task progress while ignoring irrelevant appearance changes. Its success depends on the diversity and correspondences available during training, and should be evaluated on genuinely new contexts.

Source. Laura Smith, *Advanced Robotics*, Fall 2019, Lecture 17.

Lecture 18: Policy Gradients and Trust-Region Methods

18.1 Optimize behavior when the dynamics are unknown

Reinforcement learning chooses actions using experience rather than a supplied transition model. A parameterized stochastic policy $\pi_\theta(a \mid s)$ directly represents behavior. Its objective is expected return

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)], \qquad R(\tau) = \sum_{t=0}^{H-1} \gamma^t r_t.$$

A trajectory τ includes states and actions. Assuming the environment dynamics do not depend on θ,

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^{H-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Policy optimization avoids a potentially difficult continuous-action maximum over a learned Q function, but sampling complete trajectories can be costly.

18.2 Finite differences and common random numbers

A black-box estimate of coordinate i is

$$\frac{J(\theta + \varepsilon e_i) - J(\theta - \varepsilon e_i)}{2\varepsilon}.$$

There are $2d$ objective evaluations for d parameters, and each evaluation itself needs trials. Small ε reduces truncation bias but amplifies measurement noise through division by ε.

If paired simulated trials use the same random disturbances, their outcomes can be positively correlated. The variance of their difference is $\operatorname{Var}(X - Y) = \operatorname{Var} X + \operatorname{Var} Y - 2\operatorname{Cov}(X, Y)$, so this common-random-number coupling can improve comparisons. It is not a guarantee when coupling has the wrong effect, and physical wind patterns usually cannot be replayed exactly. Fixing simulator randomness also risks overfitting to a small set of seeds; evaluate on fresh randomness.

18.2.1 Evolutionary search and CEM over policy parameters

A derivative-free alternative samples whole parameter vectors, runs each policy, and retains good candidates. CEM fits a new sampling distribution to the elite parameter vectors, repeating until its interaction budget is exhausted. This is the same sampling principle used for action-sequence optimization, but the variables now define an entire feedback policy.

For example, if a scalar gain is sampled at $-1, 0, 1, 2$ and the best two gains are 1 and 2, their fitted Gaussian mean is 1.5 and maximum-likelihood variance 0.25. Multiple rollouts per candidate can reduce ranking noise. Evaluating one lucky rollout per gain risks selecting noise rather than good behavior. Parameter search is simple and parallelizable in principle, but does not directly exploit which time-local decision caused a return difference.

18.3 The likelihood-ratio derivation

The identity $\nabla p = p \nabla \log p$ moves the derivative inside the trajectory expectation:

$$\begin{aligned}\nabla_\theta J &= \int R(\tau) \nabla_\theta p_\theta(\tau)\, d\tau \\ &= \mathbb{E}_\theta[R(\tau) \nabla_\theta \log p_\theta(\tau)] \\ &= \mathbb{E}_\theta \left[R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right].\end{aligned}$$

The initial-state and dynamics terms vanish from the log derivative because they contain no policy parameters. No derivative of the unknown environment is needed. The argument assumes appropriate smoothness and support conditions for interchanging differentiation and integration.

18.3.1 Worked example: a Bernoulli policy gradient

A one-step policy chooses action 1 with probability $p = \sigma(\theta)$ and action 0 otherwise. Rewards are 2 and 0.

1. The exact objective is $J = 2p$, so its derivative is $2p(1 - p)$.
2. The score is $\partial_\theta \log \pi(a) = a - p$.
3. The likelihood-ratio expectation is $p\,[2(1 - p)] + (1 - p)[0(-p)] = 2p(1 - p)$, matching direct differentiation.
4. At $p = 1/2$, a sampled rewarding action gives gradient estimate 1, while the other action gives 0. Their average is $1/2$, the exact gradient, but individual samples are noisy.

Multiplying each sampled score by return increases probabilities of actions associated with better outcomes, in expectation. It does not imply that every action in one lucky trajectory was beneficial.

18.4 Reward-to-go and baselines reduce variance

Rewards that occurred before action a_t cannot be caused by that action. Their expected score contribution is zero. Define reward-to-go $G_t = \sum_{k=t}^{H-1} \gamma^{k-t} r_k$. Then

$$\nabla J = \mathbb{E}\left[\sum_t \gamma^t \nabla \log \pi(a_t \mid s_t) G_t\right].$$

The outer γ^t corresponds to the explicitly discounted finite-horizon objective above. Some policy-gradient conventions absorb discounting into the sampled state distribution; formulas must use one convention consistently.

Any action-independent baseline $b_t(s_t)$ can be subtracted without changing the exact expectation:

$$\mathbb{E}_{a\sim\pi}[b(s)\nabla \log \pi(a \mid s)] = b(s)\nabla \sum_a \pi(a \mid s) = 0.$$

If a learned baseline shares parameters with the policy, treat its numerical value as fixed inside this score-function estimator; differentiating it there adds an unrelated term.

18.4.1 Worked example: a baseline removes all variance

Return to the one-step Bernoulli example at $p = 1/2$. Use baseline $b = 1$.

1. Action 1 has score $1/2$ and centered reward $2 - 1 = 1$, giving estimate $1/2$.
2. Action 0 has score $-1/2$ and centered reward $0 - 1 = -1$, also giving $1/2$.
3. Both possible samples now equal the exact gradient, so the variance is zero in this special case.

A good baseline need not be zero and need not equal a successful reward. For a scalar constant baseline minimizing squared norm of the estimator, the optimum weights returns by squared score magnitude:

$$b^* = \frac{\mathbb{E}[R\|\nabla \log \pi\|^2]}{\mathbb{E}[\|\nabla \log \pi\|^2]}.$$

The ordinary mean return is a practical approximation, not universally the variance-minimizing constant.

18.5 Critics, bootstrapping, and advantage estimates

Define $V^\pi(s)$ as expected future return and $Q^\pi(s, a)$ as expected return after action a. Their difference $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ is advantage. It asks how much better the action is than the policy's usual behavior at that state.

A critic can be fit to Monte Carlo returns, which use complete sampled futures, or to bootstrapped targets $r_t + \gamma V_\phi(s_{t+1})$. Bootstrapping lowers reliance on a long noisy return but introduces error from the approximate value function.

An n-step advantage estimate is

$$\hat{A}_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n}) - V(s_t),$$

with terminal handling when an episode ends earlier. A true terminal state has no continuation value. A time-limit truncation may still have continuation and should not automatically be treated as a terminal outcome.

Generalized advantage estimation (GAE) uses temporal-difference residuals

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \qquad \hat{A}_t^{\mathrm{GAE}} = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}.$$

For a finite rollout, compute backward as $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$ with appropriate continuation masks. Small λ emphasizes short bootstrapped estimates; λ near one uses longer returns. The tradeoff depends on critic accuracy and rollout length.

18.5.1 Worked example: compute GAE backward

Take two steps, rewards $(1, 2)$, value estimates $(V_0, V_1, V_2) = (1, 1, 0)$, discount 0.9, and $\lambda = 0.8$, with the final state terminal.

1. $\delta_1 = 2 + 0 - 1 = 1$ and $\delta_0 = 1 + 0.9(1) - 1 = 0.9$.
2. $\hat{A}_1 = 1$. Then $\hat{A}_0 = 0.9 + (0.9)(0.8)(1) = 1.62$.
3. At $\lambda = 1$, $\hat{A}_0 = 1.8$, equal to Monte Carlo return $1 + 0.9(2)$ minus $V_0 = 1$.
4. At $\lambda = 0$, $\hat{A}_0 = 0.9$, using only the one-step residual.

Actor-critic algorithms update the policy with these advantages and fit a value network to suitable targets. A3C collects data through asynchronous workers with short multi-step estimates; parallelism improves throughput but requires handling stale parameters and differing trajectories.

18.6 Why policy steps need special care

In supervised learning, a poor parameter update usually leaves the data distribution unchanged. In on-policy RL, a poor update changes the policy that collects the next data. It can lose useful behavior and visit a much worse part of the environment.

A trust region constrains the change in action distributions. Near the current parameters, average KL divergence is approximately $\frac{1}{2}d^\top Fd$, where F is a Fisher matrix. For estimated gradient g, solve

$$\max_d g^\top d \quad \text{subject to} \quad \tfrac{1}{2}d^\top Fd \le \varepsilon.$$

When F is positive definite, the direction is $F^{-1}g$ and its scaled solution is

$$d = \sqrt{\frac{2\varepsilon}{g^\top F^{-1} g}}\, F^{-1} g.$$

This follows by a Lagrange multiplier: stationarity gives $g - \lambda Fd = 0$, then the active quadratic constraint fixes the scale.

For an illustrative diagonal $F = \mathrm{diag}(1, 4)$ and $g = (1, 1)$, the natural direction is $(1, 1/4)$ rather than $(1, 1)$. The second parameter changes the policy more strongly according to the metric, so the step changes it less.

18.7 TRPO and the importance-weighted surrogate

Using trajectories from old policy π_{old}, define the action probability ratio

$$r_\theta(s, a) = \frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)}.$$

A surrogate objective is $L(\theta) = \mathbb{E}_{\text{old}}[r_\theta \hat{A}_{\text{old}}]$. Its gradient matches the old-policy gradient at the old parameters, while allowing several candidate updates to be evaluated using the same samples.

TRPO approximately maximizes this surrogate subject to an average KL bound under old-policy states. The full trajectory likelihood ratio includes products across time; the action-wise surrogate instead uses old state visitation, which becomes less accurate as the policy changes. The trust region is intended to limit that mismatch.

A large neural policy makes an explicit Fisher matrix expensive. Conjugate gradients use Fisher-vector products to approximately solve for the natural direction, followed by a line search checking surrogate improvement and KL. Sampled average KL constraints and approximate solves are practical approximations, so theoretical monotonic-improvement results do not automatically hold exactly for every implementation and batch.

The surrogate also follows from importance sampling. If the new trajectory distribution has support covered by the old one,

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\text{old}}}\left[\frac{p_\theta(\tau)}{p_{\text{old}}(\tau)} R(\tau)\right].$$

The dynamics cancel in the ratio, leaving a product of policy-action ratios. Differentiating at the old parameters recovers the likelihood-ratio gradient. Full trajectory products can have

enormous variance; the local action-wise surrogate trades this exact global correction for a practical approximation close to the old policy.

18.8 PPO: a simpler clipped objective

A KL-penalty version adds $-\beta$ KL to the surrogate and adapts β. The clipped form uses

$$L^{\text{clip}}(\theta) = \mathbb{E}\left[\min\left(r_\theta \hat{A}, \text{clip}(r_\theta, 1-\epsilon, 1+\epsilon)\hat{A}\right)\right].$$

- For positive advantage, increasing the action probability helps, but the clipped branch stops rewarding an increase beyond $1+\epsilon$.
- For negative advantage, decreasing its probability helps, but the branch stops rewarding a decrease below $1-\epsilon$.
- Harmful changes remain penalized by the minimum. Clipping is not equivalent to clipping every ratio before multiplying by advantage.

With $\epsilon = 0.2$, advantage 2, and ratio 1.5, the two terms are 3 and 2.4, so the objective uses 2.4. With advantage -2 and ratio 0.5, the terms are -1 and -1.6, so it uses -1.6. This prevents treating a large favorable probability change as unlimited improvement.

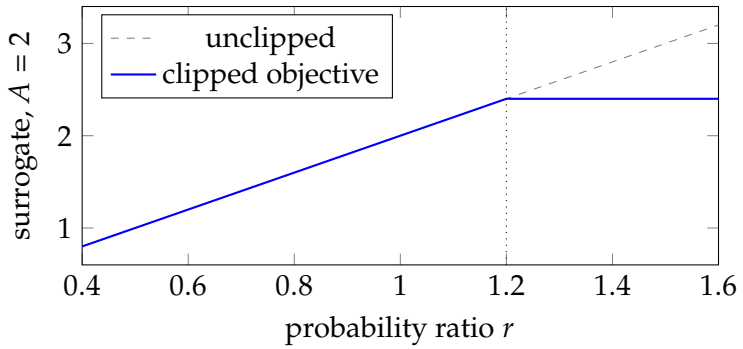


Figure 27: For positive advantage, PPO stops rewarding an excessively large probability increase.

PPO permits several minibatch epochs with ordinary first-order optimizers. It does not impose a hard global KL limit; monitor actual policy change and performance. Locomotion, games, and dexterous manipulation illustrate the flexibility of policy optimization, while the required interaction budget, reward design, and transfer conditions remain central to interpreting success.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 18.

Lecture 19: Off-Policy Reinforcement Learning

19.1 Reuse transitions through Bellman structure

On-policy gradients primarily use data from the current behavior. Off-policy methods learn from transitions generated by another policy, potentially reusing a replay dataset many times. A transition (s, a, r, s', d) records the selected action and its consequence; $d = 1$ marks genuine termination.

For the optimal action value,

$$Q^*(s, a) = \mathbb{E}[r + \gamma(1 - d) \max_{a'} Q^*(s', a') \mid s, a].$$

The target describes the best continuation, even if the behavior policy took a different next action. This is why the update can be off-policy. It still requires data covering relevant states and actions.

19.2 Tabular Q-learning and its guarantees

Given a sampled transition, update only its state-action entry:

$$y = r + \gamma(1 - d) \max_{a'} Q(s', a'), \qquad Q(s, a) \leftarrow Q(s, a) + \alpha[y - Q(s, a)].$$

1. Observe the current state and select an exploratory action.
2. Record reward and next state.
3. If the transition terminates, use $y = r$; otherwise bootstrap from the next-state maximum.
4. Blend the target with the old entry, then move to the next state or restart the episode.

For finite discounted MDPs with bounded rewards, sufficient exploration and per-pair step sizes satisfying $\sum_k \alpha_k = \infty$, $\sum_k \alpha_k^2 < \infty$ give standard convergence guarantees. A greedy behavior policy can fail by never exploring a promising action. An exploration schedule that decays too quickly can have the same problem.

19.2.1 Worked example: one continuing and one terminal transition

Suppose $Q(s, a) = 2$, reward $r = 1$, next-state values are 3 and 5, $\gamma = 0.9$, and $\alpha = 0.2$.

1. For a nonterminal transition, $y = 1 + 0.9(5) = 5.5$, temporal-difference error is 3.5, and the updated value is $2 + 0.2(3.5) = 2.7$.

2. If the same transition actually terminates, $y = 1$ and the update is $2 + 0.2(-1) = 1.8$.
3. Bootstrapping after termination would credit rewards that cannot occur. Conversely, ending a data segment due only to a collection time limit need not mean the underlying task terminates.

19.3 Function approximation and DQN stabilization

A parameterized network $Q_\theta(s, a)$ shares information across states. A squared-error update uses

$$L(\theta) = \tfrac{1}{2}\mathbb{E}_D[(Q_\theta(s, a) - y)^2],$$

with the target treated as constant during differentiation. Then the parameter step is proportional to $(y - Q_\theta)\nabla_\theta Q_\theta$. Updating parameters can change many unobserved values at once, so tabular convergence results no longer apply directly.

DQN uses two central stabilizers:

- **Replay buffer.** Store transitions and sample minibatches. This reuses interaction and reduces short-range temporal correlation in gradient batches. A large buffer can also contain stale or poorly distributed data.
- **Target network.** Use lagged parameters $\bar{\theta}$ in $y = r + \gamma(1 - d)\max_{a'} Q_{\bar{\theta}}(s', a')$. Hold them fixed temporarily or update slowly, reducing how quickly the regression target moves.

These mechanisms address instability but do not remove the interaction of function approximation, bootstrapping, and off-policy sampling. Low training loss on replay data does not prove that values are accurate on actions the dataset rarely contains.

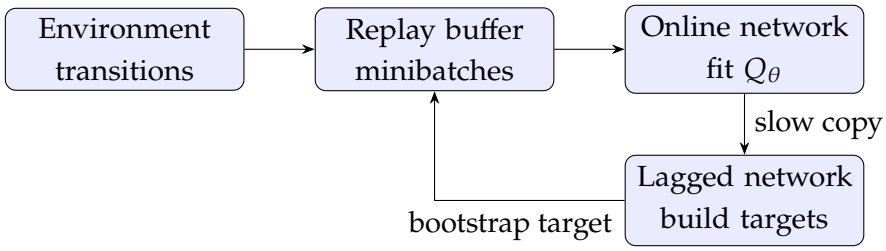


Figure 28: Replay controls data reuse; a lagged network slows target changes.

Maximization over noisy estimates can create overestimation. Double DQN selects an action with the online network and evaluates it with the target network:

$$a^* = \arg\max_a Q_\theta(s', a), \qquad y = r + \gamma(1 - d)Q_{\bar{\theta}}(s', a^*).$$

For example, online next-state estimates $(6, 5)$ select the first action, while target estimates $(3, 7)$ evaluate that action at 3, rather than taking the target maximum 7. Selection and evaluation perform distinct roles.

Other improvements include prioritized replay, which samples informative transitions and needs weighting to account for the changed sampling objective; dueling value/advantage architectures; distributional return prediction; and learned noisy exploration. These are distinct modifications, and combining them requires their own compatible targets and sampling conventions.

19.4 Soft Q-learning and entropy-regularized values

For entropy temperature $\tau > 0$, a discrete-action soft value is

$$V(s) = \tau \log \sum_a e^{Q(s,a)/\tau}, \quad Q(s, a) = \mathbb{E}[r + \gamma V(s')].$$

Its optimal soft policy is proportional to $e^{Q(s,a)/\tau}$. Continuous actions replace the sum by an integral with respect to the specified action measure. That integral can be difficult to evaluate, motivating learned sampling policies.

An implicit sampling network can be trained to generate actions in high-value regions. Stein variational methods combine attraction toward high-density regions with repulsion among particles to avoid all action samples collapsing to one mode. The sampler must balance value and entropy; simply maximizing Q for every sample would recover a deterministic action rather than the desired distribution.

For two discrete action values 0 and $\log 3$ at temperature one, the soft policy probabilities are $1/4$ and $3/4$, and $V = \log 4$. The value includes an entropy contribution and is not just the expected raw reward under the resulting policy.

19.5 DDPG for continuous actions

A continuous-action maximum can be expensive. Deep Deterministic Policy Gradient (DDPG) uses an actor $a = \mu_\theta(s)$ to propose an action and a critic $Q_\phi(s, a)$ to evaluate it. Replay transitions train the critic with

$$y = r + \gamma(1 - d)Q_{\bar{\phi}}(s', \mu_{\bar{\theta}}(s')).$$

The actor maximizes the critic by the chain rule:

$$\nabla_\theta J \approx \mathbb{E}_{s\sim D}\left[(\nabla_\theta \mu_\theta(s))^\top \nabla_a Q_\phi(s, a)\big|_{a=\mu_\theta(s)}\right].$$

This gradient passes through the critic's action input, not through the environment. Replay-state optimization is an off-policy surrogate; its practical success depends on critic accuracy and relevant state coverage.

1. Execute the actor with added exploration noise and store transitions.
2. Fit the critic using slowly changing target actor and critic networks.
3. Improve the actor using the current critic's action gradient.
4. Update targets by Polyak averaging, for example $\bar{\phi} \leftarrow (1-\rho)\bar{\phi} + \rho\phi$ with small ρ.

A fixed additive noise scale must be interpreted in the action's physical units and respect control bounds. A deterministic actor can exploit a spurious peak in the learned critic, making the critic appear better while actual performance worsens.

19.5.1 Worked example: the actor follows the critic's slope

Let an illustrative critic be $Q(s,a) = -(a-2)^2$ and actor output $a = \theta$. At $\theta = 0$, the action derivative is $-2(a-2) = 4$. Gradient ascent with step 0.1 moves the actor to 0.4, improving critic value from -4 to -2.56. If the critic incorrectly places its peak at 20 in a region without data, the same mechanism confidently pushes toward that unsupported prediction. The chain rule can be correct while its learned objective is wrong.

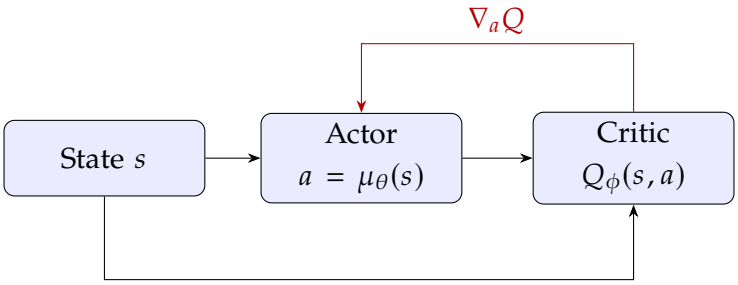


Figure 29: The actor improves using the critic's action derivative, without differentiating the environment.

19.6 Soft actor-critic

Soft actor-critic (SAC) uses a stochastic actor and entropy regularization. In one original formulation, learn Q, V, and π:

$$
\begin{aligned}
y_Q &= r + \gamma(1-d)V_{\bar{\psi}}(s'), \\
y_V &= \mathbb{E}_{a\sim\pi_\theta}[Q_\phi(s,a) - \tau \log \pi_\theta(a \mid s)], \\
L_\pi &= \mathbb{E}_{s\sim D,\, a\sim\pi_\theta}[\tau \log \pi_\theta(a \mid s) - Q_\phi(s,a)].
\end{aligned}
$$

Fit Q and V by squared error to their respective frozen targets; minimize L_π for the actor. The policy objective is, up to a state-dependent constant, a KL projection toward the Boltzmann distribution proportional to $e^{Q/\tau}$.

A widely used variant omits the separate V network and uses two critics. Its target is

$$y = r + \gamma(1 - d)\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s', a') - \tau \log \pi_\theta(a' \mid s')\right], \quad a' \sim \pi_\theta(\cdot \mid s').$$

The minimum reduces optimistic critic errors. This is a different compatible formulation, so its target should not be mixed with a separately defined V target without a deliberate derivation.

For a reparameterized Gaussian actor, draw $\epsilon \sim \mathcal{N}(0, I)$ and set $a = \tanh(\mu_\theta + \sigma_\theta \odot \epsilon)$, with action scaling if needed. The log density must include the transformation Jacobian. Omitting it changes the entropy term and therefore the objective.

19.6.1 Worked example: compute a soft target

Let $r = 1$, $\gamma = 0.9$, two target critic estimates at the sampled next action be 4 and 5, $\log \pi(a' \mid s') = -0.7$, and $\tau = 0.2$.

1. Select the smaller critic value, 4.
2. The entropy-adjusted continuation is $4 - 0.2(-0.7) = 4.14$.
3. The nonterminal target is $1 + 0.9(4.14) = 4.726$; a terminal target remains 1.

Entropy encourages a spread of useful actions and can limit premature collapse. It is not a guarantee of discovering a distant rewarding region. Data support, reward specification, critic error, and policy capacity remain important in both simulated and real-robot learning.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 19.

Lecture 20: Model-Based Reinforcement Learning

20.1 Learn a model, then use it for decisions

Model-based RL learns a dynamics model from interaction and uses it to improve a controller. The model can support many imagined rollouts per physical transition and can sometimes be reused for a new reward.

1. Collect transitions under the current exploratory behavior.
2. Fit a dynamics model to the accumulated data, using held-out data to inspect predictive error.
3. Improve a policy or plan using that model, through trajectory optimization, policy search in the learned simulator, or model-predictive control.
4. Execute the resulting behavior in the real environment, append the new transitions, and refit.

The loop is essential. A policy optimized against the first model usually visits a different distribution of states and actions, so the original dataset may no longer describe its behavior well.

A model $\hat{f}_\eta(s, a)$ can predict the next state or the change $s' - s$. Probabilistic models also predict uncertainty. A small average one-step error does not establish accurate long rollouts or good control, especially if errors align with unstable directions.

20.2 Compounding error and model exploitation

Suppose the true and modeled states differ by e_t, the dynamics are Lipschitz with factor L, and each one-step model error is at most ε in the visited region. Then

$$e_{t+1} \le L e_t + \varepsilon, \qquad e_h \le \varepsilon \sum_{k=0}^{h-1} L^k$$

from identical initial states. For $L > 1$, the bound grows geometrically. This simplified bound assumes the relevant error and Lipschitz bounds actually hold; extrapolated states may violate them.

20.2.1 Worked example: optimism selects the wrong action

Two illustrative actions have true rewards 5 and 0. A learned simulator predicts 4.8 and 10 because the second action lies outside the training region.

1. A perfect optimizer of the learned model chooses the second action.
2. Actual reward falls from a possible 5 to 0.
3. The optimizer has exploited model error, rather than failing to optimize. Improving its search can make the exploitation stronger.
4. New real data at that action can correct the model locally; uncertainty-aware planning can discourage unsupported optimism before such data are available.

This differs from ordinary supervised overfitting: the policy actively searches for inputs where the model is favorably wrong. Evaluation must therefore consider the decisions induced by the model, not only its error on a fixed test set.

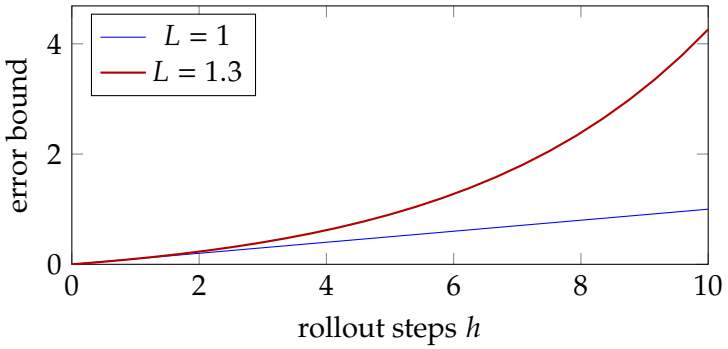


Figure 30: The same one-step bound $\varepsilon = 0.1$ can compound very differently with different dynamics sensitivity.

20.3 Ensembles and ME-TRPO

An ensemble fits several models $\hat{f}_1, \ldots, \hat{f}_K$, using different initializations or resampled data. Disagreement can expose regions where predictions are poorly constrained. It approximates epistemic uncertainty, while modeled observation or process noise represents a different source of uncertainty.

Model-Ensemble TRPO trains a policy using rollouts across learned models and conservative policy updates. A policy that succeeds in just one model but fails in the others has less opportunity to exploit one model's particular error. Real data are periodically collected to correct the ensemble.

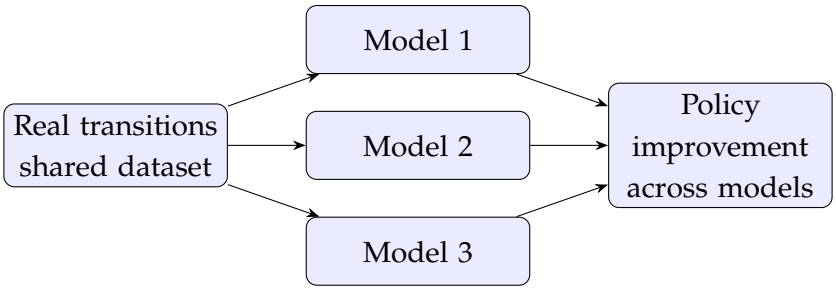


Figure 31: An ensemble tests policy improvement against several plausible dynamics models.

An ensemble is not an automatic confidence set. All models can share the same missing variable or extrapolation bias. Distinguish whether a rollout holds one model fixed or changes models at each step: the former preserves one coherent dynamics hypothesis, while the latter represents a different uncertainty process.

For an illustrative action, model rewards $(8, 8, 0)$ have average $16/3$, while another action with $(5, 5, 5)$ has average 5. Mean performance favors the first; worst-case performance favors the second. The aggregation rule expresses a risk preference and must be stated explicitly.

20.4 MB-MPO: optimize for adaptation

A single robust policy must compromise across models. Model-Based Meta-Policy Optimization (MB-MPO) instead seeks initial parameters that can adapt quickly to each learned dynamics model.

For model i, define an inner policy-gradient update

$$\theta_i' = \theta + \alpha \nabla_\theta J_i(\theta).$$

The outer objective maximizes performance *after* adaptation:

$$\max_\theta \frac{1}{K} \sum_i J_i(\theta_i').$$

1. Fit an ensemble representing plausible environment dynamics.
2. Gather model-specific experience and compute a small policy update within each model.
3. Update the shared initialization so those adapted policies perform well.
4. Collect real data with relevant adapted behavior, improve the models, and repeat.

The initialization need not be the best fixed policy. It should be easy to improve using a small amount of informative experience.

20.4.1 Worked example: robustness and adaptation differ

Consider scalar action u and two model costs $(u-1)^2$ and $(u+1)^2$. The best shared fixed action is $u = 0$, with average cost 1.

1. A gradient step on model-specific cost with step size $1/2$ moves $u = 0$ to 1 in the first model and -1 in the second.
2. Each adapted cost is zero.
3. The improvement depends on knowing, or learning from data, which gradient applies. It does not mean that a physical robot can switch to the right action without informative experience.

If the real dynamics lie outside what the ensemble and adaptation procedure cover, rapid adaptation is not guaranteed. Measuring real interaction used for adaptation is part of evaluating sample efficiency.

20.5 Asynchronous learning and the meaning of efficiency

A synchronous loop waits for data collection, then model fitting, then policy optimization. An asynchronous implementation overlaps these workers, sharing a replay dataset and updated parameters.

- **Data collection** supplies fresh transitions while learning continues.
- **Model learning** updates predictors from the growing dataset.
- **Policy improvement** uses current model snapshots to improve behavior.

This can reduce wall-clock delay and diversify behavior, but creates stale-policy and stale-model issues. A rollout should record the model and policy versions needed to interpret it. Faster hardware throughput is distinct from using fewer physical transitions.

Evaluation should therefore report both return versus environment interactions and return versus elapsed time. Reaching, insertion, and stacking stress different aspects: a model good enough for coarse reaching may miss small contact effects that determine insertion success. Ablating model count, planning horizon, and collection frequency helps identify which component supplies the improvement.

20.6 Learning models from images

Pixels contain information about the state but also lighting, texture, and other nuisance variation. A latent model uses encoder $z_t = e_\psi(o_t)$, transition $p_\eta(z_{t+1} \mid z_t, a_t)$, and optional decoder or reward

model. Planning occurs in the latent space, whose representation must retain the distinctions needed for control.

20.6.1 Reconstruction is useful but not sufficient

World-model approaches learn compact visual and temporal representations, then train a controller in that learned world. Spatial autoencoders can represent visual landmarks as feature positions and support local control. Locally linear latent models, including Embed to Control and structured models such as SOLAR, seek dynamics compatible with efficient control methods.

A representation that reconstructs backgrounds perfectly but loses contact state can be poor for manipulation. Conversely, requiring every pixel detail may waste capacity on irrelevant variation. Position-velocity encoders and task-oriented predictive representations impose structure related to motion or control rather than relying only on reconstruction.

20.6.2 Forward and inverse losses prevent trivial collapse

If the only objective is to predict the next latent vector from the current one, the encoder can set every latent vector to zero and achieve zero prediction error. An inverse loss that predicts a_t from (z_t, z_{t+1}) can discourage this collapse when different actions have distinguishable consequences. Reconstruction, contrastive constraints, or task supervision provide other sources of information. No auxiliary objective guarantees that every hidden physical variable becomes identifiable.

20.6.3 Planning with predicted images and task features

Visual foresight predicts future images under candidate action sequences, scores task-related outcomes, and uses CEM or another optimizer in MPC. PlaNet-style methods instead predict latent dynamics over multiple steps before planning. Models can predict task-relevant events, such as collision or progress, without generating every pixel, as in action-conditioned navigation predictors.

Successor features offer another form of reuse: $\psi^\pi(s, a) = \mathbb{E}_\pi[\sum_t \gamma^t \phi_t \mid s, a]$ predicts discounted feature occupancy. For reward $r_w = w^\top \phi$, $Q^\pi_w = w^\top \psi^\pi$. Changing w changes the reward evaluation without relearning those predictions, provided the feature definition, dynamics, and policy-conditioned prediction remain appropriate. A changed task may still require a different policy or new exploration.

A useful state abstraction should retain information needed to predict relevant consequences of actions while discarding nuisance variation. Whether a latent space is visually compact is a separate question from whether it supports reliable planning.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 20.

Lecture 21: How Physics Simulators Work

21.1 A simulator is a chain of approximations

A physics simulator advances a mathematical model through time. It needs a state representation, equations of motion, force and contact models, and a numerical integration rule. A rendered animation can look plausible while its forces, energy, or timing are wrong for control.

For a particle, Newton's equation is $m\ddot{r} = F$. A rigid body additionally has orientation and angular velocity. In body coordinates, rotational dynamics are

$$I\dot{\omega} + \omega \times (I\omega) = \tau,$$

where I is the inertia tensor and τ the applied torque in the same frame. The cross product accounts for angular momentum changing direction with the rotating frame. Mixing world-frame and body-frame quantities without transforming them gives incorrect dynamics.

21.2 Lagrangian mechanics removes internal constraint forces

Generalized coordinates q describe the system's independent degrees of freedom. Define kinetic energy $T(q, \dot{q})$, potential energy $U(q)$, and Lagrangian $L = T - U$. The Euler–Lagrange equations with generalized forces Q_i are

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = Q_i.$$

For many articulated robots they become

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = Bu + J(q)^{\top} f_c.$$

Joint reaction forces that merely enforce the chosen coordinate relationships need not be solved separately. Contact with the environment still supplies external constraints and forces.

21.2.1 Worked derivation: a simple pendulum

A point mass m hangs from a massless rod of length l, with angle θ measured from downward vertical.

1. Speed is $l\dot{\theta}$, so $T = \frac{1}{2}ml^2\dot{\theta}^2$.
2. Choose $U = mgl(1 - \cos\theta)$, which is zero at the bottom.

3. Then $\partial L/\partial\dot{\theta} = ml^2\dot{\theta}$ and $\partial L/\partial\theta = -mgl\sin\theta$.

4. With applied torque u and viscous damping $b\dot{\theta}$, the equation is $ml^2\ddot{\theta} + mgl\sin\theta = u - b\dot{\theta}$.

The small-angle approximation replaces $\sin\theta$ by θ near downward equilibrium. Around upright, the linearized gravitational term has the opposite stabilizing effect; the reference angle must be specified.

21.3 Coupled and underactuated models

For a two-link planar pendulum with point masses m_1, m_2 at the link ends and absolute downward angles θ_1, θ_2, the kinetic energy includes

$$T = \tfrac{1}{2}(m_1 + m_2)l_1^2\dot{\theta}_1^2 + \tfrac{1}{2}m_2 l_2^2\dot{\theta}_2^2 + m_2 l_1 l_2\cos(\theta_1 - \theta_2)\dot{\theta}_1\dot{\theta}_2.$$

The cross term produces an off-diagonal mass-matrix entry. Accelerating one link therefore affects the other even before adding control. Relative-angle coordinates give different-looking formulas for the same mechanics, so coordinate conventions must not be mixed.

An acrobot has two links but only one actuated joint. A cart-pole has a cart force but no directly commanded pole torque. Their actuation matrix B has fewer independent columns than state acceleration coordinates. A simulator that assigns independent acceleration commands to all coordinates would remove the underactuation and change the task.

A kinematic bicycle car model instead describes velocity-level motion:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = (v/L)\tan\delta,$$

with wheelbase L and steering angle δ. Dubins models restrict forward speed and turning curvature; Reeds–Shepp models permit reversing. Such models omit tire-force and inertial effects and are valid only within their intended regime.

21.4 Friction, drag, and robot description

- **Static friction** can balance tangential load up to a limit proportional to normal force. It is not always equal to that limit.
- **Sliding friction** opposes slip, often approximated by Coulomb friction with magnitude $\mu_k f_n$. Static and kinetic coefficients can differ.
- **Drag** opposes relative motion through a fluid. Linear drag is proportional to velocity; a common quadratic model is proportional to $\|v\|v$.

- **Model files** specify links, joints, geometry, masses, inertias, and actuator relationships. Denavit–Hartenberg parameters describe kinematic chains; URDF provides a practical structured robot description. Visual geometry and collision geometry may differ deliberately.

Plausible masses alone are insufficient: an incorrect center of mass or inertia tensor can substantially change behavior. Units and coordinate frames are part of the model specification.

21.5 Numerical integration changes the dynamics

For $\dot{x} = f(x, u)$ with a held control and step h, forward Euler uses $x_{k+1} = x_k + hf(x_k, u_k)$. Backward Euler instead solves $x_{k+1} = x_k + hf(x_{k+1}, u_k)$, usually requiring a nonlinear solve. Implicit integration can improve stability for stiff systems but introduces its own numerical damping and computational cost.

For second-order mechanics with acceleration $a(q, v)$:

$$\begin{aligned} \text{explicit: } & q_{k+1} = q_k + hv_k, \quad v_{k+1} = v_k + ha(q_k, v_k), \\ \text{semi-implicit: } & v_{k+1} = v_k + ha(q_k, v_k), \quad q_{k+1} = q_k + hv_{k+1}. \end{aligned}$$

Semi-implicit Euler uses the new velocity to move the position. In suitable conservative systems it has useful geometric properties, but the word symplectic is not a universal guarantee for arbitrary dissipative or constrained updates.

21.5.1 Worked example: explicit Euler invents energy

For a unit harmonic oscillator, $\dot{q} = v$, $\dot{v} = -q$, with energy $E = \frac{1}{2}(q^2 + v^2)$:

1. Explicit Euler gives $q' = q + hv$, $v' = v - hq$.
2. Expanding yields $q'^2 + v'^2 = (1 + h^2)(q^2 + v^2)$. Energy increases on every nonzero step, although the true system conserves energy.
3. At $(q, v) = (1, 0)$ and $h = 0.1$, the next energy is 0.505 rather than 0.5.
4. Semi-implicit Euler gives $v' = -0.1$, $q' = 0.99$, with energy 0.49505. It does not conserve the original energy exactly either, but its long-term behavior can be bounded for an appropriate step size.

A policy may exploit numerical energy injection. Reducing the time step and comparing integrators helps reveal whether a learned behavior relies on an integration artifact.

A fourth-order Runge–Kutta method evaluates four intermediate slopes, then combines them with weights $1, 2, 2, 1$ divided by six. Its higher smooth-system accuracy does not automatically handle impacts or discontinuities; event handling and contact treatment remain separate concerns.

21.6 Collision detection: reject cheaply, inspect precisely

Broad-phase detection uses bounding volumes or spatial partitions to reject distant object pairs. It should be conservative: false positives cause extra work, while false negatives miss collisions. Narrow-phase detection then checks the remaining geometry more precisely.

For convex shapes, the separating-axis theorem characterizes nonintersection by a separating direction. GJK uses support mappings and the Minkowski difference to test intersection or distance. EPA can estimate penetration information after overlap is found. Concave geometry is often decomposed into convex pieces, introducing a tradeoff between geometric fidelity and checking cost.

21.7 Impulses, unilateral contact, and time stepping

An impact changes velocity over a short interval. Integrating the force over that interval gives impulse j, with

$$M(v^+ - v^-) = J^\top j.$$

For a point mass striking a fixed wall in its normal direction, restitution $e \in [0, 1]$ gives $v_n^+ = -ev_n^-$. Thus the normal impulse is $j_n = m(v_n^+ - v_n^-)$. Nonpenetration and friction couple simultaneous contacts and can require a constrained solve.

21.7.1 Worked example: a wall impact

A 2-kg particle has incoming normal velocity -3 m/s and restitution $e = 0.5$.

1. Outgoing normal velocity is 1.5 m/s.
2. Impulse is $2[1.5 - (-3)] = 9$ N s in the outward normal direction.
3. Kinetic energy decreases from $\frac{1}{2}(2)9 = 9$ J to $\frac{1}{2}(2)(2.25) = 2.25$ J. Restitution is a velocity ratio, so energy is multiplied by e^2 in this one-dimensional fixed-wall case.

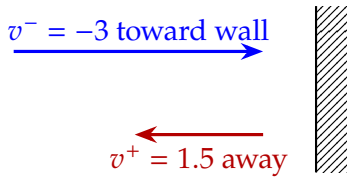


Figure 32: An impulse changes velocity at contact; the sign is defined by the wall's outward normal.

Contact softness, solver tolerances, and time steps all affect the modeled mechanics.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 21.

Lecture 22: Sim-to-Real Transfer and Domain Randomization

22.1 Why simulated experience is attractive

Physical robot data require hardware time, resets, supervision, and sometimes repairs. Simulation can run many trials, generate labels and rewards automatically, and deliberately sample rare situations. It supports algorithm prototyping, software debugging, system design, and repeated regression tests even when a learned policy is not transferred directly.

The practical problem is the reality gap: simulated sensors and mechanics differ from the deployed system. The MDP formalism can represent uncertainty if the state and model are adequate, but an implemented state often omits relevant variables, observations are partial, rewards are difficult to measure, and hardware changes over time. Treating those modeling choices as exact can hide the real engineering difficulty.

- **Dynamics mismatch** includes mass, inertia, friction, damping, delays, backlash, compliance, and external disturbances.
- **Observation mismatch** includes lighting, textures, lens distortion, noise, occlusion, rolling shutter, and missing detections.
- **Task mismatch** includes the distribution of objects, initial states, user behavior, and what the reward actually measures.

A visually realistic scene need not have accurate contact mechanics. A mechanically accurate simulator need not produce transferable images. Fidelity must be evaluated against the decisions the robot must make.

22.2 Small errors can compound through feedback

A policy chooses actions using observations produced by its previous actions. A small prediction error can move it to a different state, causing a different image and action, then a larger trajectory discrepancy. Offline replay of a fixed log cannot reveal every such counterfactual consequence because the recorded future belongs to the original actions.

For an illustrative scalar system with true evolution $x_{t+1} = 1.1x_t + u_t$ and modeled coefficient 1.0, even a controller that exactly cancels the modeled drift leaves residual true dynamics. The error depends on visited state magnitude. Testing only near zero can miss failures during larger motions.

A useful simulator-development loop is:

1. Specify the robot model and relevant sensors, timing, and environment geometry.
2. Generate meaningful scenarios, including difficult combinations and boundary cases.
3. Compare simulated and physical trajectories under matched action sequences.
4. Diagnose discrepancies, improve the model or randomization, and rerun both task and regression evaluations.

Synthetic objects can come from scanned datasets, geometric repositories, or procedural generation. Random placement alone can create impossible interpenetrations; physics-based settling or structured scene generation can produce more meaningful arrangements.

22.3 System identification and identifiability

System identification chooses simulator parameters η to match real trajectories:

$$\min_{\eta} \sum_{a \in \mathcal{D}} D(\tau_{\eta}(a), \tau_{\text{real}}(a)),$$

where a denotes an applied action sequence and D measures discrepancy. The chosen sequences must excite the relevant dynamics; uninformative data cannot distinguish parameters.

22.3.1 Worked example: fit a simple velocity model

Suppose $v_{t+1} = av_t + bu_t$. Two noiseless transitions are $(v, u, v') = (1, 0, 0.8)$ and $(0, 2, 1)$.

1. The first gives $a = 0.8$.
2. The second gives $2b = 1$, so $b = 0.5$.
3. If every experiment instead had $u = v$, only $a + b$ would be identified. Many parameter pairs would fit equally well.
4. With noisy data, use regression and validation trajectories rather than trusting an exact fit to two transitions.

Short-horizon prediction losses can be less dominated by unstable rollout divergence, but may miss long-term errors. Identification and control should therefore be evaluated together. More physical detail introduces more parameters to identify and does not guarantee better transfer if the available data cannot constrain them.

22.4 Adaptation versus randomization

Domain adaptation uses real-domain information to modify a model or representation. It may use labeled examples, paired observations, self-supervised consistency, or unlabeled distribution alignment. Examples include adapting an inverse dynamics model, adding a new network branch while retaining simulated features, or using simulation as a prior for real-data learning.

Domain randomization instead trains across many simulated environments, intending to make the real environment one plausible variation. A formal expected-return objective is

$$\max_{\theta} \mathbb{E}_{\eta \sim p(\eta)}[J(\pi_\theta; \eta)].$$

The distribution $p(\eta)$ is a design choice. Maximizing its average is not the same as maximizing worst-case performance, and neither guarantees success outside its support.

22.5 Visual randomization and what the model should ignore

Randomize textures, colors, lighting, camera pose, materials, backgrounds, and distractors while preserving the task label. A network can then learn shape or geometry that remains useful across those changes.

22.5.1 Worked example: remove a spurious color cue

Suppose all training target objects are red and all distractors blue. A classifier can minimize training error by recognizing color alone.

1. At deployment a blue target causes failure even if its geometry is familiar.
2. Randomizing target and distractor colors independently removes the reliable label-color association.
3. The network must use another cue, such as shape, if that cue remains available and identifiable.
4. Randomizing so aggressively that target shape is destroyed would also destroy the supervision. Useful invariance requires preserving the task-defining information.

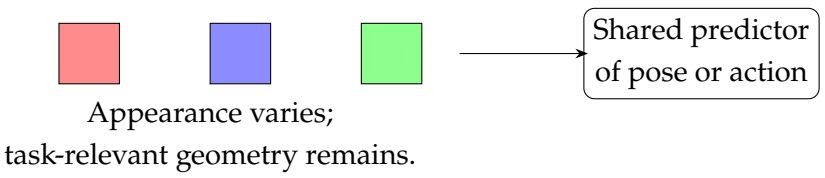


Figure 33: Randomization removes reliable nuisance cues while retaining useful structure.

Procedural grasping data can vary object geometry as well as appearance. Combining convex components into many artificial objects can expose a grasp predictor to diverse local surfaces, even when the objects are not semantically realistic. The generated geometry must still support meaningful contact and grasp labels.

Three interpretations help explain randomization:

- **Coverage.** The training distribution includes relevant real conditions. Literal dense coverage is expensive in high dimensions and impossible for effects absent from the simulator.
- **Invariance.** Variation teaches the model which aspects can be ignored for the task. This can generalize beyond memorizing every combination.
- **Adaptation.** Experience across environments teaches how to infer the current environment and adjust behavior.

These explanations are complementary, and none alone guarantees transfer for arbitrary randomization.

22.6 Dynamics randomization and recurrent adaptation

Dynamics parameters such as mass or friction often stay approximately fixed during one episode. Sample them once per episode when that matches the intended uncertainty. Resampling mass every step creates a different physical process and can prevent the agent from identifying a consistent environment.

A recurrent policy can use its hidden state to infer dynamics from the history of observations and actions. RL^2 trains the recurrent update across tasks: slow outer learning adjusts network weights, while fast within-task adaptation occurs through hidden-state changes. Reset memory between independent tasks, but preserve it across the episodes intended to belong to one adaptation task.

22.6.1 Worked example: infer an unknown response scale

Let displacement satisfy $\Delta x = bu$ with $b \in \{0.5, 1.5\}$ fixed for the episode. A probe $u = 1$ yields displacement 0.5 or 1.5 in the noiseless case.

1. Before probing, a fixed policy must compromise or risk a wrong displacement.
2. The response identifies b, allowing a later desired displacement d to use $u = d/b$.
3. A recurrent controller can store evidence about b without exposing it as an explicit parameter.
4. If the probe is costly or unsafe, the policy must balance identification against task progress. Adaptation is itself a sequential decision problem.

Noise, delays, and unmodeled state complicate this inference, so the example illustrates the mechanism rather than a general identification guarantee.

Dexterous manipulation can randomize physical parameters, correlated and independent sensor noise, dropout, simulation step size, backlash, external forces, and visual appearance. A single source of random noise does not stand in for all these mechanisms because their temporal and causal effects differ.

22.7 Structured variation and learning the simulator distribution

Independent randomization can generate nonsensical combinations. Structured domain randomization preserves relationships such as road layout, support surfaces, object placement, and lighting geometry. Meta-Sim-style methods learn scene-generation parameters to improve downstream task performance rather than merely producing random scenes.

A general nested objective chooses distribution parameters ψ so that a predictor trained on simulated data from $p_\psi(\eta)$ performs well on a real validation set. This can be expensive because each distribution proposal may require training a model. The real validation set also becomes part of the tuning process; reserve separate tests for a final transfer claim.

Randomized-to-canonical adaptation uses a network to map varied observations into a consistent canonical appearance before task prediction. Training can pair randomized and canonical renderings of the same scene, giving correspondence without labeling real images. The canonicalizer must preserve object pose and task geometry; a visually tidy output that changes those quantities defeats its purpose.

Practical randomization requires an engine, scene-generation code, sensor rendering, and reproducible parameter records. The 2019 examples include Gazebo plugins, Unity-based ORRB, Unreal-based NDDS, and DeepDrive. They illustrate different integration choices rather than a requirement to use a particular tool. Save parameter distributions and random seeds so a failure can be reproduced, then evaluate on fresh seeds to detect overfitting.

SPOTA-style transferability assessment compares performance on unseen sampled dynamics with policies trained for those dynamics, seeking evidence of overfitting to the training simulation distribution. Its conclusions depend on the sampled family and assessment assumptions, not on every possible real environment.

22.8 Hard-case generation and automatic curricula

Active domain randomization searches for simulator settings that reveal weaknesses. Network-driven appearance changes can similarly create difficult training images. Such changes must preserve semantics; otherwise the generator can "win" by deleting information needed to predict the label.

Automatic Domain Randomization (ADR) expands parameter ranges as training succeeds near their boundaries.

1. Maintain a lower and upper bound for each randomized parameter.
2. During a boundary evaluation, sample other parameters normally and fix one parameter to a chosen bound.
3. Track success at that boundary over enough trials to reduce noise.
4. Widen a boundary with strong performance; narrow or hold it when performance is poor.
5. Continue ordinary policy training on the evolving distribution.

For an illustrative friction range [0.6, 1.0], reliable success at 0.6 can justify testing a lower boundary such as 0.5. Success only at the center does not justify that expansion. Expanding all dimensions simultaneously can introduce combinations much harder than any one-dimensional boundary test, so evaluation must still consider interactions.

Policy transfer with strategy optimization trains a family of policies conditioned on a strategy or dynamics-related variable, then searches that variable on the physical robot. This reduces the real-world search dimension but does not make real trials free or remove the need for a useful policy family.

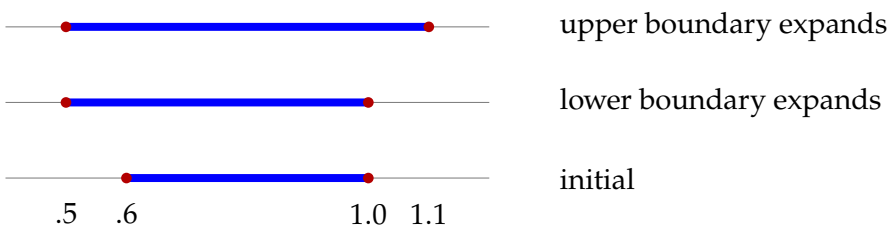


Figure 34: ADR uses performance at parameter boundaries to decide which ranges can be expanded.

22.9 Designing a meaningful transfer evaluation

Randomization ranges that are too narrow allow brittle specialization. Ranges that are too broad can include physically impossible systems or make the task unlearnable. Useful evaluation separates held-out simulator parameters, new scene combinations, and actual physical trials.

A practical real-to-sim-to-real cycle uses measured data to constrain a simulator family, trains across uncertainty that remains, and returns to physical experiments to diagnose new failures. Report the number of real interactions, how much tuning used them, the operating conditions, and which failures remain. Simulation is valuable precisely when it makes those experiments more informative, rather than serving as a substitute for evidence about the deployed robot.

Source. Josh Tobin, *Advanced Robotics*, Fall 2019, Lecture 22.

Lecture 24: Autonomous Drones: Perception, Tracking, and Control

24.1 A product must coordinate the whole robot

An autonomous camera drone must estimate its own motion, map obstacles, track a moving subject, and choose a physically feasible flight path while keeping the camera useful. Weight, power, vibration, camera placement, compute, and software interact. Improving one component in isolation can worsen the full system, for example by adding a heavy sensor or increasing processing latency.

Four functions form the central pipeline:

1. Estimate the drone's pose, velocity, sensor biases, and relevant calibration parameters.
2. Build geometry detailed enough for obstacle avoidance.
3. Detect subjects, preserve their identities, and predict their motion.
4. Plan controls balancing collision avoidance, dynamic limits, smoothness, tracking, and image composition.

The discussion concerns the system described in the 2019 talk. Its hardware examples explain design constraints rather than specifying current products.

24.2 Wide-angle vision and visual-inertial estimation

A recording camera gives a useful cinematic view, while navigation requires awareness outside that narrow view. The described drone uses navigation cameras covering the surrounding scene, with overlapping fisheye views. A wide field of view reduces blind regions but requires calibrated projection and distortion models; a pinhole formula cannot simply be applied to every fisheye pixel without transformation.

Visual-inertial odometry combines tracked visual features with inertial measurements. A feature observed from multiple poses constrains its three-dimensional location and the camera trajectory. An optimization can minimize reprojection residuals

$$r_{ij} = z_{ij} - \text{project}(T_i, P_j, \kappa),$$

where P_j is a landmark, T_i a camera pose, κ calibration parameters, and z_{ij} the measured image coordinate. Inertial factors constrain motion between camera times and estimate biases.

- **IMU biases matter.** Integrating a small acceleration bias produces growing velocity and position errors. Visual observations help constrain drift.
- **Extrinsic calibration matters.** Relative camera poses change the geometry used to triangulate points. Small angular errors can produce large depth errors at long range.
- **Intrinsics can change.** Temperature and mechanical effects alter lens behavior. Factory calibration supplies an initial model, while online estimation can correct remaining changes when the observations make them identifiable.
- **The gimbal is another moving frame.** Its commanded angle and sensor reading need not exactly describe the recording camera's actual relationship to the navigation cameras.

Adding calibration variables gives the optimizer more explanatory power, but also creates correlations and observability issues. It requires informative motion and well-designed priors or constraints.

24.2.1 Worked example: inertial bias accumulates

With an uncorrected constant acceleration bias $b = 0.05\ \mathrm{m/s^2}$, zero initial error, and no visual correction, velocity error after t seconds is bt, and position error is $\frac{1}{2}bt^2$. After 10 seconds these are $0.5\ \mathrm{m/s}$ and $2.5\ \mathrm{m}$. The arithmetic explains why inertial sensing is useful for short-term propagation but cannot by itself guarantee long-term accurate position.

24.3 Rolling shutter and vibration violate a single-pose image model

A rolling-shutter camera exposes different rows at different times. During fast rotation, the top and bottom of an image correspond to different poses. Treating all pixels as simultaneous introduces structured errors rather than independent pixel noise.

For illustrative readout time 20 ms and angular speed 200 degrees/s, the orientation changes by 4 degrees during the image. A single-pose model can then misinterpret camera motion as scene deformation. Vibration adds faster pose changes, sometimes creating a wavy appearance.

Two remedies complement each other: reduce problematic vibration and model the time-dependent camera pose. Software compensation cannot recover an image with no usable signal, while improved hardware alone may not remove all timing effects. This is one reason the physical design and estimation algorithm are developed together.

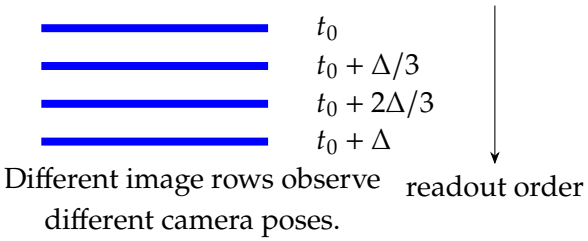


Figure 35: A rolling-shutter image is a time sequence, not one instantaneous geometric observation.

24.4 Depth estimation serves different needs from odometry

Odometry can track a modest number of reliable features and ignore difficult image regions. Obstacle avoidance must reason about regions the drone might enter, including thin branches and textureless surfaces. A good motion estimate does not imply a complete collision map.

Stereo estimation compares observations from different cameras. For an ideal rectified pinhole pair, depth is $Z = fB/d$, where f is focal length in pixels, B baseline, and d disparity. Differentiation gives $|\Delta Z| \approx fB|\Delta d|/d^2$: small disparity errors are especially costly at long range.

24.4.1 Worked example: disparity precision limits range

Take illustrative $f = 400$ pixels and $B = 0.1$ m.

1. At $Z = 5$ m, disparity is $40/5 = 8$ pixels. A 0.2-pixel disparity error gives approximate depth error $40(0.2)/8^2 = 0.125$ m.
2. At $Z = 20$ m, disparity is only 2 pixels. The same error gives $40(0.2)/2^2 = 2$ m.
3. A small calibration or matching error has a much larger effect far away. Detecting an object and measuring its exact distance are related but distinct requirements.

Learned stereo can compute shared image features, build a cost volume over candidate disparities, regularize that volume, and choose a depth estimate. An accurate benchmark model may still be too slow for onboard use. Processing resolution, direction, and computation can be allocated according to reachable motion and the camera task, while retaining awareness of hazards outside the most immediately relevant cone.

24.5 Failure cases that determine real reliability

- **Thin objects.** Branches and wires occupy few pixels and may disappear against parts of the background. Missing a small pixel fraction can still cause a collision.

- **Glare and dirty lenses.** Sun rays, smudges, droplets, and cracks can make corresponding patches look different across cameras, violating photometric matching assumptions.
- **Motion blur.** Aggressive rotation can remove the fine structure needed for matching even when the scene is otherwise textured.
- **Glass and mirrors.** A reflection can describe a geometrically consistent virtual scene behind a physical surface. A clean-looking depth estimate does not establish that the surface is traversable.
- **Textureless surfaces.** A white ceiling and a bright sky may provide similar local evidence but demand opposite actions. Wider scene context can help, without making the ambiguity disappear universally.

A low average depth error can be dominated by easy ground and walls while ignoring a small hazardous object. Evaluate occupancy and obstacles relative to possible actions, not just image-wide average pixel error.

24.5.1 Worked example: stopping distance links sensing to control

For speed v, delay Δ, and available constant braking deceleration $a > 0$, an idealized stopping distance is

$$d_{\text{stop}} = v\Delta + \frac{v^2}{2a}.$$

At illustrative $v = 10$ m/s, $\Delta = 0.1$ s, and $a = 5$ m/s^2, it is $1 + 10 = 11$ m, before adding margins for uncertainty or changing dynamics. Reducing perception delay by 0.05 s saves 0.5 m, while doubling speed quadruples the braking term. The distance at which a thin obstacle becomes detectable must be compared with this physical requirement.

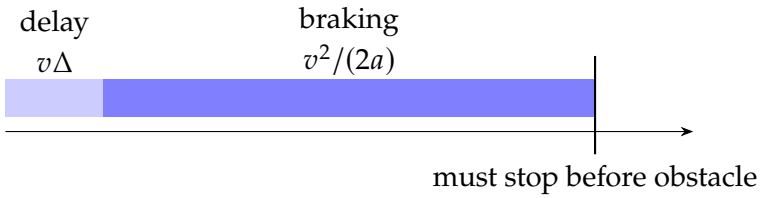


Figure 36: Detection range, latency, and braking capability jointly constrain flight speed.

24.6 Moving objects and identity tracking

Temporal depth from camera motion often assumes the world is static. A subject moving with the drone can stay at nearly the same image location, which a static-scene interpretation may mistake for a very distant object. Simultaneous stereo and explicit moving-object estimates supply different information.

Tracking combines semantic detections, appearance features, three-dimensional position, and motion prediction. Multi-object reasoning is useful even when the user selects only one subject: maintaining other identities helps avoid switching to a similar person after occlusion.

1. Detect candidate instances and extract appearance features.
2. Predict existing tracks forward using a motion model.
3. Associate detections with tracks using compatible position, velocity, and appearance evidence.
4. Maintain uncertainty through occlusion and handle newly appearing or disappearing objects explicitly.

Two similarly dressed people exchanging visibility at the same position and velocity can remain ambiguous. A system should not turn a weak association into certainty merely because its motion model prefers continuity. Additional cues, such as an external position device, can support reacquisition when available.

Synthetic data provide known geometry and labels, while real data capture difficult appearance effects without always supplying depth ground truth. Adding a synthetic glare effect to a real image can teach a consistency target: the scene's geometry should remain the same. Such augmentation must avoid teaching the system to hallucinate an obstacle-free scene when the original prediction was already wrong.

24.7 Planning, fallback behavior, and experimental evidence

A planner must encode what good flight means, then find a sufficiently good control under its model. These are different failure modes: the optimizer can solve the wrong objective, or fail to solve the intended one. Trajectory optimization, sampling, motion libraries, and MPC can play complementary roles.

Constraints include rotor dynamics, acceleration limits, obstacle clearance, and the camera's tracking requirements. Online wind estimation can improve predicted motion. A local optimizer needs a feasible or recoverable fallback when its new proposal is worse, but constant velocity is not inherently safe near an obstacle. A fallback that relaxes cinematic goals while attempting to stop and avoid obstacles has a clearer physical purpose.

Degraded navigation images can make autonomous tracking inappropriate. Automatic landing is not always safe either: the response must consider the environment, remaining sensors, and manual-control options. Communicating these limits is part of usability.

Simulation supports tests of unusual states and rotor failures. A simulated three-rotor recovery still needs physical validation of the rapid-spin estimator and vehicle dynamics.

Source. Hayk Martirosyan, *Advanced Robotics*, Fall 2019, recorded talk.

Lecture 25: How Robotics Research Ideas Develop

25.1 Start from a bottleneck that matters

A research contribution begins with a problem whose resolution changes what can be done. A working demonstration is useful evidence, but repeatedly patching its failures can expose a deeper limitation. The history connecting robot manipulation, trajectory optimization, and policy optimization illustrates how one concrete system can motivate a more general method.

A knot-tying robot can adapt a demonstrated motion through a nonrigid transformation of the rope scene. Yet every new failure can require another hand-written repair. That creates a research question: can experience improve the policy systematically, rather than extending a collection of special cases indefinitely?

1. Identify the recurring failure, such as unstable learning or an objective that misses the user's intent.
2. Separate it from incidental complexity in the original robot.
3. Construct the smallest experiment that still exhibits the failure.
4. Choose a measurement that distinguishes improvement from an attractive anecdote.

A simpler experiment is useful when it preserves the mechanism under investigation. Removing the mechanism merely makes the benchmark easier without answering the original question.

25.2 From manipulation failures to stable policy optimization

Moving from a complicated manipulation system to a pendulum can look like a retreat, but it enables controlled study. The optimal behavior is easier to understand, experiments are faster, and many random seeds reveal whether an algorithm works reliably rather than occasionally.

Trust-region policy optimization addresses a specific weakness: a large policy-gradient update can move beyond the region where the local estimate predicts improvement. A trust region limits this change. Generalized advantage estimation addresses a different weakness, the noise in estimates of how good an action was. PPO then pursues a simpler first-order optimization procedure.

- **TRPO changes the update geometry.** It asks how far the distribution of actions should move.
- **GAE changes the learning signal.** It trades long noisy returns against shorter bootstrapped estimates.

- **PPO changes the practical optimization procedure.** It seeks useful conservative updates with ordinary minibatch optimizers.

Their connection is a sequence of diagnosed bottlenecks, not a claim that one formula solves every difficulty. A promising intuition, such as repeatedly branching from the same state to compare actions, still needs an ablation to determine whether it supplies the actual improvement.

25.2.1 Worked example: choose an informative experiment

Suppose an RL method succeeds on 2 of 20 random seeds, and a modified version succeeds on 12 of 20.

1. The measured success fractions are 0.1 and 0.6. Showing only the best rollout would hide the main difference: reliability.
2. Keep environment, interaction budget, reward, and evaluation rules fixed so the comparison isolates the proposed change.
3. Test the suspected mechanism directly. If the change includes both a new baseline and a new optimizer, compare each separately as well as together.
4. Use additional held-out seeds and report uncertainty before treating the observed difference as a precise population estimate.

A failed experiment can still be progress if it rules out an explanation. An experiment whose result cannot distinguish the competing explanations is less informative even when it requires a large implementation.

25.3 Inverse RL changes what the robot is asked to learn

Demonstrations can communicate an objective rather than merely an action mapping. Feature-expectation matching makes that idea concrete: if rewards are linear in known features, matching the expert's expected feature counts controls return differences for the entire specified reward class.

The research question then shifts from programming the reward to inferring what behavior is trying to accomplish. This can help when a demonstration is imperfect or a new environment requires a different trajectory. It also reveals ambiguity: many rewards can explain the same behavior. Maximum-margin and maximum-entropy formulations resolve ambiguity differently, and interactive human-agent formulations can represent continuing uncertainty about preferences.

A useful contribution explains why this formulation matters, supplies a tractable method, and demonstrates a consequence that action copying or hand-designed reward engineering does not already explain. Generality should follow from the mathematical setup and experiments, rather than from broad language alone.

25.4 Structured prediction uses relationships between outputs

Classifying one web page from its own words ignores its links to other pages. A professor page may link to students, and students may link back. Joint inference can use those relationships when individual text is ambiguous.

The broader idea is to model related outputs together. Discriminative relational models and maximum-margin structured models provide different ways to express and train such dependencies. The lesson for robotics is similar: a set of independent predictions can discard relationships that make the full scene understandable. Joint structure helps when its assumptions reflect real relationships, but can propagate mistakes if the relationships are wrong.

25.4.1 Worked example: relational evidence changes an ambiguous label

Imagine a page containing only a person's name. Its text alone is uninformative about whether the person is a student or professor. If several reliably labeled professor pages list it under "students," those relations provide useful evidence. If the link instead means "collaborator," applying the same rule would be unjustified. A structured model needs the relationship type, not simply the presence of a graph edge.

25.5 InfoGAN: connect a missing property to an objective

Generating convincing images does not automatically produce interpretable latent variables. A generator can represent the same image distribution while entangling shape, color, and identity in arbitrary coordinates. InfoGAN adds an information-based objective to encourage a chosen code c to remain recoverable from the generated image $G(z, c)$.

Mutual information can be written as

$$I(c; X) = H(c) - H(c \mid X).$$

Since the posterior $p(c \mid X)$ is difficult to compute, an auxiliary distribution $Q(c \mid X)$ gives the lower bound

$$I(c; X) \geq H(c) + \mathbb{E}_{c,z}[\log Q(c \mid G(z, c))].$$

The inequality follows because the expected KL divergence from the true posterior to Q is nonnegative. Maximizing the bound encourages the generated image to retain information about the code.

This is an example of turning a desired property into a trainable objective. It does not prove that unsupervised coordinates uniquely match human concepts. A useful empirical result still needs to show which variations the coordinates control and where the interpretation fails.

25.6 Guided policy search connects state-space teachers to vision

Trajectory optimization can exploit a measured training state that a deployed policy will not receive. Guided policy search uses local trajectory-based controllers to produce useful actions, then trains a single policy to reproduce them from its own observations. Joint refinement encourages the local controllers and policy to agree.

The distinction between training and deployment information is essential. A teacher may know precise object pose from an instrumented setup, while the student sees only an image and proprioception. The training pipeline must pair the image with the teacher's action, and evaluation must remove the privileged state from the student's inputs. Otherwise a claim about vision-based control could actually depend on unavailable state information.

1. Fit local dynamics from robot data and optimize state-based controllers for relevant starting conditions.
2. Collect their actions together with the images the final policy will see.
3. Fit a visual policy to those examples and revise local behavior to remain compatible with it.
4. Execute the visual policy on held-out conditions and diagnose whether failures arise from perception, control imitation, or missing state information.

This progression illustrates how a general mathematical method becomes a usable system: model learning, physical execution, and visual representation are integrated rather than treated as automatic consequences of the original optimizer.

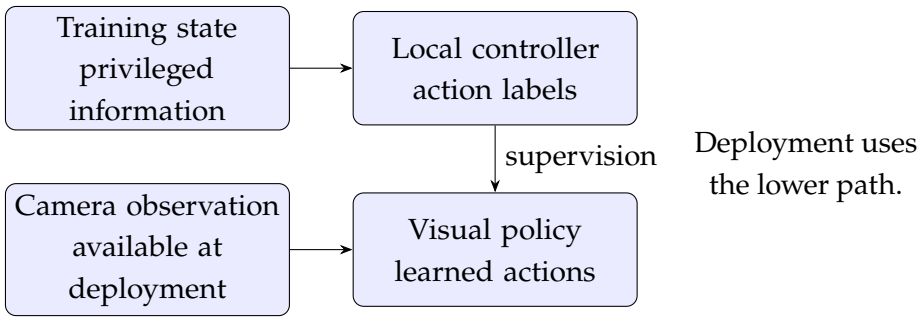


Figure 37: Guided training can use information that the final visual policy does not receive.

25.7 Benchmarking makes competing claims comparable

Different RL papers can report incompatible rankings because they use different environments, networks, tuning effort, random seeds, or stopping budgets. A common benchmark and reference implementations can expose these differences and make later algorithmic claims easier to assess.

A meaningful comparison specifies interaction budget, evaluation episodes, reward conventions, architecture, and hyperparameter selection. Report distributions across seeds rather than only a maximum. Include elapsed time when computational efficiency is part of the claim. A benchmark can reveal that every tested method fails on a long-horizon task, which is useful evidence about the next bottleneck rather than an invitation to hide that task.

Reference implementations also reduce the burden of rediscovering undocumented details. Reproducing a baseline faithfully is part of evaluating a new method; a weak accidental reimplementation can make an unhelpful modification look like progress.

25.8 RL-squared and MAML target faster learning directly

Hierarchical control is attractive because physical tasks have multiple time scales. Motor commands may run frequently, while room selection changes slowly. But obtaining a hierarchy is not itself the final objective; it should improve learning or behavior.

RL^2 reframes the problem as training a recurrent agent that learns rapidly within a new environment. Its hidden state carries information from past actions and rewards. The outer learning process trains this adaptation behavior across environments, without requiring a hand-specified internal hierarchy.

MAML asks for parameters that become useful after a few gradient steps on a new task:

$$\theta_i' = \theta - \alpha \nabla L_i^{\text{train}}(\theta), \qquad \min_\theta \sum_i L_i^{\text{test}}(\theta_i').$$

Separate within-task training and test data make the meta-objective evaluate adaptation rather than memorization of the same examples. The formulation can apply to supervised learning and reinforcement learning because it describes an optimization pattern rather than one fixed network architecture.

25.8.1 Worked example: pretraining and adaptation have different aims

For tasks with scalar targets a_i, let $L_i(\theta) = \frac{1}{2}(\theta - a_i)^2$. One gradient step gives $\theta_i' = (1 - \alpha)\theta + \alpha a_i$.

1. The adapted error is $\theta_i' - a_i = (1 - \alpha)(\theta - a_i)$.
2. The meta-objective averages the error *after* this update, rather than requiring one fixed parameter to solve every task without data.
3. At $\alpha = 1$, one noiseless observation of each exact quadratic task is sufficient to reach its target. With noisy or incomplete data, adaptation is no longer exact, and the initialization and update rule matter.

The simple example clarifies what is optimized without claiming real tasks have this easy structure.

25.9 Shared infrastructure, opportunity cost, and changing direction

Cloud and fleet robotics connect computation with shared experience. Combining data from many robots can improve a common model, while latency-sensitive control still needs an appropriate local execution path. Differences in embodiment, calibration, and operating conditions must remain represented rather than treating every transition as interchangeable.

Research choices also have opportunity costs. Continuing a successful method can address known weaknesses, while moving to a new bottleneck can enable a broader capability. Progress can include more reliable results, faster experiments, or a clearer explanation of why an approach fails. Changing direction is justified by that evidence, rather than by either persistence or novelty as an end in itself.

A strong project ties its claim to an experiment that could have contradicted it, explains which assumption makes the method work, and distinguishes the general idea from the particular demonstration that first motivated it. That connection is what lets another researcher build on the result.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, recorded talk.

Lecture 26: Learning Autonomous Helicopter Maneuvers

26.1 Why aggressive flight is a coupled learning problem

A helicopter is unstable, nonlinear, and affected by airflow, actuator behavior, and coupled motion. Its controller receives noisy estimates of position, orientation, linear velocity, and angular rate. Hover and steady flight occupy a limited regime; aerobatic maneuvers traverse changing regimes and expose errors that a hover model may not reveal.

Two problems arise before tracking can succeed:

1. **Specify a useful target.** A hand-drawn position curve omits orientation, velocity, controls, and dynamic feasibility. Several imperfect demonstrations can reveal an intended maneuver more effectively than one meticulously hand-engineered reference.
2. **Model the relevant dynamics.** Frequency sweeps provide a baseline, but aggressive motion includes effects that repeat along a maneuver and are poorly described by a single global model.

Learning the reference and learning the dynamics are connected. A good controller cannot follow an impossible target, and a good target is insufficient when the model predicts the wrong forces.

26.2 Representing demonstrations as observations of a hidden trajectory

Let $x^*_{0:H}$ denote an intended state trajectory. Demonstration i contains noisy observations $y_{i,k}$ at its own phase or time index. A generative model treats demonstrations as imperfect observations of this common trajectory, while a dynamics prior encourages physically coherent transitions.

A schematic objective is

$$\sum_{i,k} \| y_{i,k} - x^*_{a_i(k)} \|^2_{R_i^{-1}} + \sum_t \| x^*_{t+1} - f(x^*_t, u^*_t) \|^2_{Q^{-1}},$$

where $a_i(k)$ aligns demonstration time with maneuver phase. This expression illustrates observation and dynamics consistency; a complete implementation must also use suitable orientation coordinates, controls, and its chosen statistical model.

Averaging demonstrations at equal clock times can blur a maneuver if pilots perform it at different speeds. Alignment is needed before combining corresponding phases.

26.3 Dynamic time warping aligns progress through a maneuver

For demonstration sequences $y_1, \ldots, y_M$ and $z_1, \ldots, z_N$, choose local mismatch $c(i, j)$. Dynamic time warping (DTW) finds a monotone alignment path using

$$D(i, j) = c(i, j) + \min\{D(i-1, j), D(i, j-1), D(i-1, j-1)\},$$

with boundary initialization and allowed-step rules. Store the minimizing predecessor to recover the alignment.

1. A diagonal step advances both sequences.
2. A horizontal or vertical step advances one while holding the other, allowing one demonstration to spend longer in a phase.
3. Monotonicity preserves event order. It does not allow a later maneuver phase to precede an earlier one.
4. Slope or window restrictions can prevent implausible stretching and reduce computation.

The basic dynamic program costs $O(MN)$ time and full backpointer storage; score-only memory can be reduced to one or two rows. The distance should compare meaningful state components with appropriate scales. Raw angle subtraction across a wrap boundary can falsely report a large mismatch.

26.3.1 Worked example: different speeds, the same phase sequence

Use illustrative scalar progress sequences $y = (0, 1, 2)$ and $z = (0, 0, 1, 2)$ with squared mismatch. A zero-cost alignment pairs y_1 with both first zeros in z, then pairs 1 with 1 and 2 with 2. Naively matching the first three clock indices instead pairs $(0, 0), (1, 0), (2, 1)$ and incurs squared error $0 + 1 + 1 = 2$.

The zero-cost result means these scalar phase sequences can be aligned, not that every physical component of the two flights is identical. Alignment must be checked using the actual state representation.

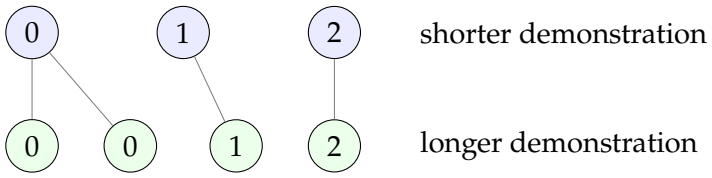


Figure 38: Alignment matches maneuver phases while allowing different execution speeds.

26.4 Smoothing extracts a coherent intended trajectory

After alignment, combine demonstrations with an extended Kalman smoother or a related latent-trajectory inference method. The observation terms average noisy evidence, while the dynamics model links adjacent states. Repeating alignment and trajectory inference can improve their mutual consistency.

26.4.1 Worked example: weighted evidence is not a plain average

Suppose two aligned scalar observations of a trajectory component are 1.0 and 1.4, with variances 0.01 and 0.04. Ignoring the dynamics prior for this local illustration:

1. Their precisions are 100 and 25.
2. The fused mean is $(100 \cdot 1.0 + 25 \cdot 1.4)/125 = 1.08$.
3. The fused variance is $1/125 = 0.008$.
4. The more reliable demonstration receives more weight. A plain average, 1.2, would ignore the stated uncertainty.

A trajectory smoother adds temporal dependencies, so neighboring observations can revise this local estimate. It can reduce noise and reconcile demonstrations, but cannot infer an arbitrary unseen maneuver without a useful model and data.

26.5 Trajectory-specific models exploit repeatable residuals

A baseline model can have large prediction errors during an aerobatic maneuver. If those errors recur at corresponding phases across demonstrations, they contain structure rather than being independent noise. Unmodeled airflow or actuator history may become approximately predictable along repeated execution of the same maneuver.

Fit a local affine model around each phase:

$$x_{t+1} \approx A_t x_t + B_t u_t + c_t.$$

For aligned data indexed by (i, k), use weights that emphasize nearby phase and relevant state similarity:

$$\min_{A_t, B_t, c_t} \sum_{i,k} w_{t,i,k} \| x_{i,k+1} - A_t x_{i,k} - B_t u_{i,k} - c_t \|^2.$$

This is weighted regression. A common illustrative phase kernel is $w_{t,i,k} = \exp[-(a_i(k) - t)^2/(2h^2)]$. Its bandwidth h controls the tradeoff between local relevance and the amount of data.

- **Too local** gives little data and poorly identified coefficients.
- **Too broad** mixes phases with different hidden aerodynamic effects.
- **Regularization or a baseline prior** can stabilize fits when demonstrations do not excite every direction.
- **Off-trajectory execution** can invalidate a phase-local model even when it predicts the demonstrated path well.

The model exploits repeatability rather than claiming to identify every hidden airflow variable. It is therefore especially important to retain feedback and evaluate deviations from the reference.

26.5.1 Worked example: phase dependence explains a residual

Suppose a baseline predicts zero vertical acceleration residual. Across three aligned repetitions at one phase, measured residuals are $1.8, 2.0, 2.2\ \mathrm{m/s^2}$, while a different phase has residuals near -1.

1. The first phase's mean correction is 2.0, with variation around that correction.
2. Pooling both phases into one constant correction could cancel the effects and leave both poorly predicted.
3. A time-local correction represents the repeatable difference, while remaining uncertainty belongs in the noise model.

A corrected nominal prediction does not certify large deviations where the same phase label no longer corresponds to the same physical conditions.

26.6 Feedback control around the learned maneuver

Given a reference and local dynamics, design time-varying feedback using LQR, DDP, or receding-horizon optimization. The controller combines feedforward actions with corrections to state deviations:

$$u_t = u_t^* - K_t(\hat{x}_t - x_t^*),$$

for the LQR sign convention. An estimator supplies $\hat{x}_t$ from inertial and external measurements. In the described experimental setup, onboard inertial sensing and offboard position observations were combined; the resulting flight evidence should be interpreted with those sensing conditions.

Receding-horizon DDP can start from the latest estimate and reoptimize a shorter future trajectory. This compensates for tracking errors and improves predictions along the actual state, subject to model accuracy and computation time. A control sequence optimized from an old state is not equivalent to the newly solved feedback problem.

26.7 An iterative experimental workflow

1. Collect excitation data, such as frequency sweeps, to build a baseline dynamics model.
2. Record several expert demonstrations of the desired maneuver sequence.
3. Align the demonstrations and infer a coherent target trajectory.
4. Fit trajectory-specific local dynamics from aligned data.
5. Optimize a feedback controller for the target and model.
6. Execute autonomously with state estimation and record the resulting transitions.
7. Improve the dynamics model using new execution data, redesign the controller, and repeat as needed.

Each stage resolves a different uncertainty: what behavior is intended, what dynamics apply, what state the vehicle currently occupies, and what corrective action is useful. The procedure combines the course's estimation, learning, and control methods in one physical system.

Aggressive airshows, autorotation, and other specialized maneuvers demonstrate the breadth possible with carefully matched targets, models, and feedback. They do not establish universal recovery from arbitrary disturbances. The scientific value is in explaining why a baseline failed, identifying repeatable structure, and using that structure to produce behavior that can be tested and improved.

Source. Pieter Abbeel, *Advanced Robotics*, Fall 2019, Lecture 26.